

Prototipo 1 - Manual de usuario

Contents

Prototipo 1 — Manual de Usuario	5
Modelador de Recintos 3D con Simulación Acústica FEM	5
Tabla de contenidos	5
1. Introducción	6
¿Qué se puede hacer?	6
Requisitos del sistema	6
2. Inicio rápido	6
3. Interfaz general	6
4. Diseño del recinto — pestaña Geometría	7
Parámetros básicos	7
Tipos de techo	7
Inclinación de paredes	7
Polígono personalizado	7
Modos de visualización	7
5. Controles del visor 3D	8
Navegación de cámara	8
Interacciones acústicas	8
6. Módulo acústico — pestaña Acústica	8
6.1 Fuentes omnidireccionales	8
6.2 Receptor	9
6.3 Materiales por cara (estilo EASE)	9
6.4 Muebles	11
7. Cálculo de modos FEM	14
Parámetros	14
Sugerencia automática de modos y densidad (v2.12)	14
Procedimiento	14
Filtro automático de modos por validez (v2.12)	15
Auto-tuner de densidad (motor = “Automático”, v2.6 — UX actualizada en v2.7)	15
Barra de progreso durante el cálculo (v2.7)	15
Leyenda de tiempo bajo el botón	16
Skip de gmsH para techos curvos paramétricos	16
Dimensionado de la malla: el “peor tet” define la validez	16
8. Visualización del campo acústico	17
8.0 Picker de modos con filtro de frecuencia (v2.12)	17
8.1 Nube de puntos 3D	17
8.2 Plano de corte 2D — flujo interactivo	18
9. Respuesta en Frecuencia (FRF)	19
Cálculo	19
Gráfico	19
Escucha — ruido rosa filtrado por la sala	19
10. RT60 y materiales — comparativa de métodos	20
10.1 Métodos de predicción	20

10.2	Diálogo comparativo	20
10.3	Código de colores	21
10.4	Cómo afecta el amortiguamiento modal	21
10.5	Parches de absorción sub-cara	21
10.6	Cargar tu propio material	22
11.	Flujo de trabajo completo	23
12.	Referencia rápida de atajos	23
13.	Solución de problemas	24
14.	Conceptos físicos clave	24
	Modos acústicos	24
	Sensibilidad del altavoz → caudal volumétrico Q	24
	Amortiguamiento modal y RT60	25
	Frecuencia de Schroeder	25
	Cadena de procesamiento de audio	25
15.	Importar CAD y motor de mallado	25
15.1	Por qué hay dos motores	25
15.2	El badge de estado	26
15.3	Importar un archivo CAD	27
15.4	Diálogo de reparación guiada	27
15.5	Override manual del motor	28
15.6	Formatos soportados	28
15.7	Persistencia en .room v3	29
15.8	Cuándo usa cada motor el router automático	29
15.9	Workflow recomendado para auditorios complejos	30
15.10	El sistema best-effort y el fallback	30
Apéndice — Archivos del módulo CAD/mallado		31
16.	Escala y orientación al importar	32
16.1	Escala — unidades del archivo	32
16.2	Orientación — eje “up” del archivo	32
16.2	Tres botones del diálogo de escala	33
16.3	Optimizaciones del reparador de mallas	33
17.	Indicador de ejes y rotación con eje fijo	33
17.1	Fijar un eje	34
17.2	Comportamiento de la rueda del mouse con eje fijo	34
17.3	Cuándo usar esta función	34
18.	Rendimiento y benchmarks	34
18.1	Cómo correr los benchmarks	34
18.2	Qué mide	35
18.3	Resultados de referencia (Intel/AMD 16 hilos, 64 GB RAM)	35
18.4	Cuellos de botella restantes	35
18.5	Recomendaciones operativas	35
19.	Predicción de geometría — pestaña Predicción	36
19.1	Cuándo usar la pestaña Predicción	36
19.2	Inputs — describir el recinto	36
19.3	Apretar “Predecir” — qué pasa internamente	37
19.4	Las cards — leer los resultados	37
19.5	Cómo se combinan los sub-scores	38
19.6	Aplicar un candidato	38
19.7	Evaluar tu diseño actual	38
19.8	Atajos	39
19.9	Casos de uso típicos	39
20.	Distribución del programa (.exe)	39
	Generar el ejecutable	39
	Flujo completo recomendado	40
	Tamaño	40

Si el .exe no arranca	40
Cómo compartirlo	40
macOS	40
Apéndice — Optimizaciones internas v2.3	41
Evaluación vectorizada del campo (acoustic_fem.FieldEvaluator)	41
Importación CAD con QProgressDialog	41
Apéndice — ¿Por qué FEM “a mano” y no FEniCS?	42
El método no es la librería	42
¿Qué automatiza FEniCS que este proyecto no tiene?	42
¿Por qué entonces no usar FEniCS?	43
¿Cuándo conviene cambiar a FEniCS?	43
La distinción “método vs implementación”, en general	43
En una línea	44
A. Documentación y configuración	46
B. Performance: vectorización del voxel mesher	47
C. UX del FEM: drag de fuentes/receptor + flujo del auto-tuner	47
D. Red de seguridad	48
A. Limpieza profunda — eliminación del solver BEM	48
B. Export de datos numéricos en los tres diálogos de gráfico	48
C. Predicción — rango constructivo de altura	49
D. Fix UX en importación CAD	49
A. Documentación interna — explicadores línea-a-línea y autoevaluación	49
B. Robustez del solver FEM — defensa en profundidad	50
C. Validación	51
D. Lo que no cambió	51
A. Primitivas de planta curva en geometry.py	51
B. Evaluación de FEM P2 — explorada y descartada	52
C. Lo que no cambió en esta versión	53
A. Benchmark modal damping vs matriz C de impedancia	53
B. Fix de calibración: factor c^2 en la fórmula modal	53
C. Lo que no cambió en v2.11	54
A. Picker de modos con filtro y leyenda informativa	54
B. Spinbox Nº modos ampliado y sugerencia Weyl	55
C. Sugerencia automática de n_per_meter (compromiso D4)	55
D. Fix de validez de malla: modos arriba de f_max_malla descartados	55
E. Fixes estéticos en diálogos	56
F. Lo que no cambió en v2.12	56
A. Fuentes reales: respuesta en frecuencia Q(f) + fase (FRD)	56
B. Librería de ratios: Cox + corrección de nombres	56
C. Altura por uso, sin cap duro de 4 m	57
D. Geometría lofteada (cortes laterales)	57
E. Baffle orientado (visual) + gestos 3D	57
F. SBIR (Speaker-Boundary Interference Response)	57
G. Métricas de respuesta forzada: FoM + cruce modal numérico	57
H. Optimizador de ubicación de fuentes	57
I. Criterios de diseño al score de Predicción (respaldo bibliográfico)	57
J. Diagnóstico de corregibilidad por EQ (fase mínima vs no mínima)	58
K. Fix del auto-tuner de malla	58
L. Lo que no cambió en v2.13	58
A. Undo/redo global (Ctrl+Z / Ctrl+Y, 10 acciones)	58
B. Gate de materiales en Predicción	59
C. Fix de color de los diagnósticos de la FRF	59
D. Editor de forma: dimensiones exactas + origen corrido	59
E. Sistema de presets de materiales (Predicción + Acústica)	59
F. Documentación: fix de fondo del auto-tuner de malla	60

A. “Evaluar mi diseño actual” respeta el criterio elegido	60
B. Geometría irregular: evaluación sobre la malla real + elección de ponderación . . .	60
C. Slider de Alto bloqueado con cortes laterales	61
D. Receptor reubicado al interior si queda afuera	61
E. Tooltips legibles	61
A. Origen (0,0,0) configurable — Auto / Centro de planta / Esquina	61
B. Frecuencia de Schroeder: panel arriba + RT desde materiales	62
C. Hover en Materiales → resaltar caras en el 3D	62
D. Carga de mediciones . trf (transfer function binaria)	62
E. Mute por fuente	62
F. Puntos de escucha nombrados	62
G. Botón “Comparar...” — análisis multi-punto	62
H. Marcadores en el heatmap 2D	63
I. Predicción de ubicación con forma irregular: malla real + fuentes adentro	63
J. Estabilidad y diagnóstico	63
A. Parches de absorción sub-carra	63
B. Editor 2D: rectángulos y polígonos, zoom, sin solapes	64
C. Overlay 3D de los parches	64
D. Parches en el diálogo Materiales (listado, edición y resaltado)	64
E. Cargar tu propio material (JSON)	64
A. Muebles como obstáculo rígido (carve)	64
B. Absorción del mueble (A36)	65
C. Reflexión del mueble (SBIR)	65
D. Interfaz: grupo Muebles + editor + wireframe	65
E. Manipulación directa (mismos gestos que las fuentes)	65
F. Colisiones — los objetos sólidos no se atraviesan	65
A. Muebles compuestos (forma física)	65
B. 27 presets en menú agrupado	66
C. Fix de picking por silueta	66
D. Fix de rotación: solo yaw	66
A. Muebles desde archivo CAD	66
B. Gizmo de rotación de 3 ejes	66
C. Tercer eje de rotación (roll)	66
D. Fix: muebles que se trababan al duplicar	67
A. Parches como prisma, con espesor	67
B. El espesor sale del material, y avisa si no coincide	67
C. Fuente y receptor nunca adentro de un mueble	67
D. La regla de sólidos ahora vale en los dos sentidos	67
E. Aviso de mueble tapando un parche	67
F. Fix: muebles y parches se mueven con el origen	68
A. Bundle 490 MB más liviano	68
B. El ZIP dejó de mentir sobre su versión	68
C. Sección §20 nueva + verificador al día	68
D. Fix crítico: el .exe salía sin las DLLs de Qt	68
A. La frecuencia de Schroeder dejó de contradecirse a sí misma	69
B. Densidad de malla hasta 30 y cobertura de modos honesta	69
C. Polaridad de fuentes (0° / 180°)	69
D. Amortiguamiento por perturbación de frontera (modelo nuevo, seleccionable) . . .	69
E. Auditoría de la simulación: dos números que estaban mal en salas no rectangulares	70
F. Correcciones menores	70
RT60 efectivo: T30 del decaimiento modal, no la media de las tasas	70
Qué cambia en pantalla (solo con el modelo de perturbación activo)	71
Frecuencia de Schroeder de dos pasadas (Etapa 2b)	71
Predicción de ubicación con amortiguamiento por modo (Etapa 2c)	71
La perturbación es ahora el modelo por defecto (Etapa 3)	71

La absorción es un dato consciente, no un default silencioso (Opción C)	71
Nº de modos necesario para cubrir hasta $f_{\text{Schroeder}}$	72
Modo Rotar: girar la vista y los objetos sin botón central	72
Delay y fase de fuentes: se aplican al Aceptar y se recuerdan	72
Arranque al correr desde código en macOS	72
SBIR con la transferencia modal de la sala	73
Guardar y cargar curvas de RT para comparar configuraciones	73
Construcciones de pared: modelar la impedancia de cada superficie	73
Cada modo, con su frecuencia corrida y su amortiguamiento a la vista	74
Un acabado acústico por superficie	74
Crear un material propio sin escribir JSON	75
El npm que escribís se respeta	75
Cargar la respuesta de un parlante en formato CLF	75
Un filtro por fuente (crossover / EQ)	75
Todos los diálogos en fondo blanco, letra negra	76
La absorción elegida se comparte entre Acústica y Predicción	76
Subs enfrentados (DBA / CABS)	76
Modelo físico de driver (Thiele-Small)	77
Lector CLF generalizado	77
Impedancia por default en cada material	77

Prototipo 1 — Manual de Usuario

Modelador de Recintos 3D con Simulación Acústica FEM

Tabla de contenidos

1. Introducción
2. Inicio rápido
3. Interfaz general
4. Diseño del recinto — pestaña Geometría
5. Controles del visor 3D
6. Módulo acústico — pestaña Acústica
7. Cálculo de modos FEM
8. Visualización del campo acústico
9. Respuesta en Frecuencia (FRF)
10. RT60 y materiales
11. Flujo de trabajo completo
12. Referencia rápida de atajos
13. Solución de problemas
14. Conceptos físicos clave
15. Importar CAD y motor de mallado
16. Escala y orientación al importar
17. Indicador de ejes y rotación con eje fijo
18. Rendimiento y benchmarks
19. Predicción de geometría — pestaña Predicción
20. Distribución del programa (.exe)

1. Introducción

Prototipo 1 es una herramienta de escritorio para diseñar recintos acústicos en tres dimensiones, visualizarlos interactivamente y calcular su comportamiento modal usando el Método de Elementos Finitos (FEM).

¿Qué se puede hacer?

- Modelar recintos con geometría arbitraria: polígonos de N lados, techos planos, de arco, a dos aguas o inclinados.
- Visualizar los **modos acústicos** del recinto en 3D (nube de puntos coloreada) y en cortes 2D (mapas de calor).
- Posicionar **fuentes** y un **receptor**, configurar las fuentes por su sensibilidad en dB (como en una ficha técnica de altavoz).
- Asignar **materiales acústicos** (hormigón, yeso, alfombra, etc.) a piso, techo y paredes, y calcular el RT60 y el amortiguamiento modal resultante.
- Obtener la **FRF** (Respuesta en Frecuencia) en el receptor.
- **Escuchar** cómo afecta la sala al sonido usando ruido rosa filtrado con la FRF.

Requisitos del sistema

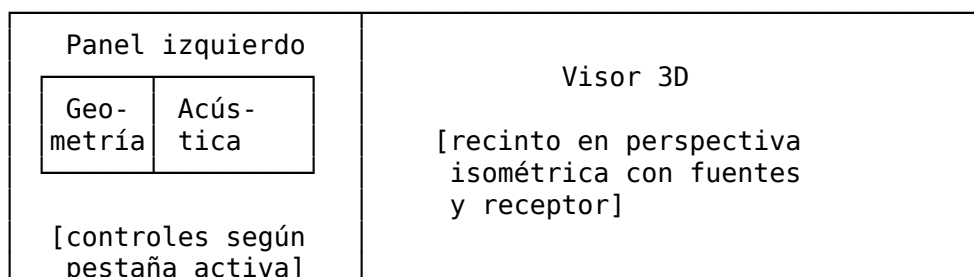
Sistema operativo	Windows 10 / 11 (64 bits)
Python	3.12 (Anaconda)
Dependencias extra	<code>pip install pyqtgraph PyOpenGL matplotlib</code>
Audio	winsound (incluido en Python) + scipy (incluido en Anaconda)

2. Inicio rápido

1. Abrir `run.bat` (doble clic). La aplicación arranca con un recinto rectangular de $6 \times 8 \times 3$ m.
2. En la pestaña **Geometría**, ajustar los sliders de dimensiones y forma.
3. Cambiar a la pestaña **Acústica**. Colocar una fuente con Ctrl + clic derecho en el visor 3D.
4. Presionar **Calcular modos (FEM)**.
5. Presionar **Actualizar campo 3D** (o Enter) para ver la distribución modal.
6. Presionar **Calcular FRF** y luego **Escuchar** para oír la sala.

3. Interfaz general

La ventana principal tiene dos áreas:



Volumen / Superficie / Vértices

4. Diseño del recinto — pestaña Geometría

Parámetros básicos

Parámetro	Rango	Descripción
Ancho / Largo / Alto	0.5 - 50 m	Dimensiones del paralelepípedo base
Nº de lados	3 - 32	Número de lados del polígono de planta
Afinamiento	0 - 1	Estrechamiento del techo respecto a la planta
Giro	0° - 360°	Rotación del techo sobre su eje vertical

Consejo: Doble clic sobre el *valor numérico* del slider para ingresar un número exacto con el teclado. Doble clic sobre el *slider mismo* lo resetea a cero.

Tipos de techo

Tipo	Descripción
Plano	Techo horizontal (default)
Arco	Bóveda de cañón; el slider “Altura de arco” controla la curvatura
Dos aguas	Techo con caballete; “Offset del caballete” lo descentra
Inclinado	Un solo plano inclinado (shed)

Inclinación de paredes

Clic derecho + arrastrar sobre una pared en el visor 3D inclina esa pared de forma interactiva.

Polígono personalizado

El botón **Dibujar forma** abre un editor de planta 2D:

- **Clic izquierdo** sobre una arista → inserta un vértice.
- **Clic derecho** sobre un vértice → lo elimina.
- El selector de grilla ajusta el paso de cuadrícula (0.25 m - 5 m).

Modos de visualización

Modo	Descripción
Aristas	Mesh traslúcido + aristas visibles (default)
Externa	Mesh opaco, sin aristas
Contorno	Solo aristas

5. Controles del visor 3D

Navegación de cámara

Acción	Efecto
Botón central + arrastrar	Órbita libre
Shift + botón central + arrastrar	Órbita horizontal (solo yaw)
Botón derecho + arrastrar	Paneo
Rueda del mouse	Zoom
0	Reset a vista isométrica

Interacciones acústicas

Acción	Efecto
Ctrl + clic derecho	Coloca una fuente en el punto del piso bajo el cursor (sensibilidad default: 90 dB)
Shift + clic izquierdo + arrastrar	Mueve la fuente o receptor más cercano en su plano horizontal (z constante)
Ctrl + Shift + clic izquierdo + arrastrar	Mueve la fuente o receptor más cercano solo en altura (x, y fijos en la posición original). Útil para subir o bajar sin desplazar horizontalmente. <i>(nuevo en v2.7)</i>
Doble clic sobre una fuente	Abre el diálogo de edición de esa fuente

6. Módulo acústico — pestaña Acústica

6.1 Fuentes omnidireccionales

Cada fuente es un monopolo acústico puntual. Se puede colocar: - Con Ctrl + clic derecho en el visor 3D. - Con el botón **Añadir** del panel.

Diálogo de edición de fuente

Fuente acústica

Etiqueta: src_0

Posición (m): X: 1.00 Y: 1.00 Z: 1.00

Sensibilidad (1W / 1m): 90.0 dB SPL

dB SPL medido a 1 W de potencia eléctrica
y 1 m de distancia (ficha técnica del altavoz).

→ Q equivalente: $|Q| = 1.045e-03 \text{ m}^3/\text{s}$
(monopolo @ 1000 Hz, 1 W)

[OK] [Cancelar]

Sensibilidad es el único parámetro de intensidad. Ingresar el valor de la ficha técnica del altavoz: nivel de presión sonora a 1 W de potencia eléctrica y 1 m de distancia, en dB SPL.

Tipo de altavoz	Sensibilidad típica
Subwoofer	85 - 90 dB/W/m
Woofers / full-range	88 - 96 dB/W/m
Tweeter de compresión	100 - 110 dB/W/m
Altavoz de línea (line array)	105 - 115 dB/W/m

El software convierte la sensibilidad a caudal volumétrico Q usando el modelo de monopolo:

$$p_0 = 20 \mu\text{Pa} \cdot 10^{(S/20)}$$

$$|Q| = p_0 \cdot 4\pi / (2\pi \cdot 1000 \text{ Hz} \cdot \rho_0)$$

Nota: Un incremento de +10 dB en la sensibilidad multiplica |Q| por $\sqrt{10} \approx 3.16$, coherente con la relación entre potencia y presión acústica.

6.2 Receptor

El receptor es el punto donde se evalúa la FRF. Se puede mover: - Con los spinboxes X/Y/Z del panel. - Con Shift + arrastrar sobre la cruz cian en el visor 3D.

6.3 Materiales por cara (estilo EASE)

A partir de la versión 2.3, la asignación de materiales **no se hace por zona** (piso/techo/paredes) sino **por grupo de caras planares**, igual que en EASE. El recinto se descompone automáticamente en regiones planares conexas y cada una recibe su propio material.

El panel de Acústica muestra ahora un botón único en lugar de los tres combos:

Materiales de superficie

[Materiales...]

6 grupos · 6 con material (Piso 48 ·
Techo 48 · Paredes 84 m²)

RT60 medio: 1.24 s · @500 Hz: 1.18 s
(V=144 m³)

[Ver RT60 calculado] [Recargar materiales]

Cómo abre el diálogo Apretar **Materiales...** abre una ventana modal con una tabla:

Materiales por cara

Asigna un material a cada grupo de caras. Los grupos se detectan automáticamente por orientación y conectividad. Las asignaciones se guardan al cerrar el dialogo y se restauran al abrirla de nuevo.

Grupo	Caras	Área m ²	Categoría	Material
Piso	2	48.00	Piso	Madera ▼
Techo	2	48.00	Techo	Yeso ▼
Pared 1 (-X (W))	2	24.00	Pared	Hormigón ▼
Pared 2 (+X (E))	2	24.00	Pared	Hormigón ▼
Pared 3 (+Y (N))	2	18.00	Pared	Alfombra ▼
Pared 4 (-Y (S))	2	18.00	Pared	Vidrio ▼

Resumen		
Áreas por categoría: Piso: 48.0 m ² · Techo: 48.0 m ² · ...		
RT60 medio (500 Hz): 1.24 s · medio 1.18 s · 6/6 grupos		
[Asignar a todos...]	[Preset piso/techo/paredes...]	[Recalcular]
[OK]	[Cancel]	[Apply]

Cómo se detectan los grupos El agrupador(`face_materials.group_faces_by_planar_region`) hace dos pasos:

1. **Cluster greedy por normal de la cara:** todas las caras cuya normal queda dentro de $\pm 15^\circ$ de un mismo eje quedan en el mismo cluster.
2. **Componentes conexas por cluster:** dentro de cada cluster se separan los grupos no conexos (dos paredes paralelas son dos grupos distintos).

Cada grupo recibe automáticamente una etiqueta legible:

- **Piso / Techo** si la normal apunta hacia abajo/arriba ($|n_z| > 0,85$).
- **Pared N (+X, NE, +Y, NW, -X, SW, -Y, SE)** para paredes verticales, indicando la dirección cardinal aproximada.
- **Cara inclinada N (...)** para superficies oblicuas (tribuna, plafón inclinado).

Para una sala $6 \times 8 \times 3$ m: 6 grupos (piso, techo, 4 paredes). Para un pentágono regular: 7. Para un auditorio CAD importado puede haber **decenas** de grupos.

Persistencia automática

- Mientras la app esté abierta, las asignaciones quedan en memoria: cerrar el diálogo y abrirlo de nuevo muestra exactamente las mismas selecciones.
- Al **guardar** el `.room`, las asignaciones se serializan en `acoustic.face_materials.assignments` como `{signature: material_name}`, donde `signature` es un hash estable de (normal, centroide, área) redondeados — sobrevive recompilaciones del agrupador y cambios menores de geometría.
- Si cargás un `.room` viejo (v2 o v3) en el cual no había asignaciones, todos los grupos arrancan con un material por defecto rígido ($\alpha \approx 0,03$).

Botones de acción rápida

- **Asignar a todos...** aplica el material elegido a todos los grupos a la vez (útil como punto de partida).
- **Preset piso/techo/paredes...** replica el esquema clásico: pide 3 materiales (uno por categoría) y los aplica automáticamente.
- **Recalcular RT60** fuerza el cómputo de RT60 con la asignación actual.

Cambio respecto al esquema anterior

Antes (v2.2)	Ahora (v2.3)
3 combos en el panel: Piso / Techo / Paredes	1 botón “Materiales...” que abre un diálogo
Un material por zona, clasificación por normal y altura	Un material por grupo de caras coplanares conexas
Las paredes paralelas comparten material a la fuerza	Cada pared (y cada cara inclinada) tiene su propio material

Antes (v2.2)	Ahora (v2.3)
No funciona bien para auditorios complejos	Funciona para cualquier geometría (paramétrica o CAD)
RT60 con 3 categorías de área	RT60 con N términos (uno por grupo)

Materiales incluidos

Material	α a 500 Hz	Uso típico
Hormigón visto	0.02	Muros crudos
Yeso pintado	0.03	Paredes y techos revocados
Ladrillo visto	0.04	Mampostería sin revocar
Madera dura	0.05	Pisos y paneles
Vidrio	0.03	Ventanas y mamparas
Alfombra fina	0.20	Alfombra de pelo corto
Alfombra gruesa	0.40	Alfombra de lana con subpiso
Panel acústico	0.70	Espuma de melamina / lana mineral

Agregar materiales propios Copiar un archivo .json a la carpeta materials/ y presionar **Recargar materiales**.

Formato del archivo:

```
[{
  "name": "Mi material",
  "category": "Porosos",
  "absorption_coef": [0.05, 0.10, 0.20, 0.40, 0.60, 0.70, 0.75, 0.75],
  "scatter_coef": [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.35, 0.35]
}]
```

Los 8 valores corresponden a las bandas de octava: **63 / 125 / 250 / 500 / 1000 / 2000 / 4000 / 8000 Hz**.

6.4 Muebles

Un **mueble** es un objeto sólido (caja, cilindro, preset armado o **mallla CAD importada**) que entra en el modelo modal como un **obstáculo real**. No es un adorno visual: afecta la física por **tres canales**:

1. **Obstáculo rígido (carve)**. La mallla del aire se **talla**: los tetraedros dentro del mueble se quitan del dominio. La superficie del hueco queda como pared rígida (condición natural, gratis). Resultado: los **modos se corren** por sí solos (exacto, no perturbativo).
2. **Absorción (A36)**. Si al mueble le asignás un **material**, sus caras absorben según la presión modal sobre ellas — igual que las paredes. Un sillón tapizado domina por **absorción**, no por desplazar volumen; modelarlo rígido es cualitativamente errado.
3. **Reflexión (SBIR)**. La cara superior del mueble (el sobre del escritorio, el respaldo del sofá) rebota con **rolloff de panel finito** (difracción de borde), y aparece en el diálogo SBIR.

El efecto se aplica al **recalcular los modos**.

Agregar y editar En el grupo **Muebles** (pestaña Acústica): **Añadir / Editar / Quitar / Duplicar**. El editor pide:

Campo	Detalle
Tipo	Caja o Cilindro
Centro (X, Y, Z)	posición del centro geométrico [m]; el mueble nuevo aparece en el medio de la sala
Tamaño	caja = Ancho × Largo × Alto; cilindro = Diámetro × Alto
Orientación (yaw)	giro alrededor del eje vertical (solo caja)
Inclinación (pitch)	inclinación adelante/atrás (solo caja); afecta el carve , no es solo visual
Vuelco (roll)	vuelca el mueble de costado (gira sobre su frente); también afecta el carve
Material	del catálogo, o Rígido (sin absorción)
Etiqueta / Procedencia CAD	nombre y trazabilidad de las medidas botón “Importar CAD (OBJ)...” : trae una malla 3D como mueble (§6.4.1)

Los tres ángulos siguen la convención de aviación: primero **yaw** (sobre el eje vertical del mundo), después **pitch** (sobre el eje transversal resultante), después **roll** (sobre el frente resultante).

El mueble se ve como un **wireframe verde-azulado** en el visor 3D (naranja al seleccionarlo en la lista). Se guarda en el .room junto con su material.

Mover y rotar en el visor 3D

Gesto	Acción
Shift + arrastrar	mover en el plano horizontal (XY)
Ctrl + Shift + arrastrar	mover en altura (Z)
Alt + Ctrl (mantener)	aparece el gizmo de rotación (3 anillos)
Alt + Ctrl + arrastrar	girar sobre el anillo agarrado
Doble-click	abrir el editor

Gizmo de rotación. Manteniendo **Alt + Ctrl** sobre un mueble aparecen tres anillos, uno por eje. El que está bajo el cursor se **resalta en magenta**: ese es el que se va a mover. Se hace click y se arrastra, y el mueble gira **solo sobre ese eje**.

Anillo	Color	Eje	Efecto
Yaw	celeste	vertical del mundo	gira sobre el piso
Pitch	ámbar	transversal local	inclina adelante/atrás
Roll	verde	frente local	vuelca de costado

Las **fuentes tienen prioridad** de selección. El mueble se agarra clickeando en **cualquier parte de su silueta**, no solo cerca del centro. El eje se elige **antes** de mover (con el click sobre el anillo), que es lo que evita inclinar sin querer al rotar. Los tres ángulos también se editan por sus campos numéricos en el diálogo.

Objetos sólidos — no se superponen Un mueble **no puede** ocupar el mismo espacio que otro mueble, que el **baffle de un parlante**, ni salirse de las **paredes o el techo del recinto**. Al arrastrarlo, **frena** al tocar el obstáculo; al Añadir/Editar con una posición inválida, avisa el motivo y no lo agrega. El **piso no atrapa** al mueble (se apoya ahí). Del mismo modo, **las fuentes y el receptor se traban en las paredes** del recinto al arrastrarlos (se deslizan pegados al límite en vez de salirse).

La regla vale en **los dos sentidos**: el parlante también frena contra los muebles (antes solo frenaba el mueble contra el parlante). Y hay una regla más estricta para los **puntos**: ni la fuente ni el receptor pueden quedar **adentro** de un mueble, ni arrastrándolos ahí ni poniéndoles un mueble encima.

Por qué esa regla es obligatoria. El mueble se modela quitando el aire que ocupa (se talla la malla). Si el punto de la fuente o del receptor queda dentro de ese hueco, ahí **no hay malla**: el campo modal evalúa **NaN** y eso contamina toda la FRF sin dar ningún error. Ojo con la diferencia: el parlante se **dibuja** como una caja pero para la física es un **punto**; el mueble sí tiene volumen real en el modelo.

Si algo quedó superpuesto (por ejemplo al agregar un mueble encima), **se puede arrastrar para afuera**: la regla solo frena los movimientos que *crean* una superposición nueva, nunca deja un objeto atrapado sin salida.

Presets armados El botón “**Insertar preset ▼**” ofrece muebles ya dimensionados, agrupados en **General**, **Aula** y **Estudio / tratamiento**. Cada uno se inserta en el centro de la sala (las **nubes acústicas** colgadas del techo) con forma reconocible y un material sugerido, y después se mueve/rota/edita como cualquier mueble.

- **General**: silla, sillón, escritorio, mesa, banqueta, velador, biblioteca.
- **Aula**: pupitre, silla escolar, escritorio docente, mesa grupal, pizarrón, armario, estantería abierta, casilleros, carrito de dispositivos, taburete.
- **Estudio / tratamiento**: gobo, bass trap de esquina, difusor QRD/Skyline, resonador Helmholtz, nube acústica, console desk, soporte de monitor, rack 19”, sofá de control, silla de mezcla.

Alcance del modelo. El FEM es de baja frecuencia (modal): capta el obstáculo, la absorción del material y la reflexión del tope. **No** simula la **difusión** (QRD/Skyline) ni la **intonía** fina de resonadores Helmholtz o bass traps; esos presets entran como geometría más un material aproximado. Los **absorbentes de banda ancha** (sofá, gobos, nubes, paneles) sí se modelan bien, que es la fortaleza del modelo en LF.

Internamente un preset es un mueble **compuesto** (unión de sub-piezas): se talla, mueve y choca como una sola pieza. Las partes finas (patas, tensores) no se resuelven en la malla, lo cual es correcto.

6.4.1 Importar un mueble desde CAD (OBJ) El botón “**Importar CAD (OBJ)...**” del editor trae una **malla 3D** como mueble. Formatos: .obj, .stl, .ply, .off, .glb, .gltf.

El caso de uso pensado es **capturar un recinto real**: escanear el estudio con el celular, abrirlo en SketchUp, separar cada pieza y exportarla por separado. La malla importada se talla, absorbe y refleja exactamente igual que un preset; se mueve, rota y choca como cualquier otro mueble.

Al importar, el mueble aparece en el centro de la sala apoyado en el piso, y el diálogo muestra la cantidad de caras. El **tipo y el tamaño quedan bloqueados** (la forma la define la malla); se editan posición, los tres ángulos, material, etiqueta y procedencia. La malla se guarda **embebida en el .room**, así el archivo sigue siendo autocontenido.

La malla tiene que ser cerrada (watertight). El tallado pregunta “¿este punto está adentro?”, y esa pregunta solo tiene respuesta confiable si la superficie está cerrada. Al importar se intenta **reparar** automáticamente (unir vértices duplicados, tapar huecos, corregir normales); si aun así queda abierta, el programa **avisa** y conviene cerrar los huecos en el CAD antes de confiar en el resultado.

Sobre la fidelidad de la forma. Un experimento controlado mostró que, en la banda modal, la forma exacta de un mueble frente a su caja envolvente es un efecto de **segundo orden** (0,3 % a 1,4 % en las frecuencias, del orden del ruido de malla), y que depende mucho más de **dónde** está el mueble (el máximo se da en las esquinas, que

son antinodos). Importar CAD sirve para **capturar** un recinto real con comodidad, no para ganar precisión de predicción.

7. Cálculo de modos FEM

Parámetros

Parámetro	Descripción
Nº de modos	Cantidad de modos a calcular (2 - 500, default 12). Subido en v2.12 desde el cap viejo de 80 para permitir cobertura completa hasta $f_{Schroeder}$ en salas chicas con f_S alta
Densidad de malla	Nodos por metro para la malla tetraédrica interna ($0.5 - 10 \text{ m}^{-1}$). Mayor densidad = mayor precisión, mayor tiempo

Atención: La **densidad de malla** controla la precisión del cálculo FEM, no la cantidad de puntos visibles en la nube 3D. Para más puntos de visualización, usar el spinner **Resolución campo 3D**.

Sugerencia automática de modos y densidad (v2.12)

Si apretás primero **Calcular $f_{Schroeder}$** (botón más abajo en el panel), tres labels se actualizan en simultáneo:

1. **$f_{Schroeder} \approx XX \text{ Hz}$** — el techo del régimen modal de tu sala.
2. **$\approx N$ modos hasta f_S (Weyl)** — estimación de cuántos modos necesitás pedir para cubrir hasta f_S , por ley de Weyl $N(f) \approx (4\pi/3) \cdot V \cdot f^3/c^3 + (\pi/4) \cdot S \cdot f^2/c^2$. Si tu N° modos actual es menor que ese N , no vas a llegar a f_S ; si es mayor, los modos extra van a quedar arriba de f_S (no son inválidos, sólo están fuera del régimen modal puro).
3. **npm sugerido: X.XX** debajo del slider Densidad voxel, con un botón **[Aplicar]** al lado. Ese npm está calculado como $\text{npm} = \text{ppw} \cdot f_S / c$ (con $\text{ppw}=6$), o sea: el valor exacto que hace que $f_{\text{max_malla}} = f_S$. Un click carga ese valor al spinbox.

El compromiso es deliberado (decisión D4): el slider sigue siendo editable porque a veces querés preview rápido con malla gruesa, y otras veces análisis riguroso con malla fina. La sugerencia te ahorra el cálculo mental sin sacarte la palanca.

Procedimiento

1. Verificar que hay al menos una fuente posicionada dentro del recinto.
2. (Opcional, recomendado) **Calcular $f_{Schroeder}$** , mirar la sugerencia Weyl, ajustar N° modos y aplicar npm sugerido si querés cobertura rigurosa.
3. Presionar **Calcular modos (FEM)**.
4. Esperar (segundos a 1-2 minutos según tamaño y densidad).
5. El panel muestra: número de nodos/tetraedros, volumen y frecuencia máxima de validez.

La **frecuencia máxima de validez** del modelo es:

$$f_{\text{max}} = c / (6 \cdot h_{\text{max}}) \quad \text{donde } c = 343 \text{ m/s}$$

Filtro automático de modos por validez (v2.12)

solve_modos devuelve siempre los N modos más bajos en frecuencia, sin conocer el techo de validez de la malla. Si N es alto (cosa frecuente con la sugerencia Weyl), los últimos modos pueden caer **arriba de f_max_malla** — frecuencia donde la malla ya no resuelve la longitud de onda y los resultados se ensucian numéricamente (dispersión del esquema, plegado de onda).

Desde v2.12 el panel **descarta automáticamente** esos modos sospechosos. En el log vas a ver, por ejemplo:

FEM: pediste 256 modos, 210 son válidos. 46 excedían
f_max_malla = 59 Hz (descartados por dispersión numérica del esquema).

Y la leyenda del picker (ver §8) nunca va a mostrarte modos por encima del techo de validez. La verificación visual es consistente: si el badge dice “*válido hasta 143 Hz*”, todos los modos en el picker están por debajo de 143 Hz.

Nota — Frecuencia de Schroeder: Presionar **Calcular f_Schroeder** para ver la frecuencia límite del régimen modal. Por debajo de ella los modos son discretos (FEM es exacto); por encima el campo es estadístico.

Auto-tuner de densidad (motor = “Automático”, v2.6 — UX actualizada en v2.7)

Cuando el combo **Motor de mallado** está en “Automático”, al apretar **Calcular modos (FEM)** el soft ya no usa los valores manuales de Densidad voxel / h_gmsh. En cambio:

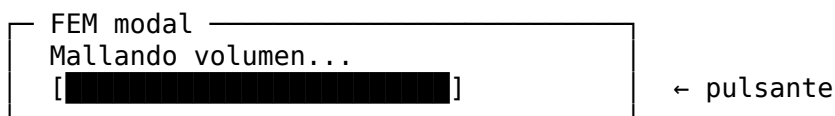
1. Calcula V y f_Schroeder del recinto actual.
2. Elige la densidad necesaria para cubrir **siempre** hasta f_Schroeder (cobertura completa garantizada).
3. Corre el FEM y reporta “válido hasta X Hz” debajo del botón.

Política de v2.7: validez antes que velocidad, sin preguntar. A partir de v2.7 el auto-tuner siempre apunta a cobertura completa hasta f_Schroeder. Ya no hay diálogo modal “Cobertura parcial / Cobertura completa / Cancelar” — la app decide sola y vos ves una barra de progreso si el cálculo tarda lo suficiente como para necesitarla.

Los spinboxes Densidad voxel y h_gmsh se actualizan automáticamente con los valores que el auto-tuner usó, así ves qué decidió. Si querés densidades específicas, cambiá el combo a “Voxel” o “Gmsh” — esos modos sí respetan los spinboxes manuales.

Barra de progreso durante el cálculo (v2.7)

Si el FEM tarda más de **200 ms**, aparece un diálogo modal con una barra pulsante y una etiqueta que cambia por fase:



Las fases visibles son: 1. Mallando volumen... 2. Ensamblando K, M (N nodos)... 3. Resolviendo X modos (Lanczos)... 4. Post-procesando...

El diálogo **no tiene botón Cancelar** porque eigsh no es interrumpible — preferimos ser honestos antes que mostrar un botón que no hace nada. La barra es pulsante (no porcentual) porque el tiempo por fase varía mucho con la geometría.

Para cálculos muy rápidos (< 200 ms en shoebox simple post-vectorización G), el diálogo nunca aparece — evita el flash visual molesto.

Leyenda de tiempo bajo el botón

Después de cada Calcular modos (FEM), debajo del botón aparece una leyenda persistente del tipo:

Último: 0,18 s · válido hasta 261 Hz

El número aparece verde clarito durante 1,5 s después del cálculo y fade a gris pasivo después. Lo mismo se ve debajo de **Predecir**, **Importar CAD**, y **Aplicar ▼** de las cards de Predicción.

Cambio en v2.7: el cronómetro ahora detiene **después** de todo el post-procesamiento visible (refresh del combo de modos, cálculo de ξ por material, actualización del slice 2D). Antes detenía solo después del FEM, lo que dejaba al usuario viendo Último: 0,1 s mientras el slice todavía se redibujaba. Ahora el número coincide con “click → resultado completo en pantalla”.

Cobertura parcial removida: en v2.6 aparecía a veces parcial · 180/344 Hz cuando el usuario elegía cobertura parcial en el diálogo de budget. En v2.7 ese path ya no existe — la leyenda siempre dice válido hasta X Hz sin ambigüedad.

Skip de gmsh para techos curvos paramétricos

Si la sala paramétrica tiene arch_height > 0 con techo en arco o dos aguas, el router **NO intenta gmsh**: la malla de visualización tiene T-junctions en el borde techo-pared que rompen gmsh con PLC Error. Va directo a voxel (que no le importa la topología del surface mesh — usa tests inside/outside con la grilla). Esto evita 1-2 s de “intentar y caer” + mensaje de error confuso.

Para tener gmsh con techo curvo necesitaríamos subdividir las paredes para que matcheen el techo — pendiente para v2.7.

Dimensionado de la malla: el “peor tet” define la validez

La validez de la malla la fija $f_{\max} = c / (6 \cdot h_{\max})$, donde $h_{\max}$ es el **tetraedro más grande** de toda la malla — no el promedio. Esto importa para el auto-tuner, porque cada motor entrega ese $h_{\max}$ distinto a partir del tamaño objetivo h_{target} que se le pide:

- **Voxel:** celda uniforme → $h_{\max} = 1/\text{npm}$ **exacto**. Lo que pedís es lo que obtenés.
- **Gmsh:** recibe MeshSizeMax = h_{target} pero **no lo respeta en el peor elemento**: el tet más grande sale $h_{\max} \approx 1.5 \cdot h_{\text{target}}$ (medido 1.42-1.51, estable y escala-invariante). Si se asumiera $h_{\max} = h_{\text{target}}$, gmsh **sub-entregaría validez**: pedís cobertura hasta f_S y la malla real solo llega a $f_S/1.5$.

Por eso, en modo **Automático** el tuner le pide a gmsh una malla **1.5× más fina** que la nominal ($h_{\text{target}} = c / (6 \cdot 1.5 \cdot f_S)$, constante `_GMSH_HMAX_OVER_HTARGET` en `mesh_router.py`), de modo que la validez **real** (la del peor tet) alcance siempre f_S . El voxel no necesita esa corrección.

Consecuencia que vas a ver: con gmsh el badge “válido hasta X Hz” puede dar un poco **por encima** de f_S (p. ej. 168 Hz con $f_S = 148$). Es correcto y del lado seguro: el 1.5 es el **peor caso** medido; cuando gmsh entrega tets mejores que ese peor caso, la malla valida un poco más alto. Lo que nunca debe pasar es que valide **por debajo** de f_S (eso era el bug).

Nota de fondo (corregido en el ciclo v2.13-v2.14): además del factor del peor tet, el auto-tuner tenía un segundo problema. El panel lo invoca con presupuesto de tiempo infinito (budget = ∞ , política “validez antes que velocidad” de v2.7), y con ese infinito la estimación de tetraedros desbordaba y el panel caía **en silencio** a la densidad manual del slider — el tuner nunca corría. Hoy un guard (`np.isfinite`) lo evita y la cobertura parcial se clampea a la completa. Si una sala lofteada te daba pocos modos válidos (la validez quedaba muy por debajo de f_S), era esto.

8. Visualización del campo acústico

8.0 Picker de modos con filtro de frecuencia (v2.12)

Antes de visualizar un campo elegís qué modo mirar con el combo **Modo:**. Con sets grandes (50, 100, 300 modos) buscar uno específico era incómodo. Desde v2.12 hay dos controles arriba del combo:

Control	Qué hace
f_min visible [Hz]	Oculto del picker los modos con frecuencia por debajo de este valor
f_max visible [Hz]	Oculto del picker los modos con frecuencia por encima de este valor

El filtro **sólo afecta la visualización**: no recalcula nada, no descarta modos de la FRF ni del cálculo de RT60. Si querés ver los modos entre 60 y 100 Hz de un set de 200, lo lográs sin perder los otros 190 para los otros usos.

Debajo del combo aparece una leyenda con el conteo total y el filtrado:

Total: 50 modos calculados (rango 10.5 – 128.5 Hz) · mostrando 23 en [100.0, 270.0] Hz
– el filtro pasa por encima del rango calculado

La segunda línea aparece sólo cuando **f_max visible** excede el último modo computado — te avisa que no hay modos “ocultos” arriba, que el set simplemente no llega. Para verlos tendrías que recalcular con más N^o modos, no con un filtro más amplio.

8.1 Nube de puntos 3D

Luego de calcular los modos, presionar **Actualizar campo 3D** (o Enter estando en la pestaña Acústica).

El selector **Campo** cambia lo que se muestra:

Opción	Descripción
Forma modal	Muestra la forma de vibración $\phi_n(x)$ del modo seleccionado. No depende de la posición de la fuente. Colormap: azul = fase negativa, blanco = nodo, rojo = fase positiva
Presión p 	Muestra la amplitud $ p(x, f_n) $ generada por las fuentes. Cambia al mover la fuente (actualización automática a los 350 ms). Colormap: azul = mínimo, rojo = máximo

Nota: En la frecuencia de resonancia la distribución de $|p|$ se parece a la forma modal porque el modo dominante escala el patrón. La diferencia se nota al mover la fuente hacia un *nodo* del modo: $\phi_n(x_s) = 0$, la excitación cae a cero y el patrón cambia completamente.

Colormaps de la nube 3D (v2.6) A partir de v2.6 los colormaps fueron reemplazados por dos paletas que se ven mejor a alta resolución:

Modo con fuente — Rainbow 7 paradas (colormap_rainbow):

t	Color	RGB
0,000	Azul	(12, 25, 242)
0,167	Celeste	(25, 165, 249)
0,333	Turquesa	(25, 234, 191)
0,500	Verde claro (centro)	(76, 234, 63)
0,667	Amarillo	(249, 242, 25)
0,833	Naranja	(249, 140, 20)
1,000	Rojo	(242, 25, 25)

Las 7 paradas son perceptualmente equidistantes (no es HSV lineal: en HSV puro el amarillo cae como chartreuse, raro). Verde en el centro = $t = 0,5$.

Modo sin fuente (forma modal) — Signed Vivid (colormap_signed_vivid):

t	Color	RGB
-1,0	Azul vibrante	(25, 76, 255)
0	Gris medio	(89, 89, 89)
+1,0	Rojo vibrante	(255, 38, 25)

Reemplazo del blanco central por gris: cuando se sube la resolución del campo 3D a 50 o 70, el blanco en los nodos (líneas con valor cero) saturaba la imagen y hacía imposible distinguir los antinodos. Con gris, los nodos se ven discretamente y los antinodos saltan en rojo o azul saturado.

La saturación de color sigue una curva $\sqrt{|t|}$ (no lineal): valores moderados ($|t| \approx 0,4$) ya muestran color claro, los extremos saturan a tope. Más vibrante que el viejo mapping lineal.

El spinner **Resolución campo 3D** controla la densidad de puntos:

Valor	Puntos aprox. (sala 6×8×3 m)
20 (default)	600 – 1500 pts (rápido)
40	5000 – 8000 pts (medio)
60	12 000+ pts (detallado, lento)

8.2 Plano de corte 2D — flujo interactivo

El plano de corte permite ver la distribución modal en una sección del recinto.

Flujo de colocación interactiva:

1. En **Plano de corte** elegir la orientación: **XY** ($z = \text{cte}$), **XZ** ($y = \text{cte}$) o **YZ** ($x = \text{cte}$).
2. Presionar **⊕ Activar plano interactivo**.
3. Mover el cursor sobre el recinto 3D → aparece un plano celeste translúcido siguiendo al cursor.
4. Clic izquierdo → confirma la posición y abre el mapa de calor 2D.
5. Clic derecho → cancela sin confirmar.

Alternativa: tipear el valor en el spinner de posición y presionar **Ver mapa de calor 2D**.

Mapa de calor 2D Se abre una ventana **no modal** (queda abierta mientras se sigue trabajando). Si se confirma un nuevo plano, la ventana se actualiza sin abrir una nueva.

Modo	Colormap	Leyenda
Forma modal Presión p	Divergente azul-blanco-rojo Inferno (negro → naranja → blanco)	Amplitud modal (normalizada) dB SPL (re 20 µPa)

La zona fuera del recinto aparece en gris. El gráfico tiene grilla sobre ambos ejes para leer posiciones en metros. Se puede exportar a **PNG, SVG, PDF** (imagen) o **CSV / TXT** (valores numéricos del corte en formato largo x; y; amplitud_modal o x; y; presion_pa; spl_db según el modo activo).

9. Respuesta en Frecuencia (FRF)

Cálculo

Presionar **Calcular FRF** en el grupo “FRF (Respuesta en frecuencia)”.

Parámetro	Descripción
f mín / f máx	Rango de frecuencias del gráfico (Hz)
Nº de puntos	Resolución en frecuencia (10 - 1000). Más puntos = curva más suave, más tiempo

Gráfico

- Eje Y: **dB SPL** (referencia 20 µPa), asumiendo fuente de 1 W de potencia eléctrica.
- Las **líneas naranjas discontinuas** marcan las frecuencias de los modos FEM calculados.
- Doble clic en el gráfico → resetea el zoom al rango completo.
- La barra de herramientas permite zoom, paneo y guardar la figura.
- **Exportar**: imagen (PNG / SVG / PDF) o datos crudos (CSV / TXT). Los archivos de datos llevan columnas freq_hz; spl_db; abs_H_pa; phase_deg para que puedas reabrirlos en Excel, Python o MATLAB.

Escucha — ruido rosa filtrado por la sala

El botón **Escuchar** permite oír cómo colorea la sala al sonido:

1. Se generan 4 segundos de **ruido rosa** (densidad espectral $\propto 1/f$).
2. Se filtra con la FRF calculada usando **convolución en frecuencia**.
3. Se aplica **boost de ganancia +6 dB equivalente** con soft-clipping tanh para que sea audible sin auriculares ni speakers de estudio.
4. Se ponen **fade-in 10 ms / fade-out 50 ms** para eliminar el “pop” al iniciar y al terminar.
5. Se anexan **100 ms de silencio** al final para que el buffer del DAC termine en cero (anticasquido extra).
6. Se reproduce por los parlantes del sistema.

Formato del audio: **16 bits / 44 100 Hz / estéreo** (L = R, campo mono).

El botón **Detener** interrumpe la reproducción.

Nota: El ruido rosa solo tiene coloración de sala en el rango $[f_{\text{mín}}, f_{\text{máx}}]$ calculado. Para incluir más frecuencias, ampliar esos parámetros antes de calcular la FRF.

v2.5: si el nivel todavía te queda corto, abrí `audio_utils.py` y subí la constante `_DRIVE` (por defecto 2.5). Cada incremento de 0.5 añade ~1-2 dB de loudness RMS. Valores arriba de 4.0 empiezan a introducir distorsión audible.

10. RT60 y materiales — comparativa de métodos

El botón **Ver RT60 calculado** abre un diálogo multi-curva donde se pueden combinar **dos métodos de predicción** (Sabine y Eyring), agregar/quitar curvas para comparar, y exportar.

10.1 Métodos de predicción

Método	Fórmula (por banda f)	Cuándo usarlo
Sabine	$RT60(f) = 0,161 \cdot V / A(f)$ $A(f) = \sum \alpha_i(f) S_i$	Clásico. Asume $\alpha \ll 1$. Sobreestima RT60 cuando α es alta (no llega a cero en sala anecoica).
Eyring (Norris-Eyring)	$RT60(f) = 0,161 \cdot V / [-S \cdot \ln(1 - \bar{\alpha}(f))] \bar{\alpha} = A(f)/S$	Corrige Sabine para absorción alta. Cuando $\bar{\alpha} \rightarrow 1$, $RT60 \rightarrow 0$ correctamente. En el límite $\alpha \ll 1$ colapsa a Sabine.

Nota: en versiones anteriores se exponía también el método **Fitzroy** y las métricas **T20 / T30**. A partir de la v2.5 se removieron de la UI: el método Fitzroy queda en el código (`face_materials.compute_fitzroy_rt60_per_face`) pero ya no aparece en el combo, y se grafica únicamente T60. T20/T30 dan el mismo valor que T60 en predicciones teóricas (decaimiento exponencial puro) — sólo tienen sentido sobre mediciones reales.

10.2 Diálogo comparativo

Tiempo de reverberación – comparativa de métodos

[gráfico de curvas $RT(f)$]

X log: 63 → 8000 Hz

Y: tiempo (s)

Curvas activas:

Sabine RT60 (azul)

Eyring RT60 (rojo)

[Quitar curva seleccionada]

Agregar curva:

Método: [Sabine ▼]

[+ Agregar con asignación actual de materiales]

[Borrar todas las curvas]

Nota: Sabine asume $\alpha \ll 1$; Eyring corrige cuando α es alta...

[matplotlib toolbar]

[Exportar PNG] [SVG] [PDF] [CSV] [TXT]

[Cerrar]

Export de datos: CSV usa ; como separador y coma decimal (abre limpio en Excel-es); TXT usa tabulador y punto decimal (universal). Una columna banda_hz y una por cada curva activa (ej.: Sabine RT60_s, Eyring RT60_s).

Flujo típico para comparar

1. Asignar materiales en la ventana **Materiales** (botón **Materiales...**).
2. Abrir el diálogo con **Ver RT60 calculado** → aparece automáticamente la curva **Sabine RT60**.
3. Seleccionar **Eyring** en el combo Método y apretar + **Agregar** → segunda curva en rojo.
4. Si se quiere comparar dos materializaciones distintas: cerrar el diálogo, cambiar materiales en la ventana **Materiales**, reabrir el diálogo y agregar de nuevo Sabine. Aparecerá como **Sabine RT60 #2** (las dos quedan superpuestas para comparar el efecto del cambio).
5. Las casillas de cada curva permiten **ocultarla** sin borrarla; el botón **Quitar curva seleccionada** la elimina.

10.3 Código de colores

Visual	Significado
Azul (#1f77b4), línea sólida con círculos	Sabine RT60
Rojo (#d62728), línea sólida con círculos	Eyring RT60

10.4 Cómo afecta el amortiguamiento modal

El RT60 promedio (Sabine, por defecto del cálculo FEM) se traduce a un amortiguamiento por modo:

$$\xi_n = 1,1 / (f_n \cdot \text{RT60}(f_n))$$

Esto hace que los picos de la FRF sean más o menos anchos según la absorción del recinto:

- **Materiales absorbentes** → RT60 corto → ξ_n grande → picos anchos en la FRF.
- **Superficies duras** → RT60 largo → ξ_n pequeño → picos agudos y altos.

(El cálculo interno del FEM usa **Sabine** para mantener la compatibilidad numérica; el método Eyring se usa únicamente en el diálogo comparativo de visualización. Si querés que el amortiguamiento modal use Eyring, basta reemplazar la llamada a `compute_sabine_rt60_per_face` por `compute_eyring_rt60_per_face` en `acoustic_panel._compute_xi_from_materials`.)

10.5 Parches de absorción sub-cara

Además de asignar **un material por cara**, se puede dibujar una **región (parche) dentro de una cara** y darle su propio material. Botón “**Parches de absorción...**” en el grupo Materiales.

Qué hace físicamente. Un parche no es una física nueva: es darle **resolución sub-cara** al amortiguamiento modal selectivo (criterio A36). Como las formas modales ϕ_n se calculan con paredes rígidas, un parche **no cambia la forma del modo ni el heatmap** — su α entra por dos vías: (1) el **RT60 de Sabine** (le resta área al material anfitrión y aporta la suya), y (2) el **ξ_n por modo**, pesado por la presión modal ϕ_n^2 **sobre la región del parche**. Efecto: un modo cuyo antinodo cae sobre el parche se amortigua más; uno con nodo ahí casi no lo ve. Lo observable es sobre $\xi_n \rightarrow \text{RT} \rightarrow \text{FRF}$.

Editor 2D. Elegís la cara de una lista (alcance v1: caras perpendiculares a un eje), y dibujás sobre su plano local:

- **Modo Rectángulo** — mantené el botón izquierdo y arrastrá.

- **Modo Polígono** — click izquierdo por cada vértice (convexo o no); cerrás clickeando cerca del primer punto, con **Enter** o doble click; **botón derecho / Esc** deshace el último vértice o cancela.
- **Rueda del mouse** = zoom in/out sobre la grilla, centrado en el cursor.
- Los parches **no pueden solaparse**: si el candidato pisaría a otro se dibuja en rojo y no se agrega.
- El material se elige del combo (se aplica al parche nuevo o al seleccionado); botón derecho sobre un parche lo borra.

Espesor del tratamiento. El parche se dibuja como un **prisma** hacia el interior de la sala, con el espesor real del panel (por defecto **10 cm**). Al elegir un material del catálogo el espesor **se autocompleta** con el de esa construcción: “Lana de vidrio 100 mm” → 10 cm, “Cielorraso 20 mm suspendido a 200 mm” → 22 cm. Si lo cambiás a mano y no coincide con el del material, el diálogo **avisa**.

El espesor es geométrico, no entra al solver. El $\alpha(f)$ del catálogo se midió (ISO 354) **con** el espesor de esa construcción: para una misma lana, el α a 63 Hz cambia unas **15 veces** entre 20 y 100 mm. O sea que el espesor **ya está afectando** el cálculo, por α . Sumarlo además como obstáculo sería contar la misma física dos veces. Y desplazar la pared hacia adentro sería peor: a 34 Hz un panel de 10 cm es $\lambda/100$, la onda lo atraviesa y rebota contra el muro rígido de atrás, así que la frontera acústica sigue siendo la pared. El diálogo muestra el $\lambda/4 = c/4d$ del espesor elegido, que es la frecuencia debajo de la cual un poroso al ras es casi transparente (10 cm → ~858 Hz, muy por encima de la banda modal).

Los parches se **pintan sobre la cara en el visor 3D** (color del material, con las **aristas del prisma resaltadas** para que se lean con cualquier color de relleno) y se guardan en el .room.

Muebles delante de un parche. Si un mueble se superpone con el prisma de un parche, aparece un **aviso** (no se bloquea): un mueble delante del absorbente lo tapa, así que su α efectivo en esa zona va a ser menor que el del catálogo.

Desde el diálogo Materiales... los parches también aparecen listados debajo de las caras, como ↵ Parche (rect/polígono) en <cara>, con su área y categoría. Ahí podés **cambiarle el material** con su combo (igual que a una cara), y al **posar el cursor sobre la fila** el parche se **resalta en el 3D** con el mismo brillo ámbar que usan las caras.

Cuadratura fina (nota importante). Sin parches, la absorción se integra como siempre (baseline intacto). **Al activar el primer parche**, el ξ_n se recalcula con **cuadratura fina** (tesela la cara en muchos puntos, más preciso que la malla de render gruesa) — los números de RT/FRF **pueden moverse** respecto de la malla gruesa: es mayor precisión, no un error. Núcleo: absorption_patch.py (compute_xi_per_mode_with_patches, sabine_rt60_with_patches); bench bench_absorption_patch.py.

10.6 Cargar tu propio material

En el diálogo **Materiales...**, el botón **“Cargar tu material...”** muestra un cuadro con la sintaxis del JSON esperado y abre un selector de archivo. El archivo se **valida** (nombre + coeficiente de absorción por banda), se **copia a la carpeta materials/** (sin pisar los del catálogo) y queda disponible en **todo el programa** (Acústica y Predicción). Formato aceptado (un material por archivo):

```
{
  "name": "Mi panel absorbente",
  "category": "Paneles perforados",
  "description": "opcional",
  "source": "opcional (ficha / medición)",
  "alpha": {"63": 0.15, "125": 0.30, "250": 0.55, "500": 0.75,
```

```

    "1000": 0.65, "2000": 0.50, "4000": 0.40, "8000": 0.35}
}

```

Alternativa a alpha: "absorption_coef": [a63, a125, a250, a500, a1000, a2000, a4000, a8000] (8 valores). Bandas de octava: 63-8000 Hz.

11. Flujo de trabajo completo

1. Diseñar recinto (pestaña Geometría)
- ↓
2. Asignar materiales (piso / techo / paredes)
- ↓
3. Colocar fuentes (Ctrl+clic derecho en el visor)
y receptor (Shift+arrastrar o spinboxes)
- ↓
4. Configurar sensibilidad de cada fuente
(doble clic sobre la fuente → editar dB/W/m)
- ↓
5. Calcular modos (FEM)
- ↓
6. Visualizar campo 3D o planos de corte
 - "Forma modal": patrón del modo, sin fuente
 - "Presión |p|": efecto real de las fuentes
- ↓
7. Mover fuentes → ver cambios en "Presión |p|" (actualización automática a los 350 ms)
- ↓
8. Calcular FRF → ver curva + escuchar la sala

12. Referencia rápida de atajos

Atajo	Acción
Ctrl + Z	Deshacer
Ctrl + Y	Rehacer
Ctrl + S	Guardar recinto
Ctrl + Shift + S	Guardar como
Ctrl + O	Abrir recinto
Ctrl + I	Importar CAD
0	Reset cámara vista isométrica
Enter (pestaña Acústica)	Actualizar campo 3D
Enter (pestaña Predicción)	Disparar Predecir
Ctrl + clic derecho (visor)	Colocar fuente en el piso
Shift + clic izquierdo + arrastrar	Mover fuente o receptor en plano XY (z fijo)
Ctrl + Shift + clic izquierdo + arrastrar	Mover fuente o receptor solo en altura (xy fijos) — (v2.7)
Ctrl + Shift + Alt + X/Y/Z	Fijar / liberar eje de rotación de cámara
Doble clic sobre fuente	Editar fuente

13. Solución de problemas

Problema	Solución
La aplicación no arranca	Usar run.bat, no python main.py directamente. El .bat apunta a la ruta de Anaconda explícitamente.
La nube 3D no aparece	Verificar que se calcularon los modos primero. En modo “Presión $ p $ ” verificar que hay al menos una fuente.
Las fuentes no se ven en el visor	Cambiar a modo de visualización Aristas (mesh traslúcido). En modo “Externa” (opaco) las fuentes interiores quedan tapadas por las paredes.
“Forma modal” y “Presión $ p $ ” lucen igual	Es física: en resonancia $ p \propto \phi_n $. Mover la fuente hacia un nodo del modo para ver la diferencia.
El audio no suena	Verificar que los parlantes del sistema están activos y el volumen de Windows no está en cero.
FRF tarda mucho	Reducir el número de modos o el número de puntos. Para exploración rápida: 8 modos, 200 puntos.
Al cambiar la geometría se borran los modos	Comportamiento esperado: la malla FEM anterior ya no corresponde al nuevo recinto. Recalcular con Calcular modos (FEM) .
Error de PortAudio al intentar reproducir	No se necesita sounddevice. El audio usa winsound (API nativa de Windows). Verificar que scipy está instalado.

14. Conceptos físicos clave

Modos acústicos

Los modos son las frecuencias naturales del recinto: a esas frecuencias la presión forma un patrón estacionario con **nodos** (presión nula) y **antinodos** (presión máxima). La forma modal $\phi_n(x)$ muestra la distribución espacial de ese patrón para el modo n .

Los modos axiales son los más fáciles de identificar: corresponden a la longitud de onda que “cabe exactamente” en una de las dimensiones del recinto.

Modo axial: $f_n = n \cdot c / (2 \cdot L)$ ($c = 343$ m/s, $L =$ longitud de la sala)

Sensibilidad del altavoz → caudal volumétrico Q

S = sensibilidad en dB SPL @ 1W/1m

p_0 = $20 \mu\text{Pa} \cdot 10^{(S/20)}$ (presión a 1W/1m en campo libre)

$|Q|$ = $p_0 \cdot 4\pi / (\omega_{\text{ref}} \cdot \rho_0)$ ($\omega_{\text{ref}} = 2\pi \cdot 1000$ rad/s, $\rho_0 = 1.21$ kg/m³)

Para $S = 90$ dB/W/m: $|Q| \approx 1.05 \times 10^{-3}$ m³/s. Para $S = 100$ dB/W/m: $|Q| \approx 3.32 \times 10^{-3}$ m³/s ($\approx 3.16\times$ mayor, coherente con +10 dB).

Amortiguamiento modal y RT60

El amortiguamiento ξ_n es la fracción de energía que se pierde por ciclo en el modo n . Determina el ancho de los picos de la FRF.

$$\xi_n = 1.1 / (f_n \cdot RT60(f_n))$$

Ejemplos: - Sala de hormigón sin tratar: $\xi_n \approx 0.004 - 0.008 \rightarrow$ picos muy agudos. - Sala con tratamiento acústico parcial: $\xi_n \approx 0.02 - 0.05 \rightarrow$ picos moderados. - Sala anecoica: $\xi_n > 0.1 \rightarrow$ sin picos visibles.

Frecuencia de Schroeder

Es el límite entre el régimen modal (modos individuales y discretos) y el campo estadístico (modos tan densos que se superponen):

$$f_S = 2000 \cdot \sqrt{(RT60 / V)}$$

El FEM es preciso y útil por **debajo** de f_S . Por encima, el campo puede describirse estadísticamente (energía uniforme).

Cadena de procesamiento de audio

Ruido rosa (4 s)

- $\rightarrow$ FFT
- $\rightarrow \times H(f)$ [FRF de la sala, rango $f_{\min} - f_{\max}$]
- $\rightarrow$ IFFT
- $\rightarrow$ Normalizar a ± 0.85
- $\rightarrow$ WAV 16 bit / 44100 Hz / estéreo
- $\rightarrow$ winsound.PlaySound (API nativa de Windows)

El resultado es la sala “impresa” en el ruido: se oyen los picos de resonancia como coloración tonal del ruido.

15. Importar CAD y motor de mallado

A partir de la versión 2.0, Prototipo 1 puede trabajar con **geometrías arbitrarias importadas desde CAD** (auditorios, salas con curvas complejas, modelos arquitectónicos profesionales) y elige automáticamente el motor de mallado más apropiado para cada caso.

15.1 Por qué hay dos motores

El cálculo modal FEM requiere descomponer el volumen del recinto en tetraedros. Hay dos estrategias para hacerlo:

Motor	Cómo funciona	Cuándo conviene
Voxel	Rejilla estructurada de cubos partida en tetraedros, filtrando los que caen fuera del recinto.	Recintos <i>axis-aligned</i> : paredes verticales, techo plano y aristas paralelas a los ejes X/Y. En ese caso las celdas voxel coinciden exactamente con las paredes y el motor es exacto sin overhead .

Motor	Cómo funciona	Cuándo conviene
Gmsh (boundary-fitted)	Mallador profesional con kernel OpenCASCADE que genera tetraedros que se ajustan exactamente a la geometría, por curva que sea.	Recintos con paredes curvas, oblicuas o cualquier malla importada de CAD. Para geometrías curvas, el motor voxel introduce error de escalera que rompe las degeneraciones modales (modos físicamente iguales aparecen separados artificialmente).

Evidencia experimental (cilindro R=4 m, H=4 m)

Motor	Volumen calculado	t _{total}	Split modos 1/2 (deberían ser idénticos por simetría)
Gmsh boundary-fitted	200.26 m ³ (error -0.4 %)	0.51 s	0.0017 Hz (degeneración preservada)
Voxel forzado	201.39 m ³ (error +0.16 %)	2.99 s	0.0275 Hz (degeneración rota)

Conclusión: en geometría curva, gmsh es 6× más rápido y 16× más preciso para preservar la simetría rotacional.

15.2 El badge de estado

En la pestaña **Acústica**, dentro del grupo *FEM modal*, hay un **badge coloreado** que indica qué motor se va a usar y por qué:

Color	Texto	Significado
verde	voxel · exacto	Recinto axis-aligned: voxel coincide con la frontera. Sin error de escalera. Óptimo.
azul	gmsh · boundary-fitted	Geometría curva o CAD importado: gmsh ajusta la malla a las paredes exactas. Óptimo.
amarillo	voxel · escalera	Geometría paramétrica con techo curvo: voxel funciona pero con error sistemático. Para boundary-fitted exacto, importá un CAD.
naranja	gmsh · forzado	Forzaste gmsh donde voxel sería exacto. Sin perjuicio, pero algo más lento.

Pasá el mouse sobre el badge para ver el detalle del razonamiento.

El badge se actualiza automáticamente cuando cambiás la geometría: por ejemplo, al subir el slider *Altura del techo* de 0 a 1 m con techo en arco, el badge salta de verde a amarillo.

15.3 Importar un archivo CAD

Tres formas de invocar el importador:

1. Botón **Importar CAD...** en el grupo *FEM modal* de la pestaña Acústica.
2. Atajo de teclado **Ctrl + I**.
3. Línea de comandos: arrastrá un *.stl/.obj/.step* sobre el ejecutable (en una futura versión).

Al elegir un archivo, el soft hace tres cosas:

1. **Carga** la malla (trimesh para mallas triangulares, gmsh+OpenCASCADE para B-rep como STEP/IGES).
2. **Diagnostica** la malla: cuenta vértices, triángulos, calcula el volumen, mide si es *watertight* (cerrada), si tiene caras degeneradas, si hay aristas no-manifold, y **lista todos los huecos**.
3. Si la malla está perfecta → la usa directamente.
4. Si tiene problemas → abre el **diálogo de reparación guiada** (sección 15.4).

Después de importar, el badge del panel acústico cambia automáticamente a gmsh · boundary-fitted y el botón × **Volver a paramétrica** se habilita para regresar a la geometría del panel Geometría.

15.4 Diálogo de reparación guiada

Cuando el CAD tiene problemas (huecos, T-junctions, vértices duplicados), aparece una ventana con tres zonas:

Resumen de la malla • V, watertight, etc. Problemas detectados ▶ Hueco 1: 4 verts ▶ Hueco 2: 8 verts ▶ Hueco 3: 3 verts Acciones (hueco actual) ✓ Cerrar hueco (auto) - Soldar a vecinos ☞ Mover vértice... → Omitir Acciones globales • Reparar TODO auto • Fusionar duplicados • Normalizar	Preview 3D [malla en gris translúcido] [hueco actual en ROJO] [centroide en amarillo]
---	---

Acciones por hueco

- **Cerrar hueco (auto)**: triangulación por abanico desde un punto central. Funciona para huecos planos chicos. La normal del parche se orienta automáticamente hacia afuera.
- **Soldar a vecinos**: si el hueco es un T-junction (dos vértices muy próximos que deberían ser uno), fusiona vértices del borde del hueco con vértices cercanos del resto de la malla. Ajustá la **tolerancia (m)** para controlar la distancia máxima.

- **Mover vértice...**: abre un sub-diálogo donde elegís un vértice del ciclo del hueco y le asignás una nueva posición XYZ con precisión. Útil cuando el hueco viene de un error de modelado y querés mover un vértice exacto a su lugar correcto.
- **Omitir**: pasa al siguiente hueco sin tocar el actual.

Acciones globales

- **Reparar TODO automáticamente**: cierra todos los huecos en cadena. Es la opción “atajo” cuando confiás en la geometría general.
- **Fusionar vértices duplicados**: junta vértices a distancia < 0.1 mm. Frecuente en STL exportado por algunos CAD.
- **Normalizar (winding/normales)**: orienta todas las normales hacia afuera del recinto y arregla winding inconsistente.

El preview 3D muestra el hueco actual seleccionado en rojo grueso, con esferas naranjas en los vértices del ciclo y una esfera amarilla en el centroide. Al aplicar una acción, el diagnóstico se recalcula y la lista se refresca en vivo.

Cuando todos los problemas se resuelven, aparece el mensaje verde “✓ **Malla lista para mado volumétrico**” y podés cerrar el diálogo con **Aceptar**.

15.5 Override manual del motor

Junto al badge hay un combo **Motor de mallado** con tres opciones:

- **Automático (default)**: el router decide según las reglas de la tabla 15.8.
- **Voxel**: fuerza voxel. Útil para benchmarking o para evitar el overhead de gmsh en recintos pequeños.
- **Gmsh**: fuerza gmsh. Útil cuando querés boundary-fitted incluso sobre un shoebox.

El override **se persiste** por proyecto (campo `acoustic.mesh_engine` en el `.room`) y como **default global** (en `%APPDATA%\Prototipo1\settings.json` en Windows, `~/ .config/Prototipo1/` en Linux/macOS). Eso significa que tu preferencia se mantiene entre sesiones.

Si forzás un motor que no coincide con el automático, el badge cambia de color para avisarte:

- Forzar **voxel sobre geometría curva** → badge amarillo “voxel · ESCALERA” con advertencia clara.
- Forzar **gmsh sobre shoebox** → badge naranja “gmsh · forzado” (sin perjuicio, solo informativo).

15.6 Formatos soportados

Familia	Formatos	Loader interno
Mallas triangulares	.stl (ASCII y binario), .obj, .ply, .glb, .gltf, .3mf, .dae (COLLADA), .off, .xyz	trimesh
B-rep paramétrico (CAD profesional)	.step, .stp, .iges, .igs, .brep	gmsh (kernel OpenCASCADE)

Los formatos B-rep se teselan automáticamente al cargar: el kernel OpenCASCADE de gmsh los convierte en triángulos respetando la parametrización original. Esto significa que un STEP de un auditorio importado desde Revit o SolidWorks llega al motor FEM **sin pérdida geométrica**.

15.7 Persistencia en .room v3

El formato .room evolucionó de v2 a v3 para guardar:

```
{
  "format": "prototipo1.room",
  "version": 3,
  "params": { ... },          // geometría paramétrica (todos los sliders)
  "acoustic": {
    "mesh_engine": "auto",    // override por proyecto: "auto" / "voxel" / "gmsh"
    "h_target": 0.40,        // tamaño de tetraedro para gmsh
    "n_per_meter": 2.5,      // densidad para voxel
    "n_modes": 12,
    "sources": [
      { "label": "L", "position": [...], "Q_real": ..., "Q_imag": ...,
        "sensitivity_dB": 90.0 }
    ],
    "receiver": [3.0, 4.0, 1.5]
  },
  "external_geometry": {     // solo si hay CAD importado
    "kind": "embedded_mesh",
    "format": "trimesh-json-v1",
    "vertices": [[x,y,z], ...],
    "faces": [[i,j,k], ...]
  }
}
```

Importante: el .room **embebe una copia completa de la malla CAD importada**. Eso lo hace **autocontenido y portable**: podés mandar el .room por mail sin necesidad de adjuntar el STEP/STL original. El tamaño extra es proporcional al número de triángulos (típicamente 5-500 KB para mallas de auditorio razonables).

Los archivos .room v2 antiguos siguen abriéndose sin problema (los campos nuevos son opcionales).

15.8 Cuándo usa cada motor el router automático

A partir de v2.1 el router opera en modo **best-effort**: para geometrías curvas intenta gmsh primero y, solo si falla por incompatibilidad topológica, cae a voxel reportando la razón.

Geometría	Auto-decisión (intento)	Badge si gmsh OK	Badge si gmsh falla
Paramétrica · shoebox axis-aligned · techo plano	voxel directo	verde “voxel · exacto”	n/a (no se intenta gmsh)
Paramétrica · pentágono regular · techo plano	voxel directo	verde “voxel · exacto”	n/a
Paramétrica · arch_height > 0	intenta gmsh	azul “gmsh · boundary-fitted”	amarillo “voxel · fallback” (razón en tooltip)
Paramétrica · twist/taper/pitch ≠ 0	intenta gmsh	azul	amarillo “voxel · fallback”

Geometría	Auto-decisión (intento)	Badge si gmsh OK	Badge si gmsh falla
Paramétrica · base_polygon custom no axis-aligned	intenta gmsh	azul	amarillo “voxel · fallback”
CAD importado (cualquier formato)	intenta gmsh	azul “gmsh · boundary-fitted”	amarillo “voxel · fallback”

Con la geometría paramétrica actual de Prototipo 1, los techos en arco generan T-junctions entre el techo subdividido (muchos vértices en aristas del polígono) y las paredes (solo 4 vértices por arista). Gmsh detecta esto como “Wrong topology of boundary mesh for parametrization” o “PLC Error: A segment and a facet intersect at point” y el sistema cae automáticamente a voxel con badge amarillo. Para evitar este fallback en recintos curvos, importé la geometría como CAD limpio.

15.9 Workflow recomendado para auditorios complejos

1. **Modelar en CAD externo** (Revit, SketchUp, SolidWorks, Blender, FreeCAD...) — modelado intuitivo y materiales reales.
2. Exportar como **STEP** (para geometría paramétrica de calidad) o **STL** (universal). Si tu CAD soporta IFC o glTF también sirven.
3. En Prototipo 1, Ctrl + I → seleccionar el archivo.
4. Si hay huecos, reparar con el diálogo guiado (sección 15.4) — generalmente “Reparar TODO automáticamente” funciona en mallas razonables.
5. Posicionar fuentes y receptor desde el panel Acústica.
6. Asignar materiales (piso/techo/paredes) — la asignación inteligente por área aplica a CAD también.
7. **Calcular modos (FEM)** — el router elige gmsh automáticamente.
8. Visualizar modos en 3D, hacer FRF, escuchar la sala.
9. Ctrl + S para guardar el proyecto completo (incluido el CAD embebido).

15.10 El sistema best-effort y el fallback

El router de Prototipo 1 funciona en modo **best-effort**: para cualquier geometría no axis-aligned, intenta gmsh primero (por rigor científico). Solo si gmsh detecta una incompatibilidad topológica (T-junctions, huecos imperceptibles, normales inconsistentes) cae automáticamente a voxel y muestra un badge amarillo “**voxel · fallback**” con la razón exacta del fallo en el tooltip.

Por qué existe el fallback Una malla puede ser **visualmente correcta** pero **topológicamente inválida** para un mallador profesional. Ejemplos típicos:

- **T-junctions**: dos triángulos comparten una arista, pero un tercer triángulo tiene un vértice sobre esa arista que los otros dos no conocen.
- **Aristas no-manifold**: una arista compartida por 3 o más triángulos (imposible físicamente).
- **Caras superpuestas o invertidas**: caras coplanares duplicadas, o normales que apuntan hacia adentro en lugar de hacia afuera.

Para visualizar la sala en 3D estos problemas son irrelevantes — el render OpenGL los tolera. Para tetraedrizar con gmsh, son fatales. La opción anterior era abortar el cálculo con un error; con best-effort, el sistema sigue funcionando y avisa qué pasó.

Cómo se ve en la UI Cuando hay un fallback, después de presionar **Calcular modos (FEM)**:

1. El log de estado (parte inferior del panel) muestra una línea de advertencia con el motivo:
Gmsh intentado pero falló (PLC Error: A segment and a facet intersect at point).
Cayendo a voxel.
FEM listo (voxel). Malla: 2386 nodos, 9504 tets, $V \approx 177.84 \text{ m}^3$, $\bar{h} \approx 0.42 \text{ m}$.
2. El badge cambia de **azul “(previsto) gmsh · boundary-fitted”** a **amarillo “voxel · fallback”**.
3. Hover sobre el badge: tooltip con la razón completa del fallo de gmsh.

Cuándo el fallback no aplica

- **Override manual del usuario = “Voxel”**: el router respeta tu elección sin intentar gmsh.
- **Override manual del usuario = “Gmsh”**: si gmsh falla, el sistema **lanza una excepción explícita** en lugar de caer silenciosamente, porque vos pediste gmsh expresamente.
- **Geometría shoebox axis-aligned**: el router elige voxel directo (es exacto, no hay nada que ganar con gmsh).

Qué hacer cuando ves “voxel · fallback”

1. **Mirá la razón en el tooltip del badge**. Si dice “PLC Error” o “T-junctions”, probablemente tu malla CAD tiene aristas mal conectadas.
2. **Opción rápida**: aceptás el voxel con error de escalera ($\sim 0.5\text{--}1\%$ en frecuencias, degeneraciones rotas en simetrías rotacionales).
3. **Opción profesional**: re-importás el CAD usando `Ctrl + I` y aplicás reparación guiada (sección 15.4). El diálogo te muestra los huecos uno por uno; cerralos y volvé a calcular.
4. **Opción exhaustiva**: revisás la malla en el CAD original. Para STEP/IGES esto suele no pasar; para STL exportado por algunos editores (SketchUp viejo, Blender con boolean ops sin manifold3d), conviene re-exportar con la opción “merge close vertices” activada.

Apéndice — Archivos del módulo CAD/mallado

Para usuarios técnicos que quieran inspeccionar o modificar el código:

Archivo	Función
mesh_gmsh.py	Wrapper de gmsh con limpieza automática de trimesh y manejo de errores (retry de <code>classifySurfaces</code> con ángulos progresivamente más permisivos).
mesh_router.py	Detector de geometría axis-aligned y lógica de selección de motor (voxel/gmsh). Incluye <code>badge_for()</code> que produce {color, texto, tooltip}.
geom_import.py	Loaders dispatched por extensión, <code>diagnose()</code> , <code>find_holes()</code> , y las primitivas de reparación (<code>fill_hole_planar</code> , <code>snap_hole_vertices</code> , <code>move_vertex</code> , <code>normalize_mesh</code>).
geom_repair_dialog.py	MeshImportDialog con <code>_MeshPreview</code> (<code>GLViewWidget</code>) y <code>_VertexEditDialog</code> .
app_settings.py	Persistencia de preferencias globales en <code>%APPDATA%/Prototipo1/settings.json</code> .

Archivo	Función
acoustic_analysis.py	Función <code>run_fem_modal_routed(...)</code> que orquesta router → FEM con un único llamado.

16. Escala y orientación al importar

Cuando se carga un archivo CAD (Ctrl + I), antes del diálogo de reparación aparece un **diálogo de escala y orientación**. Resuelve dos problemas comunes que aparecen sistemáticamente al importar:

16.1 Escala — unidades del archivo

Los formatos CAD no llevan información de unidades estandarizada: un archivo STL puede estar dibujado en milímetros, centímetros, metros o pulgadas, y el visor no tiene forma de saberlo. Si el archivo viene en milímetros y el soft lo interpreta como metros, el recinto queda 1000 veces más grande de lo esperado y se sale completamente de la grilla.

El diálogo detecta automáticamente la unidad probable midiendo la **diagonal del AABB** y aplicando una heurística:

Diagonal del AABB	Sugerencia	Razón
> 5 000 m	÷ 1 000 (mm → m)	Archivo probablemente en milímetros
> 500 m	÷ 100 (cm → m)	Archivo probablemente en centímetros
> 60 m	÷ 10	Grande para un recinto típico
0,5 - 60 m	× 1	Rango plausible — no se sugiere escalar
< 0,5 m	× 100 o × 1 000	Demasiado pequeño

El usuario puede:

- **Aceptar la sugerencia** (radio button preseleccionado).
- Elegir un preset distinto: mm→m, cm→m, dm→m, m→mm, m→cm, in→m, ft→m.
- Ingresar un **factor manual** con precisión.
- Apretar **Auto-encajar diagonal a 20 m** para forzar que la malla tenga exactamente esa diagonal, independiente de la unidad original. Útil cuando la heurística falla.

El preview muestra en vivo las nuevas dimensiones y avisa si quedaron muy chicas (insumible) o muy grandes (fuera de la grilla del visor).

16.2 Orientación — eje “up” del archivo

Los formatos CAD también difieren en qué eje consideran **vertical**:

Formato	Convención típica
Prototipo 1, FreeCAD, AutoCAD, STEP, IGES, STL técnico	Z+ up (eje Z apunta hacia arriba)
OBJ, glTF, GLB, COLLADA, FBX, 3MF (Blender, Unity, Unreal)	Y+ up (eje Y apunta hacia arriba)

Si un OBJ Y-up se importa sin reorientar, lo que en el archivo era “pared trasera” termina siendo el “piso” del recinto. El diálogo permite elegir la convención del archivo con un combo:

- **Z+ up** (sin transformación) — Prototipo 1, FreeCAD, AutoCAD.
- **Y+ up** — OBJ / glTF / Blender / Unity. Aplica rotación -90° alrededor de X.
- **X+ up** — caso raro pero existe.
- **Z-, Y-, X-** — versiones con eje invertido (modelos espejados).

Al confirmar el diálogo, primero se aplica la **reorientación** y después la **escala**, así las nuevas dimensiones del preview ya están expresadas en los ejes del soft (X, Y horizontales; Z vertical).

Sugerencia automática por extensión: si el archivo termina en .obj, .gltf, .glb, .dae, .fbx o .3mf, el combo arranca preseleccionado en **Y+ up**. Para el resto (STL, STEP, IGES, BREP, PLY), arranca en **Z+ up**.

16.2 Tres botones del diálogo de escala

Botón	Qué hace
Aplicar escala	Usa el factor + orientación elegidos y continúa la importación.
No escalar	Saltea el escalado (factor = 1,0) pero igual continúa la importación . Útil cuando ya sabés que el archivo está en metros y la heurística sugiere algo distinto que no querés aplicar.
Cancelar import	Aborta toda la importación. La tecla Esc y el botón × de la ventana hacen lo mismo.

Cambio en v2.8: antes “No escalar” cancelaba la importación entera (bug histórico — el botón decía “Cancelar” con texto “No escalar”). Ahora son tres opciones explícitas con semántica clara.

16.3 Optimizaciones del reparador de mallas

Las funciones de reparación en `geom_import.py` se reescribieron para que escalen bien con mallas grandes:

- **find_holes** quedó vectorizado con `numpy.unique`. Para mallas de $\sim 100k$ caras: ~ 50 ms en vez de ~ 3 s del bucle Python.
- **fill_all_holes_auto** pasa en un único pase: detecta todos los huecos, construye los parches en arrays NumPy y materializa el `trimesh.Trimesh` una sola vez. Para K huecos sobre N triángulos: $O(N + K)$ en vez de $O(N \cdot K)$.
- **snap_hole_vertices** usa `scipy.spatial.cKDTree` para nearest-neighbor en lote. Para mallas con >10 k vértices es $\sim 100\times$ más rápido que el bucle anterior.

En la práctica: un cilindro de ~ 4 k triángulos con 50 huecos forzados se repara completo en ~ 350 ms.

17. Indicador de ejes y rotación con eje fijo

En la **esquina inferior derecha** del visor 3D aparece un pequeño rectángulo con tres cuadrados etiquetados X, Y, Z. Es el **indicador de ejes**, y sirve para restringir la rotación de la cámara a un eje mundial específico.

17.1 Fijar un eje

Dos formas equivalentes:

1. **Click izquierdo** sobre el cuadrado X, Y o Z en el indicador.
2. **Atajo de teclado:** Ctrl + Shift + Alt + X (o Y, o Z).

Ambos comportamientos son **toggle**: aplicar de nuevo el mismo libera el eje.

Cuando un eje queda fijo, su cuadrado se ilumina con un color asociado a la flecha del eje en la escena:

Eje	Color del cuadrado al activarse
X	rojo
Y	azul
Z	verde

La barra de status confirma la operación con un mensaje del tipo “Eje Y fijado. Rueda del mouse (presionada) gira alrededor de ese eje”.

17.2 Comportamiento de la rueda del mouse con eje fijo

- **Sin eje fijo** (comportamiento original): rueda del mouse presionada + arrastrar = orbit libre (cambia azimuth y elevation).
- **Con eje fijo**: rueda del mouse presionada + arrastrar **horizontalmente** = el recinto rota únicamente alrededor del eje mundial fijado. El componente vertical del movimiento del mouse se ignora para que el control sea predecible.

Internamente: el visor calcula la posición actual de la cámara en coordenadas esféricas, aplica una matriz de rotación 3D alrededor del eje fijo y recalcula azimuth/elevation/distance para los nuevos valores.

17.3 Cuándo usar esta función

- **Inspeccionar el perfil lateral** del recinto: fijar Y o X, rotar horizontalmente → la altura (Z) queda preservada.
- **Inspeccionar la planta** rotándola sin perder la vista superior: fijar Z, rotar → solo cambia el azimuth.
- **Documentar la geometría desde un eje específico** para una figura del informe.
- **Comparar perfiles simétricos** del recinto entre rotaciones controladas.

18. Rendimiento y benchmarks

A partir de la versión 2.3 el repositorio incluye `benchmark_v2.py`, una suite headless (sin GUI) que mide los principales caminos de la app y deja un reporte completo en **BENCHMARK_RESULTS.md**. Es útil para diagnosticar lentitud en tu equipo o para comparar entre versiones.

18.1 Cómo correr los benchmarks

```
%USERPROFILE%\anaconda3\python.exe benchmark_v2.py
```

El script demora **2-5 minutos** (depende del CPU). Mientras corre imprime cada bloque en consola y al final escribe **BENCHMARK_RESULTS.md** en la raíz del proyecto.

18.2 Qué mide

Bloque	Qué evalúa
B1	Tiempo de FEM completo (mallado voxel + ensamblaje K/M + Lanczos) para 5 geometrías paramétricas.
B2	Campo 3D (forma modal y $ p $) a resoluciones 20, 30, 40, 50, 60, 70.
B3	Comparativa exacta loop Python (antes) vs KDTree vectorizado (ahora) — incluye la diferencia numérica.
B4	Tiempo de agrupación de caras por región planar para 6 tipos de recinto.
B5	Importación CAD descompuesta en fases (load / diagnose / suggest_scale) para 5 tamaños de STL.
B6	Cálculo de RT60(f) con asignación por grupo.
B7	Costo de memoria del KDTree del FieldEvaluator.

Cada test corre **3 veces** y reporta mediana + min/max.

18.3 Resultados de referencia (Intel/AMD 16 hilos, 64 GB RAM)

Los números varían con el equipo, pero estos son representativos:

- **Campo 3D a resolución 50** (62 500 puntos): ~ 280 ms. Antes de la optimización: 15-25 s.
- **Campo 3D a resolución 70** (170 000 puntos): ~ 740 ms.
- **Comparativa loop vs KDTree**: 50-170× más rápido, diferencia numérica < 1e-15.
- **Agrupación de caras**: < 2.5 ms para cualquier recinto paramétrico.
- **Importación CAD**: 20 ms (5 k tris) → 1.2 s (327 k tris).
- **Cálculo de RT60 por grupo**: < 1 ms.

18.4 Cuellos de botella restantes

Una vez vectorizada la evaluación del campo (el cuello histórico), el tiempo dominante del cálculo FEM está en el **mallado volumétrico** (`acoustic_mesh.build_volume_mesh`):

Sala	npm	tets	mallado
6 × 8 × 3	2,5	14 400	1,2 s
6 × 8 × 3	3,5	35 280	3,0 s
10 × 10 × 4	2,5	28 140	3,4 s

Por eso BENCHMARK_RESULTS.md recomienda mantener **npm ≤ 3,0** para diseño cotidiano. Subir a 3,5-4,0 solo cuando se necesita precisión a más de 300 Hz y se acepte esperar 5-10 s por cada recálculo de modos.

18.5 Recomendaciones operativas

1. **Resolución del campo 3D**: usar **40-50** para exploración (< 0,3 s); subir a **70** sólo para figuras finales (~ 0,7 s).

2. **Densidad de malla FEM** (n_per_meter): **2,5** para el día a día; **3,0-3,5** cuando se requiera precisión a alta frecuencia.
3. **Importación de CADs grandes**: archivos de > 100 k triángulos van a tardar 0,5-1,5 s por la fase de diagnose. El nuevo QProgressDialog muestra qué está pasando y permite cancelar.
4. **Materiales por cara**: el diálogo se puede abrir y cerrar libremente; cambiar asignaciones recomputa RT60 en menos de 1 ms.

19. Predicción de geometría — pestaña Predicción

A partir de **v2.6** hay una tercera pestaña, **Predicción**, que invierte el flujo de trabajo: en vez de diseñar una sala y después medir si es buena, le decís al soft **qué necesitás** (uso, capacidad, restricciones del local) y te propone **3 alternativas de dimensiones** que cumplen tus objetivos, scoreadas por 13 criterios acústicos y prácticos.

19.1 Cuándo usar la pestaña Predicción

- Vas a diseñar una sala desde cero y querés un punto de partida razonable (en vez de tirar números a ojo).
- Diseñaste algo y querés saber cómo se compara contra ratios clásicos validados (Bolt, Bonello, Louden).
- Querés explorar qué pasa si cambiás capacidad, restricciones de planta o uso.

19.2 Inputs — describir el recinto

El panel tiene 4 grupos de controles:

1. Uso del recinto

Control	Qué hace
Uso	Combo con 8 presets: Sala de conferencias, Aula, Estudio control room, Estudio live room, Home theater, Sala de música cámara, Sala sinfónica, Sala polivalente. Cada uno tiene RT60 objetivo y V/persona típicos basados en Beranek / ANSI / Long.
Programa	Combo dependiente (voz hablada / amplificada / música acústica / cine, etc.).
Prioridad	Slider Inteligibilidad ↔ Envoltura. Afecta los pesos del scoring.

2. Audiencia

Control	Qué hace
Capacidad m ² por persona	Cantidad de personas. Por defecto auto (depende del uso: 0,80 m ² /p para voz, 1,00 m ² /p para música, 1,20 m ² /p para cine). Desactivás “auto” para tipear un valor manual.
Área total	Calculada, sólo informativa.

3. Restricciones del local (opcional)

Control	Qué hace
Limitar planta Override altura	Caps de ancho máx y largo máx en metros. Por defecto los candidatos tienen muros entre 2,5 y 4,0 m — rango constructivo realista. Activar este check para forzar otro techo máximo (mayor o menor).
Paredes paralelas	“Permitir” (default) o “Evitar” — esto último agrega taper=0.15 al candidato.
Forma de techo	Plano / Inclinado / Abovedado.

Por qué 2,5-4 m por defecto: una jornada típica de albañilería coloca ~13 hiladas de ladrillo (≈ 10 cm cada una), o sea 1,3 m/día. Un muro de 5 m son 3 jornadas + apuntalamiento + riesgo de derrumbe. Si tu volumen objetivo daría una H más alta (por ej. 200 personas $\times 6$ m³/p $\rightarrow V=1200$ m³ y Bolt sugiere $H \approx 7,7$ m), el clamp lleva H a 4,0 m y reescala W y L preservando su proporción para mantener el volumen. Si efectivamente querés un techo de 10 m (sala sinfónica, iglesia), activá **Override altura** y subí el spinbox.

4. Objetivos acústicos (auto-llenado por uso)

Control	Qué hace
RT60 @ 500 Hz V por persona	Se rellena del preset; editable. Idem.

19.3 Apretar “Predecir” — qué pasa internamente

1. Se calcula el volumen objetivo: $V = \text{capacidad} \times V/\text{persona}$.
2. Se generan **3 candidatos** con ratios clásicos (Bolt 1,9 : 1,4 : 1, Bonello 1,59 : 1,26 : 1, Loudon 2,33 : 1,6 : 1), escalados para llegar a V y respetando las restricciones de planta/altura. **Clamp constructivo:** si la H del ratio textbook cae fuera de [2,5 ; 4,0] m (o del override del usuario), se recorta H y se reescalan W y L preservando su proporción para conservar V. Esto rompe deliberadamente el ratio L:W:H ideal a favor de una sala construible; la proporción L:W (la que más afecta la distribución modal lateral) se mantiene intacta.
3. Sobre cada candidato se corre un **FEM ligero en paralelo** (ThreadPoolExecutor con 3 workers, malla coarse, 40 modos). Cubre hasta ~125 Hz.
4. Se calculan los **13 sub-scores** y un total ponderado por grupo.
5. Se ordenan los candidatos por score y se muestran como **cards** debajo.
6. Si los 3 candidatos quedan dentro de ± 5 puntos, se agrega una **4ª card “Control negativo” (Cubo 1:1:1)** con border rojo punteado para que veas visualmente qué NO funciona.

Tiempo total: ~**4 segundos** con un QProgressDialog que muestra el avance.

19.4 Las cards — leer los resultados

Cada card está organizada en 5 secciones (algunas se ocultan según el uso):

MODAL (siempre visible — sin esto la sala no funciona acústicamente): - **Modos 30-125 Hz** y su distribución por bins de 5 Hz (buenos / grumos / huecos). - **Modos audibles ($Q > 30$)**: cuántos modos son tan poco amortiguados que se oírían individualmente como resonancia. - **Cobertura Schroeder**: cuántos modos hay debajo de la frecuencia de Schroeder. - **RT60 obj**: el α que la sala necesita para llegar al RT60 objetivo, traducido a tipo de material (ej: “ $\alpha = 0,13 \cdot \text{madera dura} / \text{panel rígido}$ ”).

VOZ (oculto para usos de música pura): - **STI** estimado (Bradley): $STI = 0,9482 - 0,1845 \cdot \ln(RT60)$. - **%Alcons** estimado (Peutz): $Alcons = 200 \cdot d^2 \cdot RT60^2 / V$, capeado a $9 \cdot RT60$ en campo reverberante. - **d_crit** vs receptor típico (media diagonal del piso).

MÚSICA (oculto para usos de voz puro): - **Soporte de bajos**: cantidad de modos < 80 Hz vs lo teóricamente esperable. - Nota: la calidez real (Bass Ratio) depende mucho de materiales; el proxy mide sólo el aporte geométrico.

PRÁCTICO (siempre): - **Forma**: L/W y H/W con etiquetas (“ok”, “túnel”, “pozo”, “pancake”). - **Planta**: aprovechamiento de la planta (audiencia vs área calculada). - **Construcción**: muros > 12 m, planta > 800 m², L/W > 5 — penalty para casos extremos.

ROBUSTEZ (siempre): - Margen del α requerido respecto al rango razonable [0,08 ; 0,30]. Si el margen es grande, la sala tolera variabilidad en la elección de materiales; si es chico, es “frágil”.

Cada sub-score aparece como un chip de color al final de cada sección: **verde** (≥ 80), **amarillo** (50-80), **rojo** (< 50).

19.5 Cómo se combinan los sub-scores

El **score total** es un promedio ponderado de los 5 grupos, con pesos condicionales por uso:

Voz: Modal 40 % + Voz 30 % + Música 0 % + Práctico 25 % + Robustez 5 %
Música: Modal 45 % + Voz 0 % + Música 20 % + Práctico 25 % + Robustez 10 %
Mixto: Modal 40 % + Voz 15 % + Música 10 % + Práctico 25 % + Robustez 10 %

Dentro del grupo **MODAL** las métricas están ponderadas: 40 % Bolt + 25 % RT60-feas + 20 % Modal-Q + 15 % Schroeder. Bolt-spacing pesa más porque es la única métrica modal que discrimina la geometría real (las otras dependen mayormente de V/RT60 que son inputs constantes para los 3 candidatos).

19.6 Aplicar un candidato

El botón **“Aplicar ▼”** de cada card abre un menú con dos opciones:

1. **Como parámetros (editable)**: setea los sliders de la pestaña Geometría (ancho, largo, alto, n_walls, taper, arch_height). Te lleva automáticamente a la pestaña Geometría con la sala lista para seguir editando.
2. **Como CAD (geometría fija)**: construye la malla del candidato y la inyecta como geometría externa (igual que un STL importado). Más rígido, pero queda exactamente como predijo.

Debajo del botón aparece una leyenda “Render: 0,12 s · 6,0×8,2×4,3 m” con el tiempo del último apply.

La card de **Control negativo (Cubo 1:1:1)** tiene el botón Aplicar **deshabilitado** (con tooltip explicativo) — esa card existe sólo como referencia visual de qué NO usar.

19.7 Evaluar tu diseño actual

Debajo de “Predecir” hay un segundo botón: **“Evaluar mi diseño actual”**. Toma la geometría que tenés diseñada en la pestaña Geometría (ancho, largo, alto, taper, arch_height — todo lo que hayas tocado) y la corre por el **mismo pipeline de scoring**.

Útil cuando: - Diseñaste una sala “a ojo” y querés validar si los 13 criterios la aprueban. - Importaste un CAD y querés saber qué tan buena es acústicamente antes de gastar tiempo simulando. - Iteraste con sliders y querés comparar versiones (apretás “Evaluar” varias veces a medida que ajustás).

El resultado se muestra como **una card individual** con el mismo formato que los candidatos predichos. Si después querés ver alternativas, apretás “Predecir” y aparecen las 3 sugerencias clásicas — podés comparar visualmente cuál te conviene más.

19.8 Atajos

- **Enter** dentro de la pestaña Predicción dispara “Predecir”.
- Los presets de “Uso” auto-lleñan RT60 objetivo y V/persona. Podés tipear valores manuales después.
- Activá “Limitar planta” sólo si efectivamente tenés un local con dimensiones fijas — sino el escalado libre suele dar mejores ratios modales.

19.9 Casos de uso típicos

Diseñar una sala de música de cámara para 80 personas, sin restricciones: 1. Uso = “Sala de música cámara”, Capacidad = 80, todo lo demás por default. 2. “Predecir” → Bolt sale ~70 puntos, Bonello ~70, Louden ~71. El control negativo (cubo) aparece automáticamente porque los 3 son parecidos. 3. Apretás “Aplicar como parámetros” en Louden → te lleva a Geometría con sala ~9 × 13 × 6 m.

Validar un diseño existente: 1. Diseñás en Geometría con sliders → 8 × 10 × 4 m, n_walls=4. 2. Vas a Predicción, ajustás “Uso” y “Capacidad” al uso real previsto. 3. “Evaluar mi diseño actual” → ves los 13 sub-scores. Si Bolt-spacing < 20, hay distribución modal mala; pensá en agregar taper o cambiar el ratio. 4. Compará apretando después “Predecir” — si tu score es similar al ganador, vas bien; si está 10+ puntos abajo, hay margen de mejora.

Restricción de planta apretada: 1. Activás “Limitar planta” con ancho máx = 5 m, largo máx = 8 m. 2. “Predecir” → los 3 candidatos pueden caer abajo de 70 puntos (porque sus ratios óptimos no caben). El sub-score “Fit” baja a 30 si tuvieron que recortar. 3. Es información válida: el local elegido **no puede** producir una sala óptima. Considerá buscar otro local o aceptar el trade-off.

20. Distribución del programa (.exe)

Para pasarle el programa a alguien que **no tiene Python instalado**, se empaqueta en un .exe autocontenido con PyInstaller.

Generar el ejecutable

build.bat

Deja todo en dist\Prototipo1\:

Archivo	Qué es
Prototipo1.exe	el programa (doble click, no necesita Python)
MANUAL.pdf	este manual
ejemplo.room	sala 5×4×3 de muestra
LEEME.txt	instrucciones de 5 párrafos para el destinatario
_internal\	dependencias, DLLs y los 19 JSON de materiales

El destinatario copia **la carpeta entera** y hace doble click en el .exe. La primera vez Windows puede mostrar SmartScreen: “Más info” → “Ejecutar de todos modos” (el ejecutable no está firmado digitalmente).

Flujo completo recomendado

1. `build.bat` — compila (unos 8-10 minutos).
2. `python verify_distribution.py` — chequea que estén el `.exe`, los materiales, PyQt5 y que el tamaño sea razonable.
3. `python test_distribution_smoke.py` — copia el bundle a otra carpeta, lo lanza y verifica que sobreviva 15 s. Detecta el caso clásico de “anda en la carpeta fuente por casualidad”.
4. Prueba visual humana: abrir el `.exe`, ver 19 materiales, cargar `ejemplo.room`, correr el FEM.
5. `python pack_distribution.py` — genera `Prototipo1_vX.YY.zip` (el número de versión se lee del changelog de este manual).

Tamaño

La carpeta pesa **~1,0 GB** y el ZIP **~414 MB**. Es grande porque incluye el intérprete de Python, Qt y las librerías numéricas. Lo que no se puede sacar sin romper nada:

- **~370 MB de `mkl*.dll`** — el BLAS de numpy/scipy. Son variantes de despacho por tipo de CPU; sacarlas hace que el programa falle en máquinas con otro juego de instrucciones.
- **~86 MB de `gms4-4.15.dll`** — el mallador boundary-fitted opcional.
- **~81 MB de PyQt5** — la interfaz.

`build.bat` ya excluye ~490 MB de dependencias que Anaconda arrastra y el proyecto no usa (SDK de AWS, dashboards, JIT, navegador embebido). Si alguna vez el bundle vuelve a superar 1,4 GB, `verify_distribution.py` avisa: se coló peso muerto de nuevo.

Si el `.exe` no arranca

“DLL load failed while importing QtCore” al abrirlo. El bundle salió sin las DLLs del runtime de Qt. Pasa porque Anaconda no las guarda junto a PyQt5 sino en `<entorno>\Library\bin\` (con nombre `Qt5Core_conda.dll`), y PyInstaller las busca recorriendo el PATH: si se compila desde una consola sin el entorno conda activado, no las encuentra y **las omite sin avisar** — el build igual termina diciendo “BUILD OK”.

Desde v2.22 `build.bat` arma ese PATH por su cuenta, así que no debería volver a pasar. Para comprobarlo en un bundle ya generado, tiene que haber al menos una DLL de Qt en `_internal\`:

```
dir /s /b dist\Prototipo1\_internal\Qt5*.dll
```

Si no aparece ninguna, recompilá. `verify_distribution.py` también lo verifica.

Cómo compartirlo

No subas el `.exe` ni el ZIP al repositorio. GitHub bloquea archivos de más de 100 MB, y aunque entraran quedarían en el historial para siempre, inflando el repo de manera irreversible. El `.gitignore` ya excluye `dist/`, `build/`, `*.exe` y `*.zip` justamente por eso.

La vía correcta es **GitHub Releases**: acepta hasta 2 GB por archivo, no toca el historial del repositorio y le da a quien lo descargue un link directo asociado a un número de versión.

```
gh release create v2.22 Prototipo1_v2.22.zip --title "... " --notes "... "
```

Alternativas si preferís no usar GitHub: WeTransfer (2 GB) o Google Drive.

macOS

PyInstaller **no compila cruzado**: no se puede generar un `.app` de Mac desde Windows. Para ese caso está `Prototipo1_Mac.zip`, que es un paquete *correr desde fuente*: el destinatario instala

Python 3 y lanza `run.command`. Para un `.app` real de doble click hay que correr el build en una Mac.

Apéndice — Optimizaciones internas v2.3

Para usuarios técnicos que quieran inspeccionar el código:

Evaluación vectorizada del campo (`acoustic_fem.FieldEvaluator`)

Antes (loop Python):

```
def evaluate_many(self, field_nodal, points):
    out = np.full(len(points), np.nan, dtype=complex)
    for i, x in enumerate(points):
        e, N = _locate_one(self.v0, self.A_inv, self.tets, x)
        if e is not None:
            out[i] = complex(np.dot(field_nodal[self.tets[e]], N))
    return out
```

Para `len(points) = 62 500` y `Ne = 14 400`, esto generaba 62 500 llamadas a `_locate_one`, cada una computando barycentric contra TODOS los tetraedros = $O(N_p \cdot N_e)$ bajo el interpreter de Python. En la práctica: **15-25 segundos** por refresh del campo.

Ahora (KDTree + numpy vectorizado):

```
# En __init__: precomputar centroides + cKDTree.
self._centroids = self.nodes[self.tets].mean(axis=1)
self._tree = cKDTree(self._centroids)

# En evaluate_many: para cada punto, K=12 candidatos vía KDTree, después
# barycentric vectorizada en arrays (Np, K, 3, 3).
_, cand = self._tree.query(points, k=12)
stu = np.einsum("pcij,pcj->pci", A_inv[cand], rel_pc)
# ... un solo argmax por punto, sin loops.
```

Resultado medido: **50-170 x más rápido**, diferencia numérica $< 1 \times 10^{-15}$ (puro redondeo IEEE-754). El algoritmo es idéntico, sólo cambia cómo se busca el tet contenedor. Si más del 1 % de los puntos no se localiza con $K = 12$ (puede pasar en bordes con tetraedros muy alargados), se reintenta con $K = 48$ solo para esos puntos.

Importación CAD con `QProgressDialog`

El flujo de import en `main.MainWindow._open_cad_import()` ahora:

1. Abre un `QProgressDialog` modal con `setMinimumDuration(200)` — sólo aparece si la importación realmente tarda.
 2. Pasa un callback `progress(msg)` a `geom_import.load_geometry` para que mensajes de `gmsh/trimesh` aparezcan en vivo.
 3. Mide cada fase con `time.time()` y al final reporta el desglose en la barra de estado: `load 230 ms · scale 5 ms · diagnose 410 ms · render 12 ms`.
 4. Para mallas limpias (`diag.ok == True`) **salta el QMessageBox de confirmación**: antes había un “¿Usar esta geometría? Sí/No” que el usuario siempre aceptaba.
-

Apéndice — ¿Por qué FEM “a mano” y no FEniCS?

Es la pregunta natural que se hace cualquiera con background numérico al ver el código: si existen librerías como **FEniCS**, **deal.II** o **MFEM** que ya implementan FEM, ¿por qué este proyecto escribe todo con numpy y scipy? ¿Es realmente FEM lo que hace?

El método no es la librería

FEM es un método matemático, no una librería. FEniCS es *una* implementación del método; este proyecto es *otra*. El resultado numérico es el mismo cuando ambos resuelven el mismo problema con los mismos elementos.

Cualquier libro clásico (Zienkiewicz-Taylor, Hughes, Reddy) describe FEM como **seis pasos canónicos**. Cada implementación tiene que hacerlos. Mirar dónde está cada paso en este proyecto:

#	Paso del método FEM	En el código
1	Forma fuerte de la EDP	$\nabla^2 p + k^2 p = -i\omega \rho_0 q(x)$ (Helmholtz)
2	Forma débil (Galerkin + integración por partes)	derivada en acoustic_fem_explicado.md §1.3
3	Discretización del dominio en elementos	acoustic_mesh.build_volume_mesh produce tetraedros
4	Funciones de forma N_j por elemento	lineales en P1, codificadas en Vinv[:, 1:4, :]
5	Ensamblaje de K y M globales	acoustic_fem.build_KM con coo_matrix
6	Resolución de $K \cdot \phi = \lambda \cdot M \cdot \phi$	scipy.sparse.linalg.eigsh con shift-invert

Los seis pasos están todos. Lo único que se delega a librerías es: - numpy.linalg.inv / det para invertir matrices 4×4 (aritmética). - scipy.sparse para almacenamiento sparse (mecánico). - scipy.sparse.linalg.eigsh para resolver el problema de autovalores generalizado (álgebra lineal pura).

Eso no es delegar FEM. Es delegar **álgebra lineal**. Igual que hacer una eliminación gaussiana con calculadora: la calculadora hace la aritmética, el método lo seguís pensando vos.

¿Qué automatiza FEniCS que este proyecto no tiene?

Cinco cosas que vale la pena saber:

1. **Un DSL para la forma débil.** En FEniCS escribís `a = dot(grad(u), grad(v)) * dx` y la librería **compila** eso a código C++ que arma K. Acá lo escribimos a mano con `np.einsum("eij,ekj->eik", grads, grads)`. Más verboso, pero ves exactamente qué matriz se calcula y cómo.
2. **Elementos de orden superior** (P2, P3, ...). FEniCS los cambia con una línea: `V = FunctionSpace(mesh, "Lagrange", 2)`. Acá habría que reescribir varias funciones — los gradientes dejarían de ser constantes dentro del tet y la cuadratura cambiaría.
3. **Mallado adaptativo** y mallas mixtas (tet + hex + prisma).
4. **Condiciones de borde de impedancia ensambladas en matriz de superficie.** Acá usamos damping por modo (ξ_n derivado del RT60), que es más simple pero menos general.
5. **Solvers paralelos** (PETSc, MUMPS, MPI). Para problemas de millones de DOFs en clusters.

¿Por qué entonces no usar FEniCS?

Cuatro razones que justificaron la elección de implementación a mano para este proyecto:

1. **El problema concreto es pequeño.** Helmholtz + paredes rígidas + P1 + monopolos puntuales en salas de 10^3 – 10^5 nodos. eigsh tarda segundos. Cualquier ganancia de performance de FEniCS sería invisible.
2. **Cero dependencias pesadas.** FEniCS arrastra DOLFINx (C++ core), UFL, FFCx, PETSc, MPI. En Windows + Anaconda son horas de pelea con builds. Este proyecto vive con numpy + scipy + matplotlib + PyQt5, lo que ya tiene cualquier instalación científica de Python.
3. **Transparencia pedagógica.** Para un usuario que quiere *entender* qué pasa numéricamente, una caja negra esconde la mecánica. Con NumPy directo se puede:
 - imprimir K y M y miraras;
 - modificarlas a mano para experimentar;
 - saber exactamente dónde nace cada coeficiente.

Esto fue determinante para implementar las **cuatro capas de robustez** que se agregaron en v2.9 (filtro de slivers, simetrización forzada de K/M, retry de Lanczos con sigma dinámico, métricas de calidad de malla). Ninguna se puede hacer con la misma precisión cuando el ensamblaje está dentro de una librería compilada.

4. **Portabilidad.** Corre en cualquier máquina con Python — sin builds, sin admin, sin GPU. Para un prototipo o un MVP, eso es enorme.

¿Cuándo conviene cambiar a FEniCS?

Si el proyecto crece hacia alguno de estos escenarios, sí valdría la pena migrar:

- Elementos **P2 o superiores** para precisión $O(h^3)$ en alta frecuencia.
- Mallas con $> 10^6$ DOFs (cuando `scipy.sparse` no entra en RAM).
- **Impedancia angular-dependiente** en paredes con absorción ensamblada en una matriz C de superficie (no por modo).
- Acoplamiento **estructura-fluido** (paneles vibrantes), termoacústica, problemas multifísicos.
- **Paralelismo distribuido** en clusters.

Mientras nada de eso aparezca en los requerimientos, numpy + scipy es la elección correcta.

La distinción “método vs implementación”, en general

Es una distinción importante en ingeniería numérica. Algunos ejemplos paralelos:

Método matemático	Librería conocida	Implementable a mano
FEM	FEniCS, deal.II, MFEM	sí (este proyecto, NumPy/SciPy)
FFT	FFTW, MKL	sí (Cooley-Tukey en 30 líneas)
Quicksort	stdlib de cualquier lenguaje	sí, 20 líneas
Newton-Raphson	<code>scipy.optimize</code>	sí, 5 líneas
Runge-Kutta	<code>scipy.integrate.solve_ivp</code>	sí, 15 líneas (RK4)

Usar la librería no te hace más “real” en el método. Te hace más rápido en implementación y, a veces, en performance. El método **es el mismo**.

En una línea

FEM define qué hacer matemáticamente. FEniCS automatiza cómo escribirlo en código. Este proyecto hace el “qué” a mano con NumPy/SciPy en lugar de delegar el “cómo” a una librería compilada.

Prototipo 1 — Modelador de Recintos 3D con Simulación Acústica FEM
Manual de Usuario · v2.12 · 30 de Mayo de 2026

Cambios v2.0 (20 de mayo): import CAD (STL/OBJ/PLY/STEP/IGES/glTF), motor de mallado boundary-fitted con gmsh, router automático con badge UI, diálogo de reparación guiada, persistencia .room v3.

Cambios v2.1 (21 de mayo): router en modo **best-effort** con fallback automático gmsh → voxel. El sistema intenta siempre el motor más riguroso primero; cae a voxel solo cuando gmsh no puede mallar la geometría, reportando la razón al usuario.

Cambios v2.2 (21 de mayo): (1) diálogo de **escala y orientación** al importar, con sugerencia automática de unidad por tamaño y eje “up” por extensión del archivo; (2) función `apply_up_axis` que resuelve la conversión Y-up ↔ Z-up (corrige el caso típico del OBJ que aparece con la pared como piso); (3) reparador de mallas vectorizado y reescrito en un solo pase (~ 50-100× más rápido para mallas grandes); (4) **indicador de ejes** en esquina inferior derecha del visor 3D, con cuadrados X / Y / Z clicables o accesibles por Ctrl + Shift + Alt + X/Y/Z para fijar la rotación a un eje mundial.

Cambios v2.3 (22 de mayo):

1. **Materiales por cara, estilo EASE.** El esquema clásico de un material por zona (piso/techo/paredes) se reemplaza por un diálogo dedicado (botón **Materiales... Materiales...**) que detecta automáticamente cada región planar conexa y permite asignarle un material independiente. Las asignaciones persisten al cerrar el diálogo y se serializan en .room v4 con firma estable por grupo. Detalles en sección 6.3.
2. **Campo 3D 50-170× más rápido.** `FieldEvaluator.evaluate_many` se vectorizó con `scipy.spatial.cKDTree` + `numpy.einsum`. Resolución 50 (62 500 puntos): antes 15-25 s, ahora ~ 280 ms. Diferencia numérica < 1e-15. La forma modal en resolución 70 (170 000 puntos) tarda ahora ~ 740 ms. Detalles en sección 18.
3. **Importación CAD con feedback visible.** Un `QProgressDialog` modal muestra cada fase (load → scale → diagnose → repair → render) con un Cancelar funcional. La barra de estado al terminar reporta el desglose de tiempos por fase. Para mallas limpias, se saltea el `QMessageBox` de confirmación redundante.
4. **.room v4.** El archivo de proyecto guarda ahora la asignación de materiales por grupo (`acoustic.face_materials.assignments`). Los .room v2/v3 siguen abriéndose sin problema (los campos nuevos son opcionales).
5. **Suite de benchmarks benchmark_v2.py.** Script headless que mide 7 bloques (FEM, campo 3D, comparativa loop/KDTree, agrupación de caras, import CAD, RT60, memoria) y deja un reporte en `BENCHMARK_RESULTS.md`. Detalles en sección 18.

Cambios v2.4 (22 de mayo): el gráfico de RT60 se reemplaza por un **diálogo comparativo multi-curva** (sección 10):

1. **Tres métodos de predicción:** Sabine (clásico), Eyring (corrige sesgo de α alta), Fitzroy (corrige asimetría direccional de la absorción).
2. **Tres métricas de decaimiento:** T20, T30, T60. En predicciones teóricas matemáticamente equivalentes (decaimiento exponencial puro), pero se exponen como alias por compatibilidad con el lenguaje de mediciones.
3. **Curvas agregables y quitables:** el usuario arma combinaciones de método + métrica y las superpone para comparar. Cada curva guarda un snapshot numérico, así se pueden cambiar materiales entre clics y comparar variantes (Sabine T60, Sabine T60 #2, etc.).

4. **Casillas de visibilidad** por curva (ocultar sin borrar) y exportación a PNG, SVG, PDF, CSV (separador ; , decimal ,) y TXT (tab, decimal .).
5. **Código de colores determinístico**: azul = Sabine, rojo = Eyring, verde = Fitzroy. Estilos de línea: sólida = T60, discontinua = T30, punteada = T20.

Cambios v2.5 (23 de mayo):

1. **Diálogo de RT60 simplificado**. Se removieron el método Fitzroy y las métricas T20/T30 de la UI por pedido del usuario (T20=T30=T60 en predicciones teóricas; Fitzroy era ruido visual). Quedan **Sabine** y **Eyring** sólo en **T60**. El código de Fitzroy se conserva en `face_materials.compute_fitzroy_rt60_per_face` por si se quiere re-habilitar.
2. **Botón “Materiales” sin emoji**. Antes tenía un ícono de paleta; ahora dice Materiales...
3. **Audio +6 dB con saturación suave**. Reescrita la cadena de `audio_utils.apply_frf_filter`:
 - Normalización al **peak $\pm 1,0$** (antes $\pm 0,85 \rightarrow +1,4$ dB extra).
 - **Soft-clipping tanh** con `drive = 2,5` $\rightarrow$ boost equivalente a **$\sim 2\times$ amplitud RMS** sin distorsión audible para ruido rosa filtrado.
 - Escalado final a $\pm 0,98$ para dejar headroom del DAC.
4. **Anti-pop al finalizar la reproducción**. Tres mecanismos en cascada:
 - **Fade-in 10 ms** y **fade-out 50 ms** lineales en la señal antes de escribirla al WAV.
 - **100 ms de silencio** apendidos al final del WAV — el último sample que el OS reproduce siempre es 0, eliminando el chasquido del buffer de hardware.
5. **Herramienta de cobertura de materiales** (`check_materials_coverage.py` + `MATERIALS_COVERAGE.md`). Script que cruza un listado externo de materiales contra la librería interna usando matching difuso con stemming español + diccionario de sinónimos en/es. Reporta MATCH / SIMILAR / FALTA con top-3 candidatos y plantilla JSON para agregar los que faltan.
6. **Bug fix**. Resuelto un `UnboundLocalError` en `acoustic_panel._build_ui` (variable local `fm = QFormLayout(...)` sombreaba al `import import face_materials as fm` y rompía la inicialización del `FaceMaterialMap`). Renombrada a `fmode`. Resuelto también el ruido de consola “Unknown property font-variant-numeric” (propiedad CSS no soportada por Qt5) removiéndola de `controls.py` y `shape_dialog.py`.

Cambios v2.6 (24-25 de mayo):

1. **Pestaña Predicción** (`prediction.py` + `prediction_panel.py`). Tercera pestaña a la derecha de Acústica. Inputs: uso, audiencia, restricciones, objetivos acústicos. Genera 3 candidatos con ratios clásicos (Bolt, Bonello, Louden) escalados al volumen objetivo, corre FEM ligero en paralelo (~ 4 s), y los muestra como cards scoreadas por **13 sub-scores** agrupados en 5 categorías:
 - **MODAL**: RT60 feasibility, Bolt-spacing (bins 5 Hz), Modal Q audibility, Schroeder coverage.
 - **VOZ**: STI (Bradley), %Alcons (Peutz), distancia crítica.
 - **MÚSICA**: Bass support proxy (geométrico — la BR real depende de materiales).
 - **PRÁCTICO**: Volumen vs target, aspect ratio L/W y H/W, ajuste a restricciones, aprovechamiento de planta, constructabilidad.
 - **ROBUSTEZ**: margen del α requerido respecto a [0,08 ; 0,30].

Pesos condicionales por uso (voz / música / mixto). El botón **“Aplicar ▼”** de cada card mueve los sliders de Geometría (“Como parámetros”) o inyecta la malla como geometría externa (“Como CAD”). Si los 3 candidatos quedan dentro de ± 5 puntos, aparece una 4ª card **Control Negativo (Cubo 1:1:1)** con border rojo para enseñar visualmente qué NO usar. Detalles en sección 19.

2. **“Evaluar mi diseño actual”** — botón secundario en la pestaña Predicción. Toma la geometría diseñada en la pestaña Geometría y la corre por el mismo pipeline de scoring. Útil para validar diseños propios contra los 13 criterios objetivos, o iterar y comparar versiones.
3. **Auto-tuner de densidad FEM** (`mesh_router.auto_density`). Cuando el motor está en

“Automático”, el sistema calcula la densidad necesaria para cubrir hasta $f_{\text{Schroeder}}$ en un budget de tiempo (5 s para shoebox simple, 10 s para CAD o curvas). Si no entra, aparece un diálogo “Cobertura parcial / Cobertura completa (~Y s) / Cancelar”. Prioridad: validez antes que velocidad. Sección 7.

4. **Estimaciones de tiempo honestas.** Recalibración de los throughput estimates en mesh_router: voxel ~7 000 tets/s (antes 10 000), gmsh ~12 000 tets/s (antes 15 000), `_GMSH_INIT_OVERHEAD_S` = 1,0 (antes 0,5), nuevo `_SAFETY_FACTOR` = 1,30 multiplicativo. Antes el diálogo decía “~6 s” y tardaba ~12 s; ahora dice ~10 s y tarda 10-13 s.
5. **Skip de gmsh para techos curvos paramétricos.** Si la sala tiene `arch_height > 0` con techo “arch” o “gable”, el router va directo a voxel (la malla con subdivisiones para suavizar la curva tiene T-junctions que romperían gmsh con PLC Error). Voxel usa tests inside/outside y no le importa la topología del surface mesh.
6. **Leyendas de tiempo bajo botones de cómputo.** Cada botón pesado (Calcular modos (FEM), Predecir, Aplicar ▾ de cards, Importar CAD...) tiene una etiqueta persistente debajo “Último: X,XX s · contexto”. Verde clarito al recién terminar, fade a gris pasivo 1,5 s después. Implementado en `timed_button.TimedButton`.
7. **Nuevos colormaps de la nube 3D** (`acoustic_viewer`):
 - **Rainbow 7 paradas** (`colormap_rainbow`) para “Presión $|p|$ ”: azul, celeste, turquesa, verde claro (centro), amarillo, naranja, rojo. 7 colores perceptualmente equidistantes (no HSV lineal — en HSV puro el amarillo cae como chartreuse).
 - **Signed Vivid** (`colormap_signed_vivid`) para “Forma modal”: azul vibrante → **gris medio** (89, 89, 89) → rojo vibrante. Reemplaza el blanco central que saturaba la imagen a alta resolución. Curva de saturación $\sqrt{|t|}$ para que los valores moderados ya muestren color claro.
8. **Slice plane interactivo visible para shoebox.** Bug-fix: la preview translúcida del plano de corte (`SlicePlanePreview`) era invisible para una caja perfecta porque sus 4 vértices coincidían exactamente con las aristas wireframe rosa de las paredes — z-fighting. Solución: el quad se encoge un 2 % hacia adentro (`SHRINK_RATIO` = 0,02), opacidad subida de 0,28 → 0,40, y se agrega un **borde wireframe cian** (`GLLinePlotItem`, ancho 2,5 px) que siempre es visible aunque el fill quede detrás de algo.
9. **Fixes de UI panel.** Ancho del tab subido de 360-440 px a 380-500 px. Botones de Acústica (Añadir/Editar/Quitar/Duplicar) y CAD (Importar / Volver a paramétrica) con `setMinimumWidth(0)` para que no se recorten en panel angosto. Receptor XYZ con spinboxes capados a 95 px. Restricciones de Predicción reorganizadas en `QFormLayout` vertical (una fila por control) en vez de `HBoxLayout`. Repair dialog: lista de huecos capada a 140 px de alto con `addStretch(1)` al final para que no estire los grupos.
10. **Bug-fix del auto-tuner / fallback gmsh.** Antes, cuando el auto-tuner elegía gmsh, hacía `override = "gmsh"`, lo que desactivaba el fallback automático a voxel del router (que asume que si el usuario forzó gmsh manualmente lo quiere sí o sí). Resultado: gmsh fallaba con PLC Error y el error se propagaba al usuario. Ahora: si auto-tuner elige voxel, sí se fuerza; si elige gmsh, se mantiene `override = "auto"` para preservar el fallback.

Cambios v2.7 (25-26 de mayo): sesión grande de saneamiento técnico + UX. Diez ítems agrupados en cuatro bloques:

A. Documentación y configuración

1. **README.md, PROYECTO.md, setup.py, verify_setup.py** actualizados de cero: ahora reflejan las tres pestañas (Geometría / Acústica / Predicción), el mapa real de los ~30 módulos, todas las dependencias (scipy, gmsh, trimesh, matplotlib), y los archivos críticos. README cita

run.bat (el lanzador real) en lugar de quick_start.bat (que no existía). setup.py ahora lee dependencias desde requirements.txt para mantener una sola fuente de verdad.

2. **Limpieza de artefactos LaTeX.** Se borraron 11 archivos auxiliares (.aux, .log, .toc, .out, .fdb_latexmk, .fls, .synctex.gz) de MANUAL.tex. Son regenerables con pdflatex; ocupaban ~600 KB en el repo sin agregar información.

B. Performance: vectorización del voxel mesher

3. **acoustic_mesh.points_inside_surface vectorizado** — *cuello de botella histórico resuelto*. El test point-in-polyhedron de Möller-Trumbore antes hacía un bucle Python sobre los puntos (un punto contra todos los triángulos por vez). Para una sala $6 \times 8 \times 3$ m con $n_{pm}=2.5$ (14 400 centroides $\times$ 12 triángulos): **1 141 ms**. Reescrito en una sola expresión broadcasted ($N_p, N_t, 3$) con chunking de memoria a 10 M pares — ahora **25,8 ms** (44× más rápido). El bucle triple de cand_tets también se vectorizó vía np.meshgrid + indexación con HEX_T0_TETS. Speedup medido: **mediana 44×, mínimo 1,3× (CAD con $N_t > 1\,000$), máximo 85× (shoebox simple)**. Verificación bit-exact en 14 casos (paramétrico, no-convexos L/U/+, gable, shed, taper+twist, OBJ icosphere roundtrip): verify_voxel_equivalence.py. Frecuencias modales coinciden a $r_{tol} < 1e-10$. Detalle en BENCHMARK_RESULTS.md sección B8.
4. **Impacto al usuario:** para una sala $6 \times 8 \times 3$ con $n_{pm} 2.5$, Calcular modos (FEM) pasó de ~1,3 s a ~180 ms end-to-end (mesh + K/M + Lanczos). La app se siente instantánea para uso paramétrico cotidiano. Para CAD pesado ($N_t > 1\,000$) el speedup baja a 1,3–3× porque el bucle Python ya pesaba poco; ahí el router elige gmsh automáticamente (que tiene su propio camino).

C. UX del FEM: drag de fuentes/receptor + flujo del auto-tuner

5. **Diálogo de “cobertura parcial / completa / cancelar” eliminado.** Política nueva: validez antes que velocidad, sin preguntar. auto_density ahora se llama siempre con `time_budget_s=float('inf')` desde acoustic_panel, garantizando cobertura completa hasta `f_Schroeder`. Ya no hay diálogo modal interrumpiendo el flujo. Sección 7.
6. **QProgressDialog pulsante durante el FEM.** Aparece solo si el cálculo tarda más de 200 ms (`setMinimumDuration(200)` evita el flash en cálculos rápidos). Barra indeterminada (no porcentual — el tiempo por fase varía mucho), label dinámico (Mallando volumen... → Ensamblando K, M... → Resolviendo X modos... → Post-procesando...), sin botón Cancelar (porque eigsh no es interrumpible y mentir es peor que esperar). Sección 7.
7. **Timer “Último: X s” cubre todo el pipeline visible.** Antes detenía después del FEM pero antes del refresh del combo de modos / RT60 / slice 2D, lo que daba números más bajos que la percepción real. Ahora detiene después de `_update_slice()`, así “Último: X s” coincide con “click → resultado visible en pantalla”.
8. **Estimaciones de tiempo recalibradas.** El throughput voxel pasó de 7 000 a 50 000 tets/s en `mesh_router.py` para reflejar el post-G. Las estimaciones solo afectan los mensajes de log (la decisión es siempre cobertura completa con `budget=inf`).
9. **Shift+drag mucho más confiable** — tres bugs corregidos:
 - **Sincronización viewer ↔ panel:** `_refresh_sources_list` ahora llama a `_sync_source_positions` al final. Antes, cargar un .room con fuentes / importar CAD / borrar fuente del medio / duplicar fuente dejaba al viewer._source_positions desincronizado, y el picking del Shift+drag no encontraba las fuentes. Una línea arregla 6 callsites distintos.
 - **Drag fantasma por release perdido:** `mousePressEvent` ahora resetea `_dragging_source_idx = -1` siempre que el press no sea Shift+Left. Antes, si el `mouseRe-`

leaveEvent se perdía (foco fuera del widget, ventana minimizada), el siguiente click hacía mover una fuente fantasma.

- **Drag del receptor sin scene-graph thrashing:** ReceiverMarker.update(pos) ahora actualiza el GLLinePlotItem in-place con setData en lugar de removeItem + crear nuevo + addItem por frame. Antes pyqtgraph reconstruía el scene graph a 60 Hz durante el drag, especialmente molesto con CAD pesado cargado. Mismo patrón que SourceMarkers.set_positions ya tenía.

10. **Ctrl + Shift + drag para mover en altura (nuevo).** El Shift+drag plain sigue siendo XY plane (z fijo) — sin cambios. Sumado un modo “solo Z”: cuando se mantiene Ctrl también, la fuente o receptor se mueve únicamente en altura (x, y fijos en la posición original). Implementado proyectando el cursor sobre una línea vertical que pasa por la posición original (`_pick_vertical_line` resuelve $\min |line(t) - ray(s)|^2$). Si la cámara apunta exactamente hacia abajo, la altura no se puede inferir y el move se ignora silenciosamente. Sección 5.

D. Red de seguridad

- **smoke_test_shift_drag.py** — 7 tests state-level (headless con QApplication, sin GUI visible) que verifican los flows de Shift+drag, la sincronización viewer-panel, y el comportamiento del modo Z (Ctrl+Shift). Todos pasan después de los cambios.
- **verify_voxel_equivalence.py** — 14 casos bit-exact. Compara la implementación original (inlineada) contra la vectorizada actual. Cubre paramétrico, no-convexos, gable, shed, OBJ icosphere roundtrip. Reusable como red de regresión.
- **bench_voxel_mesh.py + bench_voxel_extended.py** — baseline guardado en `baseline_voxel_mesh.json` (con hash md5 determinista) y side-by-side con la implementación original. Reusables.
- **acoustic_fem_explicado.md** — documento técnico bloque-por-bloque del solver FEM (matemática + líneas clave + flujo típico).
- **instrucciones_p2_intermedio.txt** y **contextoparaclaudemediop2.txt** — runbook para la futura implementación de FEM P2 selectivo en la rama gmsh + protocolo de trabajo pre-scriptivo para mantener cambios seguros sobre el proyecto.

Cambios v2.8 (27 de mayo): saneamiento + dos features de export y un fix UX:

A. Limpieza profunda — eliminación del solver BEM

1. **BEM removido por completo del proyecto.** El módulo `acoustic_bem.py` (BEM por colocación P0 sobre la malla triangular) y `bem_modal.py` (BEM shoebox de referencia) ya no estaban enganchados desde la UI desde v2.6 — eran legacy didáctico. Se borraron también `benchmark.py` (comparativa FEM vs BEM, perdía sentido), `fem_bem_completo.tex/.pdf` (paper técnico), y todas las referencias en `acoustic_analysis.py` (BEMResult, `run_bem_sweep`, `run_bem_frf`, `import`), `acoustic_panel.py` (atributo `bem_result`), docstrings de `acoustic_mesh.py` y `sources.py`, `setup.py`, `verify_setup.py`, `README.md`, `PROYECTO.md`. La función `schroeder_frequency` se mantiene — el concepto físico (frontera campo modal / campo difuso) no requiere BEM. El pipeline ahora es 100 % FEM.

B. Export de datos numéricos en los tres diálogos de gráfico

2. **CSV y TXT en FRF, heatmap y RT60.** Hasta v2.7 los tres diálogos (FRFDialog, Slice-HeatmapDialog, RTComparisonDialog) exportaban sólo imagen (PNG / SVG / PDF). Ahora también escriben los valores numéricos del gráfico:
 - **FRF:** columnas `freq_hz`; `spl_db`; `abs_H_pa`; `phase_deg`.
 - **Heatmap:** formato largo `x`; `y`; `amplitud_modal` (forma modal) o `x`; `y`; `presion_pa`; `spl_db` (presión). Celdas enmascaradas se omiten.

- **RT60:** columna `banda_hz` + una columna `<curva>_s` por cada curva activa. Bandas faltantes en alguna curva quedan como celda vacía.
- CSV usa `;` como separador y coma decimal con codificación UTF-8 BOM (abre limpio en Excel-es); TXT usa tabulador y punto decimal (universal, parsea fácil con `numpy.loadtxt`). Implementado vía un helper `_write_tabular` compartido en `acoustic_panel.py`.

C. Predicción — rango constructivo de altura

3. **Clamp constructivo $H \in [2,5 ; 4,0]$ m por defecto.** Antes el predictor escalaba el ratio textbook (Bolt / Bonello / Louden) uniformemente al volumen objetivo, lo cual generaba muros de 5-8 m para salas grandes. En la práctica: 13 hiladas de ladrillo / jornada $\times 10$ cm = 1,3 m/día; un muro de 5 m son 3 jornadas + apuntalamiento + riesgo. Solución: nueva función `_clamp_height_constructive` en `prediction.py` que, después de `_scale_ratio_to_volume`, recorta H al rango y **reescala W y L preservando su proporción $W:L$** para conservar V_{target} . El ratio textbook $L:W:H$ se rompe deliberadamente; la proporción $L:W$ (la que más afecta la distribución modal lateral) se preserva.
4. **UI:** el control de la pestaña Predicción pasó de “Limitar altura” a “Override altura” para reflejar que el default ya es un cap implícito y este checkbox lo sobrescribe. El spinbox arranca en **6,0 m** (antes 4,0 — coincidía con el default y la activación no se notaba) y tiene tooltip explicando la razón constructiva.

D. Fix UX en importación CAD

5. **Diálogo “Cargando” ya no reaparece encima del modal de escala.** Causa raíz: `QProgressDialog.hide()` **no detiene** el `forceTimer` interno (el que decide cuándo auto-mostrar después de `setMinimumDuration`). Cuando `gi.load_geometry` era rápido (< 200 ms, caso usual), prog nunca llegaba a mostrarse durante la carga, así que `hide()` no hacía nada y el timer quedaba armado; mientras estabas mirando el diálogo de escala, el timer disparaba `prog.show()` por su cuenta encima del modal, dando la falsa impresión de que la app se colgaba. Fix: reemplazo `prog.hide()` por `prog.close()` (que sí cancela el `forceTimer` vía `reset()`) antes de cada modal (escala + reparación), y se crea una **nueva** `QProgressDialog` para las fases siguientes (diagnose/repair). El closure `_set_progress` captura prog por nombre, así que apunta automáticamente a la instancia nueva. `main.py:_open_cad_import`.
6. **Botón “No escalar” del diálogo de escala ahora hace lo que dice.** Antes llamaba a `self.reject()`, lo que el orquestador en `main.py` interpretaba como cancelación total del import (combinado con el bug anterior, cualquier movimiento abortaba). Ahora el diálogo tiene tres botones explícitos con roles separados: **Aplicar escala** (`AcceptRole` $\rightarrow$ factor elegido), **No escalar** (`ActionRole` $\rightarrow$ `accept()` con `factor=1,0` $\rightarrow$ sigue el import), **Cancelar import** (`RejectRole` $\rightarrow$ `reject()` $\rightarrow$ aborta). Esc / X siguen siendo cancelación real. `geom_scale_dialog.py:179-194` y `_on_skip()`.
7. **Bonus:** el branch de cancelación del diálogo de reparación (`MeshImportDialog`) ahora también dispara `cad_timer.fail("cancelado")` para que la leyenda bajo el botón “Importar CAD” refleje el cancel.

Cambios v2.9 (28-29 de mayo): sesión de documentación interna profunda + cuatro mejoras de robustez del solver FEM. Cero cambios en la API pública; todo retrocompatible.

A. Documentación interna — explicadores línea-a-línea y autoevaluación

1. **`acoustic_mesh_explicado.md`** — nuevo explicador bloque-por-bloque del mallador volumétrico. Cubre: AABB, descomposición de Freudenthal (1 hex $\rightarrow$ 6 tets conformes), raycast Möller-Trumbore vectorizado ($(N_p, N_t, 3)$ broadcasted con chunking de memoria), filtro por centroide y remapeo de nodos. Pensado para acústicos sin background en CS:

cada operación NumPy se anota con la forma (shape) de los arrays en cada paso, y se explica el “porqué” del truco vectorizado vs el bucle ingenuo equivalente. Incluye glosario de patrones (broadcasting, einsum, fancy indexing, coo_matrix con suma automática) y referencias bibliográficas (Möller-Trumbore 1997, Bey 1995 para Freudenthal, Ihlenburg para ppw).

2. **acoustic_fem_explicado.md** profundizado sobre la base v2.7. Se agregaron:

- **Box “Cómo leer nodes[tets] — explicación desde cero”** en sección 3.2 (fancy indexing 2D, con ejemplo concreto de 5 nodos + 2 tets, diagrama de formas, y la regla “índice + ejes sobrantes” que reaparece en `field_nodal[tet_nodes]` y `self.tets[best_tet]`).
- **Box “¿Qué es un KDTree? — explicación desde cero”** en sección 7 (BST espacial, construcción por partición de medianas, queries $O(\log N)$, cuándo falla la hipótesis del centroide en slivers, mini-experimento corredor).
- Sección final con **glosario de patrones NumPy/SciPy** usados en el archivo (einsum batch, broadcasting con None, inv/det en lote, fancy indexing 2D, shift-invert de eigsh).
- **Referencias bibliográficas** (Zienkiewicz-Taylor, Ihlenburg, Saad para Lanczos/ARPACK, Kuttruff para damping modal).

3. **cuestionario_acoustic.html** — autoevaluación interactiva, **86 preguntas en 13 categorías** (física y forma débil, mesh: voxelización, mesh: raycast, ensamblaje K/M, autovalores, barycentric, KDTree, FRF modal, NumPy tricks, límites y validez, integración, capciosas, código snippet, numéricas con cálculo a mano). Cada respuesta se bloquea al marcar con explicación inline. Resultado final con desglose por categoría coloreado (verde $\geq 75\%$, amarillo $\geq 50\%$, rojo $< 50\%$) para identificar qué reforzar. Botón “Reintentar” reinicia sin recargar. Doble-clic → abre en navegador, sin servidor ni dependencias.

B. Robustez del solver FEM — defensa en profundidad

Cuatro mejoras enfocadas en evitar **no-convergencia de Lanczos** en mallas con paredes oblicuas (donde la voxelización genera slivers en los bordes escalonados, que ensucian K y M con entradas mal escaladas). Implementadas como cambios estrictamente aditivos: las firmas públicas de `build_KM`, `build_volume_mesh`, `solve_modes` y `mesh_info` se mantienen retrocompatibles.

4. **Filtro de slivers en build_volume_mesh** (Capa 1). Después del filtro por centroide se descartan los tetraedros con $V_e < 1e-6 \cdot V_{promedio}$. Estos tets degenerados (vértices casi coplanares) producen $\det(V_4) \approx 0 \rightarrow$ gradientes enormes en las funciones de forma $\rightarrow$ entradas de K con magnitudes desbalanceadas (mal condicionamiento) y “modos espurios” cerca de $\lambda \approx 0$ que confunden a Lanczos. Helper privado `_tet_volumes` agregado y reutilizado por `mesh_info`. Demo de caja $5 \times 4 \times 3$ valida sin cambios ($h_{ratio}=1.0$, $n_{slivers}=0$ — la caja axis-aligned no genera slivers nativos).

5. **Métricas de calidad en mesh_info** (Capa 4). Tres claves nuevas en el dict devuelto:

- `h_min`: tamaño característico mínimo ($V_e^{(1/3)} \cdot \text{factor del tet más chico}$).
- `h_ratio`: h_{max} / h_{min} . Valores > 50 indican malla muy heterogénea (slivers o tets alargados sobrevivientes al filtro).
- `n_slivers`: cantidad de tets con $vol < 1e-4 \cdot vol_{promedio}$ (umbral 100× más laxo que el del filtro, para detectar también “casi slivers” que pasaron pero están en zona borderline).

Las claves existentes (`n_nodes`, `n_tets`, `volume`, `h_avg`, `h_max`) se mantienen idénticas, así que consumers viejos no se rompen. El panel puede usarlas para alertar al usuario antes de que `solve_modes` falle.

6. **Solver robusto en solve_modes** (Capa 2). Try/except ArpackNoConvergence con tres planes en cascada:

- **Plan A:** si los autovalores convergidos alcanzan `n_request`, usarlos directamente.
- **Plan B:** reintentar con $\sigma \times 10$ (esquiva clusters de autovalores cerca del σ actual). Hasta 2 reintentos.
- **Plan C:** levantar `RuntimeError` con mensaje accionable que sugiere chequear `mesh_info()['n_slivers']`, reducir `n_modes`, o pasar σ distinto.

`maxiter` ahora se setea explícitamente a `max(300, 20·n_request)` en lugar del default de ARPACK ($10 \cdot N$), que puede ser corto para mallas con autovalores juntos. Parámetros `_attempt` y `_max_attempts` agregados como internos (con guion bajo, no parte de la API pública). Cero impacto en el caso feliz: el `try` solo cuesta cuando hay excepción.

7. **Simetrización forzada de K y M** en `build_KM`. Al final del ensamblaje: $K = (K + K.T) * 0.5$, ídem para M . Aunque K y M son simétricas por construcción (porque $K_{ij}^e = K_{ji}^e$ y $M_{ij}^e = M_{ji}^e$), el `scatter` via `coo_matrix` suma contribuciones de los N_e tets en orden no controlado, y la aritmética IEEE-754 puede dejar asimetrías residuales del orden de $1e-15$. `eigsh` con `shift-invert` asume simetría **estricta**; sin esto puede devolver autovalores con parte imaginaria pequeña o fallar convergencia en mallas patológicas. Costo $O(nnz)$, despreciable.

C. Validación

8. **Demo acoustic_fem.py** (caja $5 \times 4 \times 3$ m, 8 modos, `n_per_meter=2`): error 0.40–3.22 % vs analítico (rango idéntico a pre-cambios). Pico de FRF en 70 Hz con $|H| = 0.00133$ — mismo número, lo que confirma que las cuatro mejoras son matemáticamente inocuas en el caso feliz.
9. **mesh_info en malla limpia** (caja axis-aligned): `n_slivers=0`, `h_ratio=1.0`, `h_min=h_max=h_avg=0.5`. Confirma que el filtro de Capa 1 no descarta tets legítimos y que la malla de Freudenthal es perfectamente homogénea cuando la geometría lo permite.
10. **Test sintético con sliver inyectado** (tet con vértices casi coplanares, $vol \approx 1.67e-13$): `n_slivers=1` reportado correctamente, `h_ratio = 1e4` (alarma de heterogeneidad). El filtro detecta el caso patológico que dispararía la no-convergencia.

D. Lo que no cambió

- Firmas públicas de `build_volume_mesh(surface_verts, surface_tris, n_per_meter, max_nodes)`, `build_KM(nodes, tets)`, `solve_modes(K, M, n_modes, c, sigma, drop_zero_mode)` — sin cambios.
- `n_per_meter` sigue siendo controlable por el usuario; no se reemplaza por un auto-tuner que oculte la palanca.
- `mesh_info` devuelve un **superset** de las claves anteriores: el panel sigue mostrando `n_tets`, `volume`, `h_avg` igual que antes.
- No se agregaron dependencias externas: sigue todo en `numpy + scipy`.

Cambios v2.10 (29 de mayo): evaluación de FEM P2 (elementos cuadráticos) — explorada, validada, y **descartada por costo/beneficio**. La sesión incluye también primitivas de planta curva para `make_room`.

A. Primitivas de planta curva en geometry.py

1. `make_room(..., shape="circle" | "ellipse", curve_samples=96)`. Nueva opción para generar plantas circulares y elípticas:
 - `shape="circle"`: $radio = \min(width, length)/2$, centrado en (0, 0).
 - `shape="ellipse"`: semiejes ($width/2, length/2$).
 - Internamente se muestrea la curva en `curve_samples` puntos uniformes en el parámetro t (96 por defecto → desviación < 0.05 % vs curva real para excentricidad moderada).

- Default shape="polygon": comportamiento original retrocompatible.
2. **Helper público sample_room_curve(shape, width, length, n_samples)**. Devuelve los puntos (x, y) que materializan la curva. Útil para consumers que necesiten la información paramétrica (p. ej. mallador isoparamétrico futuro o visualizador con render suave).

B. Evaluación de FEM P2 — explorada y descartada

Se implementó un **smoke test completo de elementos cuadráticos** (10 nodos por tet en lugar de 4) y se midió rigurosamente el trade-off entre precisión y costo. El experimento incluyó:

- **P2 subparamétrico** (geometría lineal + campo cuadrático): shape functions Lagrange en baricéntricas ($N_j = L_j (2L_j - 1)$ para vértices, $4L_j L_k$ para aristas), upgrade de malla P1 a P2 con deduplicación de aristas, ensamblaje vectorizado de K con cuadratura Keast 5-pt y M en forma cerrada (multinomial — la cuadratura introducía error de signo al integrar el grado 4).
- **P2 isoparamétrico** (campo Y geometría cuadráticos): Jacobiano $J(L)$ por punto de cuadratura, snap de midpoints a curvas reales (probado contra la planta elíptica), validación numérica con sanity check $||K_{iso} - K_{sub}||_F < 10^{-13}$ cuando los midpoints están en promedios aritméticos.

Validación numérica Caja 5×4×3, 8 primeros modos vs analítico:

Modo	P1 (n_per_meter=2)	P2 sub	P2 iso
0	+0.40 %	+0.00 %	+0.00 %
4	+2.04 %	+0.01 %	+0.01 %
7	+3.22 %	+0.03 %	+0.03 %

P2 reduce el error a precisión cercana a la del solver mismo (~100× más preciso que P1).

Costo computacional (5 salas × 30 modos)

Recinto	P1 total	P2 sub total	Razón
shoebox_chico	33 ms	216 ms	6.5×
shoebox_grande	129 ms	4.65 s	36×
elipse	232 ms	1.15 s	5×
convexo+arco	190 ms	2.27 s	12×
no-convexo+arco	272 ms	1.95 s	7×

Conclusión No hay ventaja práctica en migrar a P2 para el caso de uso del proyecto.

La razón:

- El error de P1 con n_per_meter=2 es **ya muy bajo** (0.4–3 % en modos altos de cajas, ~1 % típico). En el régimen modal de salas — la única zona donde el FEM domina, por debajo de la frecuencia de Schroeder — ese error es indistinguible del error de modelado (RT60 estimado, posiciones de fuentes, materiales). Es decir, el ruido físico del problema es mayor que el error numérico de P1.
- P2 sub gana ~100× en precisión, pero **cuesta 5× a 36× más tiempo de cómputo**. Para el shoebox grande se llega a casi 5 segundos de Lanczos contra 130 ms — diferencia que sí se nota desde la UI.
- P2 iso adicional aporta otro 0.5–1 % en geometrías curvas pero **sólo si la malla tiene nodos sobre el borde**. La malla voxel actual deja la mayoría de los midpoints en el interior; sólo ~9 % califican para snap. El beneficio queda subutilizado.

- La inversión en mejorar precisión por encima de los ~1 % de P1 **no se traduce en ninguna decisión acústica distinta**: las predicciones de RT60, FRF, distribución modal y campo nodal son materialmente las mismas con cualquiera de los dos solvers.

Resultado de la evaluación: **se mantiene P1 como solver de producción**. Los archivos `acoustic_fem_p2.py`, `bench_p1_p2.py`, `bench_p2_iso.py`, `planP2.md` se removieron tras esta entrada de changelog. El trabajo queda documentado aquí como evidencia de que la elección de P1 es deliberada y respaldada por medición, no una limitación.

C. Lo que no cambió en esta versión

- Solver de producción sigue siendo P1 lineal con todas las protecciones de v2.9 (filtro de slivers, solver robusto, simetrización forzada).
- Las primitivas de planta curva (`shape="circle"`, `shape="ellipse"`) no están aún expuestas en el panel de UI; quedan disponibles vía API para uso programático o integración futura.

Cambios v2.11 (30 de mayo): sesión de auditoría física del solver modal. Dos resultados principales:

A. Benchmark modal damping vs matriz C de impedancia

Se evaluó empíricamente si conviene migrar el modelo de absorción desde el esquema actual (un ξ_n por modo derivado de RT60 Sabine, sin localización espacial) hacia una matriz C de impedancia ensamblada en la frontera, con Z derivado del α del catálogo via $r = \sqrt{1-\alpha}$.

Setup: shoebox 5×4×3, $\alpha = 0.30$ uniforme, `n_per_meter = 2`, 12 modos, FRF en 40 puntos entre 20 y 150 Hz, fuente en esquina, receptor central. Script reproducible: `bench_modal_vs_impedance.py` (raíz del proyecto), datos crudos en `bench_modal_vs_impedance.json`.

Resultados:

Métrica	Modal damping (actual)	C-matrix directo
Pipeline completo	26 ms	242 ms
Ratio de costo	1×	9.5×
Picos modales localizados	exactos	exactos
RMS diff en banda 30-100 Hz	—	1.6 dB
Hermiticidad numérica	preservada (Lanczos directo)	rota (requeriría Arnoldi)

Conclusión: para el caso de uso de la app (acústica arquitectónica con α de catálogo, salas hasta ~10⁵ nodos, decisiones a nivel de modo identificable), **modal damping es estrictamente mejor**. La matriz C sólo ganaría si el usuario tuviera Z(ω) medida en tubo de impedancia (escenario que no aparece en el flujo de trabajo). Detalle completo en `acoustic_fem_explicado.md` §16.

B. Fix de calibración: factor c^2 en la fórmula modal

Hallazgo lateral del benchmark: las funciones `frequency_response` y `modal_pressure_field` de `acoustic_fem.py` (y `frequency_response` de `fem_modal.py`, el solver shoebox-only legacy) **omitían un factor c^2** en la prefactorización. La derivación canónica de la Green function modal de Helmholtz en cavidad da:

$$\begin{aligned}
 p(xr) &= i\omega p_0 \cdot \sum \phi_n(xr) \phi_n(xs) / (\lambda_n - k^2) \\
 &= i\omega p_0 \cdot c^2 \cdot \sum \phi_n(xr) \phi_n(xs) / (\omega_n^2 - \omega^2) \quad (\text{con } \omega_n^2 = c^2 \cdot \lambda_n, \quad k^2 = \omega^2 / c^2)
 \end{aligned}$$

El código histórico calculaba sólo $iwp_0 \cdot \sum \dots / (\omega n^2 - \omega^2)$, sin el c^2 . Resultado: **toda la FRF y todos los campos de presión modales reportados en dB SPL estaban ~101 dB por debajo del valor físico real** ($20 \cdot \log_{10}(c^2) = 101.4$ dB).

Por qué no se había notado:

- Los gráficos de FRF y de slice heatmap se interpretan como **forma relativa** (posición de picos, profundidad de nulls, ancho de banda) — todo eso es correcto, el bug es sólo de calibración absoluta.
- La cadena de auralización (`audio_utils.apply_frf_filter`) **normaliza a peak = 0.98** antes del DAC, así que el offset es invisible al playback.
- No hay funciones que comparen contra mediciones SPL reales, así que nada disparaba la inconsistencia.

Fix aplicado: se agregó $* c^{**2}$ al prefactor de las tres funciones. Validado con:

1. **Cálculo analítico** del pico SPL para mode (2,0,0) en setup conocido: 74.8 dB esperado.
2. **Smoke test en acoustic_fem.__main__**: la app debe arrojar 50–100 dB SPL para $Q = 1 \text{ mm}^3/\text{s}$, $\xi = 0.05$. Obtiene 74.2 dB (coincide).
3. **Benchmark vs C-matrix de impedancia**: post-fix, RMS diff 1.6 dB en banda modal 30–100 Hz, sin shifts manuales. Los dos métodos físicamente diferentes convergen al SPL correcto.

Compatibilidad rota:

- **Auralización**: invariante (normaliza a peak). El sonido es idéntico antes y después.
- **FRF plot en pantalla**: ahora muestra dB SPL físicos (antes mostraba dB SPL “-101”). Los valores se ven mucho más altos que antes — esto es correcto.
- **Slice heatmap “Presión |p|”**: idem, leyenda dB SPL ahora calibrada.
- **Exports CSV/TXT** generados con $v \leq 2.10$: quedan **+101 dB desfasados** respecto de los nuevos. Si comparas contra archivos viejos, restá 101.4 dB a los nuevos para llegar a la convención vieja.

Aplicabilidad de la corrección:

- Cualquier intento de calibrar la auralización a un SPL absoluto (ej.: “este altavoz de sensibilidad 90 dB en esta sala debería medir X dB SPL en el receptor”). Antes era imposible; ahora coincide con la física.
- Comparación contra mediciones reales en la sala con sonómetro calibrado.
- Análisis de margen de headroom de altavoces / amplificadores.

C. Lo que no cambió en v2.11

- Pipeline de solver, mallado y auralización idénticos. Sólo se corrigió el prefactor de la FRF / campo modal.
- API pública intacta (firmas de `frequency_response`, `modal_pressure_field` no cambian).
- Solver de producción sigue siendo P1 con todas las protecciones de v2.9.
- Modal damping con $\xi_n = 1.1 / (f_n \cdot RT60)$ sigue siendo el modelo de absorción (decisión D5b en `notas_para_claude.md`, refrendada con datos en este ciclo).

Cambios v2.12 (30 de mayo): sesión de UX del solver modal + un bug físico de validez que estaba escondido. Tres ejes.

A. Picker de modos con filtro y leyenda informativa

Antes: el picker Modo: mostraba la lista cruda de modos por índice (0, 1, 2, ..., N-1). Cuando pedías 50 modos era incómodo buscar uno específico, y no quedaba claro qué frecuencias cubría el set.

Ahora, en el grupo **Visualización modal**:

- **Filtro $f_{\min}$ visible y $f_{\max}$ visible** (en Hz): oculta del picker los modos fuera de ese rango sin tocar el cálculo. Sirve para “quiero ver sólo los modos entre 60 y 100 Hz” en un set de 200.
- **Leyenda dinámica** debajo del picker: muestra cuántos modos hay en total calculados, el rango real $f_0 - f_n$ y cuántos quedan filtrados. Si tu $f_{\max}$ visible excede el último modo computado, te avisa con “*el filtro pasa por encima del rango calculado*” (ese caso significaba antes confusión silenciosa).
- **Índice real preservado vía `userData` del combo**: aunque filtres, cuando seleccionas un modo el slice y el heatmap reciben el índice absoluto correcto. El refactor del picker tocó cuatro call-sites y un helper `_current_mode_idx()`.

B. Spinbox N° modos ampliado y sugerencia Weyl

- **Rango de N° modos** subido de (2, 80) a (2, 500). Permite cubrir hasta $f_{\text{Schroeder}}$ en salas chicas con f_S alta (un cuarto de 60 m^3 con $RT60 \approx 1 \text{ s}$ tiene $f_S \approx 270 \text{ Hz}$ y necesita ~ 170 modos por ley de Weyl).
- **Label $\approx N$ modos hasta $f_{\text{Schroeder}}$ (Weyl)** debajo del spinbox. Se actualiza al apretar Calcular $f_{\text{Schroeder}}$. La estimación usa el término de volumen más la corrección de superficie:

$$N(f) \approx (4\pi/3) \cdot V \cdot f^3 / c^3 + (\pi/4) \cdot S \cdot f^2 / c^2$$

Si el conteo Weyl supera el cap del spinbox, la leyenda lo avisa con “*considerará refinar la malla o aceptar cobertura parcial*”. El usuario decide cuántos pedir; el número es informativo.

C. Sugerencia automática de $n_{\text{per_meter}}$ (compromiso D4)

La decisión histórica D4 mantiene el slider Densidad voxel controlable (preview rápido vs. análisis fino requieren mallas distintas). Pero el usuario tenía que hacer la cuenta mental $\text{npm} = \text{ppw} \cdot f_S / c$ cada vez que quería cobertura exacta hasta $f_{\text{Schroeder}}$.

Ahora hay un compromiso:

- **Label** npm sugerido: X.XX debajo del slider, con el valor calculado para cubrir exactamente hasta $f_{\text{Schroeder}}$.
- **Botón [Aplicar]** al lado que carga ese valor al spinbox de un click.
- Si la sugerencia excede el rango del spinbox (sala muy chica con f_S muy alta), se clipea y el label te avisa.
- Tooltip del spinbox menciona la regla $\text{npm} = 6 \cdot f_{\max} / 343$ para que entiendas qué hace el botón.

El slider sigue siendo editable. Quien quiera preview rápido ignora la sugerencia y deja $\text{npm} = 2.5$; quien quiera análisis riguroso aprieta Aplicar.

D. Fix de validez de malla: modos arriba de $f_{\max_malla}$ descartados

Bug físico encontrado y corregido. Antes, si pedías más modos de los que la malla puede resolver (algo que pasaba fácil al combinar Weyl + npm sugerido, porque Weyl es aproximada), el solver te devolvía los N modos más bajos sin chequear si alguno superaba $f_{\max_malla} = c / (\text{ppw} \cdot h_{\max})$. Esos modos arriba del techo de validez de la malla son **numéricamente sucios**: la dispersión del esquema FEM les corre la frecuencia y la forma modal tiene plegado de onda. Aparecen sin error pero son basura física.

Ahora hay un paso de **post-clip automático** en el panel: tras cada solve, se filtran las frecuencias con $f > f_{\max_malla}$ y se descartan del set. El usuario ve en el log:

FEM: pediste 256 modos, 210 son válidos. 46 excedían
f_max_malla = 59 Hz (descartados por dispersión numérica del esquema).

Coherente con la leyenda del picker (que ahora nunca muestra modos por encima del techo) y con el badge válido hasta XX Hz.

E. Fixes estéticos en diálogos

- **Diálogo de importar CAD / reparar agujeros:** los botones largos (✓ Cerrar este hueco (auto), Soldar a vértices cercanos, Reparar TODO automáticamente) se cortaban al primer carácter por una combinación de Qt centrando el texto y la métrica irregular del Unicode al inicio. Fix con triple defensa:
 1. setMinimumWidth(440) en el panel izquierdo del splitter.
 2. setMinimumWidth(380) por botón.
 3. text-align: left; padding-left: 16px vía styleSheet local, para evitar que el centrado clipee el primer carácter cuando el sizeHint() subestima por el Unicode ancho.
- **Diálogos de FRF, RT60 y Slice heatmap:** los botones Exportar PNG/SVG/PDF/CSV/TXT se cortaban con setMinimumWidth(100) que no alcanzaba para el padding QSS Catppuccin. Bumpeados a 140 con sizePolicy(Preferred, Fixed).

F. Lo que no cambió en v2.12

- Solver FEM (solve_modes, build_KM, frequency_response) intacto. Cambios sólo en el panel UI y en el post-procesamiento (clip de validez).
- Auralización y cadena de FRF→audio igual.
- API pública intacta. Cualquier script externo que use acoustic_femo acoustic_analysis.run_fem_modal sigue funcionando idéntico — el clip por validez vive en el panel, no en el solver, así que el usuario programático mantiene control total.
- Decisión D4 vigente: n_per_meter sigue siendo palanca controlable, no auto-tuner.

Cambios v2.13 (junio 2026): batch grande post-v2.12. Criterios de diseño nuevos respaldados por bibliografía, fuentes reales con respuesta en frecuencia, geometría no-prismática, optimizador de ubicación de fuentes, y diagnóstico de corregibilidad por EQ. Once ejes.

A. Fuentes reales: respuesta en frecuencia Q(f) + fase (FRD)

Las fuentes dejan de ser monopolos de Q constante: pueden cargar una **medición real** (archivo **FRD** de VituixCAD: frecuencia | magnitud dB | fase). Núcleo en sources.SourceResponse — la respuesta es una **ganancia compleja g(f) relativa al Q baseline** (decisión “opción 1”: sin curva → $g \equiv 1$ → FRF idéntica a la baseline, regresión exacta, calibración c^2 intacta). frd.py parsea FRD y, si falta la fase, la sintetiza por **fase mínima** (Hilbert). Dos anclajes al cargar: **absoluto** (el nivel medido manda) y **relativo** (solo forma + fase, nivel desde la sensibilidad). Atajo manual sin FRD: **delay [ms] + invertir polaridad + offset de fase [°]** ($g = \pm e^{i(\phi_0 - 2\pi f\tau)}$). UI en SourceEditDialog (cargar/quitar FRD, combo de anclaje, preview mag+fase). Se embebe en .room **v5** (sin curva → Q constante, compat hacia atrás). La distribución modal (f_n, ϕ_n) **no** depende de la fuente; Q(f) solo mejora la respuesta forzada (FRF, campo, audio, interferencia multi-fuente).

B. Librería de ratios: Cox + corrección de nombres

prediction.RATIO_LIBRARY tenía los nombres cruzados. Corregidos: **Louden** (1:1.4:1.9), **Bolt** (1:1.26:1.59), **Sepmeyer** (1:1.6:2.33), y se agregó **Cox** (1:1.56:1.86). generate_candidates genera un candidato por ratio y predict() recorta al **top-3 por score**. El relabel no rompe .room (las predicciones no persisten el nombre del ratio).

C. Altura por uso, sin cap duro de 4 m

USE_PRESETS ahora tiene `h_default` por uso (home theatre / aula / estudio = 3 m; live 3.5; conferencias 3.2; polivalente 5; cámara 6; sinfónica 12). El cap duro histórico de 4 m se reemplaza por la altura del uso, con override del usuario (checkbox “Override altura” en el panel de Predicción).

D. Geometría lofteada (cortes laterales)

Nuevo modelo de recinto **no-prismático** (Modelo 1: perfil de tope por pared, piso plano, techo que sigue los topes). `geometry.make_lofted_room(base_polygon, wall_profiles)` + `dispatcher.build_room_geometry`. Wizard `SectionWizard(section_dialog.py)`: se dibuja el corte lateral por pared, con simetría opcional. Se persiste en `.room v6` (perfiles en params; v4/v5 → prisma). Watertight garantizado (techo triangulado por el rim).

E. Baffle orientado (visual) + gestos 3D

OmniSource gana orientation (azimut del frente), pitch (inclinación), `baffle_size` y `mounted` (montaje en pared) — **puramente geométricos**: la fuente sigue siendo acústicamente omni (la directividad se descartó; en banda modal los parlantes son casi omni). El baffle se dibuja como **wireframe** (12 aristas + woofer/tweeter) en el viewer 3D. Gesto directo: **Alt+Ctrl + arrastrar** orienta (horizontal = azimut, vertical = pitch). Botón “Pegar a pared más cercana” (flush). Persistido en `.room`.

F. SBIR (Speaker-Boundary Interference Response)

Nuevo `sbir.py`: fuentes imagen de 1er orden contra las 6 superficies. Cada cara es un plano (centroide + normal del face group); presión con el mismo monopolo de la app, atenuada por $R(f) = \sqrt{1 - \alpha(f)}$. Salida en **dB relativo al directo** (0 dB anecoico, +6 dB boundary-lift en LF, peine de notches arriba). Una curva por fuente + la suma compleja (estéreo). `SBIRDialog` (espejo del de FRF): eje log 20–500 Hz, grilla 1/3 oct, `axvline` en cada notch teórico $c/(4d)$, export.

G. Métricas de respuesta forzada: FoM + cruce modal numérico

Nuevo `modal_metrics.py` (cómputo puro): **(1) figura de mérito** `FoM_flat` (planitud de la respuesta media) y `FoM_espacial` (consistencia asiento-a-asiento), con damping de materiales, suavizado en energía 1/3 oct, sobre una grilla de receptores y solo en la banda válida — corrige los defectos del σ_{SPL} de un punto. **(2) cruce modal** `f_cross` por solapamiento modal numérico (estilo MDCF): $M(\bar{f}) = B_{HP} \cdot n(f)$ con densidad modal numérica (ve la forma de la sala). Ambas cableadas a la UI: `f_cross` junto a `f_Schroeder`, `FoM` junto a la FRF.

H. Optimizador de ubicación de fuentes

`location_opt.py` + nuevo modo de Predicción. Optimiza posiciones (+ delay/polaridad) de las fuentes para una sala fija, minimizando un objetivo combinado de **FoM + SBIR + suavidad modal** con pesos por uso (ajustables). Tres modos de predicción combinables: **Geometría / Ubicación / Combinado**. La card de Ubicación tiene “Aplicar” que coloca las fuentes optimizadas en la pestaña Acústica.

I. Criterios de diseño al score de Predicción (respaldo bibliográfico)

Minado de un corpus de salas chicas/estudio (Everest, Newell, Cox&D’Antonio, BBC/Rose, Beranek, Howard&Angus, Toole, etc.) → `criterios_room_geom_fuente.md`. Gaps implementados con benchmark: ratio BBC (A33), **damping per-modo pesado por la forma modal en cada cara** (A36), **FSI $\psi(25)$ de Rindel** (A6) y **densidad Bonello no-decreciente** (A3) — ambos

cableados al grupo MODAL del score con pesos calibrados; Bass Ratio (D5), umbral perceptual de **Fazenda** (C9, reemplaza el $Q > 30$ fijo, curva elegida por el programa de la sala). FSI/Bonello visibles en la card de Predicción (chips + lectura del ψ).

J. Diagnóstico de corregibilidad por EQ (fase mínima vs no mínima)

eq_correctability en modal_metrics.py: clasifica, por frecuencia, qué problemas de la respuesta arregla un **EQ global** (corregibles) y cuáles **exigen acústica/ubicación** (no corregibles). Método: **consistencia espacial** (varianza asiento-a-asiento, invariante a EQ global = cota irreducible) + **envolvente sin cancelación** (referencia de fase mínima construida de los propios modos, sin cepstrum). Cierra el loop: simula el EQ global de referencia y **mide** la mejora (improvement_flat) y lo irreducible (fom_espacial). El diagnóstico se corre en la **sub-banda confiable** (necesita $\sim \text{ppw} \geq 15$, más resolución que el solver: el veredicto vive de los signos de ϕ_n cerca de los nodos). Overlay en el diálogo de FRF: zonas en rojo = no ecualizables. Validado contra teoría (toda respuesta de sala real es de fase no mínima) y contra Welti 2003 (multi-sub baja la varianza espacial).

K. Fix del auto-tuner de malla

mesh_router: dos bugs corregidos. (1) auto_density crasheaba con presupuesto infinito (caía a densidad manual). (2) gmsh sub-entregaba validez porque $h_{\text{max}} \approx 1.5 \cdot h_{\text{target}}$. Resultado: de 81 a 308 modos válidos en el caso de referencia.

L. Lo que no cambió en v2.13

- Solver FEM (solve_modes, build_KM, frequency_response) intacto; todos los cambios son aditivos o viven en el panel / módulos nuevos.
- Decisiones vigentes: P1 de producción (D1), FEM a mano sin FEniCS (D2), n_per_meter palanca controlable (D4), modal damping sin matriz C (D5b), directividad de fuentes descartada.
- Calibración c^2 de la FRF intacta (la $g(f)$ de fuente es relativa al Q baseline $\rightarrow$ cero riesgo).
- API pública de build_KM, build_volume_mesh, solve_modes, compute_forced_response intacta.

Cambios v2.14 (27 de junio 2026): features pedidos por el usuario tras cerrar el batch v2.13 — undo/redo global, un sistema de presets de materiales (Predicción + Acústica), y edición numérica exacta en el dibujo de forma. Cinco ejes.

A. Undo/redo global (Ctrl+Z / Ctrl+Y, 10 acciones)

El undo histórico solo deshacía cambios de geometría. Ahora es **global**: deshace y rehace **cualquier** acción (mover/girar/inclinar/agregar/editar fuentes, mover el receptor, asignar materiales, importar/quitar CAD, aplicar una predicción, cambiar dimensiones). Implementación por **snapshot del estado completo** (mismo contenido que el .room: params + fuentes + receptor + materiales + CAD), no por comando-por-acción: un **timer de polling (~400 ms)** compara el estado serializado contra el último snapshot y, si cambió, lo apila. Es **global por construcción** — no puede “perderse” una acción aunque ocurra dentro de un panel, sin instrumentar cada mutación. Un **drag continuo cuenta como una sola acción** (espera a que el gesto se asiente). El stack guarda las **últimas 10** acciones; el snapshot guarda los **inputs**, no los resultados pesados del FEM (deshacés y recalculás con Enter). Vive en main.py (_capture_state / _restore_state / _maybe_snapshot). Smoke: smoke_test_undo.py (5 tests).

B. Gate de materiales en Predicción

Antes, Predicción trabajaba de un RT60 objetivo tipeado, sin relación con materiales. Ahora, al apretar **Predecir** sin haber definido la absorción, aparece un **aviso** con tres caminos: **(1) que elija el programa** (el RT60 típico del uso, comportamiento anterior); **(2) preset** placeholder *piso madera · paredes ladrillo · techo madera*; **(3) coeficiente de absorción uniforme** (α editable, todas las caras). En los modos (2) y (3) los **materiales determinan el RT60 por candidato** (Sabine hacia adelante: cada geometría candidata, con su volumen y superficie propios, da su propio RT60 $\rightarrow$ `effective_rt60` en `prediction.py`), que alimenta los scores dependientes de RT (modal Q/Fazenda, Schroeder, feasibility, STI/%Alcons, damping de ubicación). Un botón “Materiales” en el grupo de objetivos muestra y permite cambiar la elección. El preset usa α reflectantes reales del catálogo (madera/ladrillo absorben poco $\rightarrow$ sala viva, ~ 1.8 s; es lo honesto — el sistema de presets completo se discutirá aparte). La asignación fina de materiales **por cara** sigue siendo de la pestaña Acústica. Bench: `bench_prediction_materials.py` (7 tests).

C. Fix de color de los diagnósticos de la FRF

Los dos labels de diagnóstico del diálogo de FRF (FoM y corregibilidad EQ) estaban en gris/rojo oscuro $\rightarrow$ ilegibles sobre el fondo oscuro Catppuccin. Pasados a blanco. El sombreado rojo/amarillo del plot (semántico) no cambia.

D. Editor de forma: dimensiones exactas + origen corrido

El dibujo de planta y el de cortes laterales ganan edición numérica exacta, además del arrastre a ojo:

- **Planta** (`shape_dialog.py`): cada arista del polígono muestra su **longitud** en un chip clickeable. Click sobre el número $\rightarrow$ se tipea el valor exacto y la arista se redimensiona: el primer vértice queda **fijo** y el segundo se desliza en la **misma dirección** de la arista (sin snap a grilla). Para shoebox siguen estando los sliders de Geometría; el dibujo es para formas irregulares.
- **Origen corrido**: el (0,0) del canvas de planta pasó del centro a la **esquina inferior-izquierda** (con un margen negativo chico, escalable con la grilla). Así se dibuja el recinto en el cuadrante positivo y se puede fijar una esquina en (0,0) — o no.
- **Cortes laterales** (`section_dialog.py`): cada punto del perfil de pared muestra su **altura (z)** en un chip clickeable (editables los puntos arrastrables; las esquinas fijas las sigue mandando el rim). Click $\rightarrow$ altura exacta. Y la relación con la pared opuesta pasa de un checkbox a un selector **Libre / Espejo / Igual**: *Espejo* copia el perfil de la opuesta reflejado en t (alineado en el espacio físico, lo de antes); *Igual* lo copia directo (misma forma, reescalada a este largo). Ambas bloquean el dibujo de esa pared.

Smoke tests: `smoke_test_shape_edge.py`, `smoke_test_section_edit.py`.

E. Sistema de presets de materiales (Predicción + Acústica)

El placeholder de absorción del eje B se reemplaza por un sistema real de presets, **compartido** entre las dos pestañas (definido una sola vez en `material_library.py`):

- **Presets con nombre** (`MATERIAL_PRESETS`): *Reflectante / viva · Estudio tratado · Home theatre · Aula / conferencia · Neutra*. Cada uno mapea piso/paredes/techo a materiales **reales del catálogo**, con α por banda. La resolución es por nombre **normalizado sin acentos** (`resolve_material`), robusta a variaciones del catálogo; si una clave no matchea, cae al rígido por defecto.
- **En Predicción**, el gate de materiales pasa a tres caminos: *que elija el programa · coeficiente uniforme · materiales por superficie*. Este último trae un **combo de presets + tres combos para armar el tuyo** (cualquier material del catálogo, ~ 430). El RT60 de cada candidato

sale de esos materiales **por banda** (effective_rt60 modo “materials”, representativo a 500/1 kHz; opción A: los materiales determinan el RT). Botón **“Aplicar a Acústica”**: asigna esos tres materiales a las caras del recinto (piso/paredes/techo) — y queda deshacible con Ctrl+Z (el undo global captura el cambio).

- **En Acústica**, el diálogo de materiales gana un botón **“Preset nombrado...”** que aplica los mismos cinco presets por zona (g.kind), junto al preset manual que ya existía.
- Fix de layout: el label de estado del main ahora tiene **word-wrap** — un mensaje largo (nombres de materiales) estiraba el panel y cortaba el contenido de Acústica.

Bench: bench_prediction_materials.py (9 tests: resolver, reflectante vs tratado, RT por candidato, end-to-end).

F. Documentación: fix de fondo del auto-tuner de malla

Se documenta (sin código nuevo) un fix de fondo del router de mallado del ciclo anterior. La validez de la malla la define el **peor tetraedro** (h_max), y gmsh entrega $h_{\max} \approx 1.5 \cdot h_{\text{target}}$ — así que el auto-tuner ahora le pide a gmsh una malla 1.5× más fina (`_GMSH_HMAX_OVER_HTARGET` en mesh_router.py) para que la validez **real** alcance f_S en vez de quedarse en $f_S/1.5$. Además, un guard `np.isfinite` evita que el tuner muera en silencio cuando se lo llama con `budget = ∞` (caía a la densidad del slider). Verificado en GUI: sala lofteada por **voxel** valida $\approx f_S$ (165 Hz); hexágono por **gmsh** valida $\geq f_S$ (148 → 168 Hz, margen del lado seguro). Detalle en §7 (“Dimensionado de la malla: el peor tet define la validez”).

Cambios v2.15 (30 de junio 2026): correcciones de la pestaña Predicción al evaluar el diseño propio, soporte de geometría irregular en la evaluación, y dos fixes de UI. Cinco ejes.

A. “Evaluar mi diseño actual” respeta el criterio elegido

Antes, el botón **Evaluar mi diseño actual** scoreaba **siempre la geometría**, ignorando el combo **Optimizar** (Geometría / Ubicación de fuentes / Combinado) y las fuentes que tuvieras colocadas. Ahora respeta el eje:

- **Geometría** → scorea la forma del recinto (como siempre).
- **Ubicación de fuentes** → evalúa **tu layout real** de fuentes en tu recinto (no optimiza: usa las posiciones que pusiste, vía `location_opt.evaluate_layout`).
- **Combinado** → geometría + tu layout real, mezclados.

Lee las fuentes reales del recinto por un callback nuevo (`get_sources`). Sin fuentes en *Ubicación* avisa; en *Combinado* degrada a solo geometría con aviso. Núcleo en `prediction.evaluate_design`; UI en `prediction_panel._on_eval_design`. Bench: `bench_predict_location.py`.

B. Geometría irregular: evaluación sobre la malla real + elección de ponderación

Antes, evaluar un recinto con **planta dibujada** (`base_polygon`) o **cortes laterales** (`wall_profiles`) reconstruía una **caja** con `make_room` e ignoraba la forma real → evaluaba una sala equivocada (y las fuentes caían “afuera” del recinto reconstruido). Ahora, cuando la forma es irregular, el FEM corre sobre la **malla real renderizada** (callback `get_surface`).

Como el score de geometría (proporciones, Bolt, ratios) está **definido para cajas**, al evaluar por Geometría o Combinado aparece un diálogo **“Forma irregular”** con dos opciones:

- **Aproximar con la caja envolvente (AABB)** — el score de geometría se calcula sobre el *bounding box* del recinto; el FEM va sobre la forma real.
- **No ponderar la forma** — no se scorea la geometría de caja. Con un aviso: en *Geometría* no se puede predecir (elegí Ubicación/Combinado o aproximá); en *Combinado* degrada a solo ubicación.

`prediction.evaluate_design(surface, shape_mode) + is_irregular_shape + prediction_panel._ask_irregular_shape`. Esto, además, **corrige el aviso espurio de “fuentes fuera del recinto”** que aparecía al evaluar una forma dibujada.

C. Slider de Alto bloqueado con cortes laterales

El editor de forma bloqueaba **Ancho** y **Largo** con forma personalizada, pero dejaba el slider de **Alto (Z)** editable aunque, con **cortes laterales**, la altura la definen los perfiles → mover el slider no hacía nada (era solo estético). Ahora el **Alto se deshabilita cuando hay cortes laterales** (`wall_profiles`); con forma de **solo planta** (sin cortes) queda editable, porque ahí sí define la altura del prisma. `controls.py (_refresh_custom_state)`.

D. Receptor reubicado al interior si queda afuera

El editor de forma corre el origen a la **esquina inferior-izquierda**, así que una planta dibujada vive en coordenadas positivas que **no contienen el (0,0)** del receptor por defecto → el receptor quedaba **fuera** del recinto y disparaba el aviso “Fuera del recinto” al calcular modos (atribuido por error a las fuentes). Ahora, al **cambiar la geometría** o al **cargar un .room**, si el receptor cae fuera se **reubica a un punto interior** (centro del recinto, validado con `test point-in-mesh`). Solo actúa si está afuera. `acoustic_panel._relocate_receiver_if_outside`.

E. Tooltips legibles

Los tooltips (texto al pasar el cursor) heredaban el color de texto claro del tema sobre el fondo amarillo por defecto de Qt → ilegibles. Regla `QToolTip` nueva: **fondo blanco, texto negro**. `style.py`.

Cambios v2.16 (5 de julio 2026): origen de coordenadas configurable, análisis multi-punto con mute por fuente (tabla MSV/VSA, curvas FRF/SBIR comparadas), carga de mediciones `.trf`, `f_Schroeder` desde materiales, y varios fixes de Predicción y de estabilidad. Diez ejes.

A. Origen (0,0,0) configurable — Auto / Centro de planta / Esquina

Combo nuevo “**Origen (0,0,0)**” en Geometría → Dimensiones. Define dónde queda el origen del sistema de coordenadas respecto del recinto, y aplica a los **tres caminos** de diseño:

- **Auto (según diseño)** — comportamiento histórico de cada camino: paramétrico centrado, planta dibujada como se dibujó, CAD centrado al importar. Default (compatibilidad con `.room` guardados).
- **Centro de planta** — el centro del AABB en planta cae en (0,0).
- **Esquina inf.-izq.** — el recinto vive en el cuadrante positivo, esquina en (0,0).

Al cambiar la convención, **las fuentes, el receptor y los puntos de escucha se trasladan junto con el recinto** — solo cambia el sistema de coordenadas, nada se mueve físicamente. La elección se guarda en el `.room`. Núcleo: `geometry.origin_offset/anchor_vertices` + re-anclaje en `build_room_geometry`; compensación en `main._on_params` (paramétrico/dibujado) y `main._reanchor_cad` (CAD). Bench: `bench_origin_mode.py` (18 oráculos).

Fix asociado: el cargador de `.room` re-centraba el CAD embebido incondicionalmente (compat v3) → un recinto guardado con origen en la esquina volvía centrado y las fuentes quedaban desubicadas. Ahora el loader re-ancla según el `origin_mode` **guardado**; archivos viejos sin la clave cargan idéntico que antes.

B. Frecuencia de Schroeder: panel arriba + RT desde materiales

El bloque de `f_Schroeder` (valor analítico + `f_cross` numérico + botón) salió de “Campo acústico 3D” a su propio grupo “**Frecuencia de Schroeder**”, entre Materiales y FEM. Y el cálculo ya no usa un $\alpha=0.05$ fijo: usa el **RT60 de los materiales asignados** (o el default del mapa), resolviendo el punto fijo $f_S = 2000 \cdot \sqrt{RT(f_S)/V}$ con interpolación por bandas — la transición modal→estadística ocurre con el RT *local* en f_S , no el de una banda arbitraria. Converge en 2-3 iteraciones; $\alpha=0.05$ queda como fallback sin datos, y el log dice explícitamente qué se usó.

C. Hover en Materiales → resaltar caras en el 3D

En el diálogo **Materiales...**, al posar el cursor sobre la fila de un grupo, esas caras **brillan** en el render 3D (render aditivo: visible aun ocluido, sin rotar la cámara). Se apaga al salir de la tabla o cerrar. `IsoViewer.set_highlight_faces` + señal `hovered` en `MaterialsDialog`.

D. Carga de mediciones .trf (transfer function binaria)

El botón de respuesta de fuente ahora es “**Cargar FRD/TRF...**” y acepta los `.trf` binarios de trazas de transfer function (firma JACKREF!; formato descifrado por ingeniería inversa: eje de frecuencias MTW `f32` + magnitud/fase/coherencia `f64`, directorio de offsets en el header — spec en el docstring de `frd.load_trf`). Detección por contenido, no por extensión. Como la magnitud del TRF es una TF **relativa** (0 dB = canales iguales, no SPL absoluto), al cargar uno el anclaje salta automáticamente a **Relativo**. Si la coherencia mediana de la medición es < 0.7 , avisa. Bench: `bench_trf.py` (20 oráculos con `Focal_L/R.trf`).

E. Mute por fuente

Cada fuente de la lista tiene un **checkbox**: desmarcada queda gris con [MUTE] y **no radia** — FRF, SBIR, FoM, campo 3D y Comparar la excluyen — pero conserva posición, curva y bafle. Permite el análisis parlante-por-parlante sin borrar/recrear. Persistido en el `.room` (active). `Omnisource.active` + `SourceArray.active_only()`.

F. Puntos de escucha nombrados

Lista nueva en el grupo Receptor: posicionás el receptor y apretás **Agregar** (el primero se llama “Sweet Spot”, después “Mic 2”, “Mic 3”...). **Renombrar**, **Quitar**, y doble click lleva el receptor a ese punto. Se ven como esferas cyan en el 3D, se guardan en el `.room` y se trasladan con los cambios de origen.

G. Botón “Comparar...” — análisis multi-punto

Compara los puntos de escucha con las **fuentes activas**, en cuatro vistas (Figuras de mérito / Respuestas en frecuencia / SBIR / Todas):

- **Tabla FoM**: una fila por posición con *planitud local* (σ en frecuencia de su curva suavizada a 1/3 de octava) y *desvío vs promedio espacial* (RMS contra la curva media); fila final **CONJUNTO** con **VSA** (σ_f del promedio espacial = `FoM_flat`) y **MSV** (media del σ entre posiciones = `FoM_espacial`), las métricas comparables entre configuraciones. Calculada sobre **tus posiciones reales** (no la grilla interna que usa el FoM del diálogo FRF).
- **FRF**: las curvas SPL de todas las posiciones superpuestas, leyenda por nombre.
- **SBIR**: ídem, con las mismas paredes/materiales que el SBIR normal.
- Export por vista: tabla → CSV/TXT/PNG; curvas → PNG/CSV.

Flujo típico L/R: silenciar R → Comparar → exportar; silenciar L, activar R → repetir. `CompareDialog` + `AcousticPanel._compute_compare_data`.

H. Marcadores en el heatmap 2D

El plano de corte 2D sobreimprime el setup: **fuentes activas como círculo** ○ y **receptores como** × (puntos de escucha + receptor actual), con el **nombre debajo**. Blanco con borde negro (legible sobre cualquier colormap); si un marcador está a más de 0.5 m del plano (en el eje fijo) va semi-transparente — está en la sala pero no sobre ese corte. Sale en los exports PNG/SVG/PDF.

I. Predicción de ubicación con forma irregular: malla real + fuentes adentro

Dos correcciones al eje **Ubicación de fuentes** con planta dibujada:

1. El FEM del recinto fijo corre sobre la **malla real renderizada** (mismo sistema de coordenadas que Geometría), no sobre una caja reconstruida y centrada — las posiciones recomendadas salen en el frame correcto. La leyenda bajo el botón Predecir lo confirma con “· malla real”. `fixed_room_from_design + surface` hilado por `predict_axis→predict_locations→_build_location_context`.
2. Las semillas del optimizador viven en el AABB; con una planta no rectangular podían caer **fuera de la sala** (la cuña AABB=sala). Ahora el contexto conoce la superficie real (`LocationContext.inside_fn`) y las semillas que caen afuera se **reparan** (bisección hacia un ancla interior, preservando la estrategia mono/estéreo/esquina — el label gana “≈”); los refinamientos se filtran. Bench: `bench_predict_location.py` (18 oráculos, incluye el pentágono real donde las 6 semillas caían fuera).

J. Estabilidad y diagnóstico

- **Fix freeze del checkbox**: silenciar una fuente reconstruía la lista desde su propio evento → Qt quedaba con el mouse-grab colgado y la app dejaba de responder a clicks/draggs. Ahora el item se actualiza in place.
- **Fix freeze en resize**: los puntos de escucha usaban un scatter GL persistente (point sprites), sospechoso de stalls del driver en Windows al redimensionar; reemplazado por esferas `GLMeshItem` (patrón de los markers, estable).
- **Watchdog opcional**: correr con `PROT01_WATCHDOG=1` imprime en consola los stacks de todos los threads si la GUI queda colgada > 20 s — diagnóstico exacto en vez de adivinar.

Cambios v2.17 (14 de julio 2026): parches de absorción sub-cara (dibujar una región dentro de una cara con su propio material) y carga de materiales propios desde JSON. Cinco ejes.

A. Parches de absorción sub-cara

Nuevo botón **“Parches de absorción...”** (grupo Materiales) que abre un editor 2D para dibujar **regiones de absorción dentro de una cara**, cada una con su material. Da **resolución sub-cara** al amortiguamiento modal selectivo (A36): el α del parche entra por el RT60 de Sabine (restándole área al anfitrión) y por el ξ_n por modo, pesado por la presión modal ϕ_n^2 sobre la región. Como las ϕ_n se calculan con paredes rígidas, un parche **no** cambia la forma del modo ni el heatmap; el efecto es sobre $\xi_n \rightarrow RT \rightarrow FRF$. Detalle de uso en §10.5.

Decisión de cuadratura. Sin parches, el ξ_n se integra con el método histórico (A36 sobre centroides de la malla de render) → los `.room` sin parches **no cambian ni un dígito**. Con al menos un parche, se conmuta a **cuadratura fina**: cada cara se tesela en muchos puntos y a cada uno se le asigna α del parche o del anfitrión, integrando $\alpha \cdot \phi_n^2 / \phi_n^2$. La cuadratura fina es **más precisa** que la malla gruesa; los números pueden moverse al activar el primer parche (no es un error). Reduce **exacto** a A36 cuando el material es uniforme. Núcleo: `absorption_patch.py`; `.room v8` (`absorption_patches`); bench `bench_absorption_patch.py` (oráculos de área, reducción a A36, equivalencia parche-full-face, monotonía, Sabine patch-aware, geometría de polígonos).

B. Editor 2D: rectángulos y polígonos, zoom, sin solapes

- **Dos modos de dibujo:** *Rectángulo* (arrastrar el botón izquierdo) y *Polígono* (click por vértice — convexo o no —; cerrar cerca del primer punto, con Enter o doble click; botón derecho / Esc deshace o cancela).
- **Rueda del mouse** = zoom in/out sobre la grilla de la cara, centrado en el cursor (con paneo automático para no mover el punto bajo el mouse).
- **Sin solapes:** un parche que pisaría a otro se dibuja en **rojo** y no se agrega (test de solape de polígonos que permite adyacencia por arista). Snapping a grilla configurable (0.1–1 m).
- Núcleo geométrico en `absorption_patch.py`: `poly_area` (shoelace), `points_in_poly` (ray casting), `triangulate_uv` (ear clipping, para no convexos), `polys_overlap`. UI en `patch_dialog.py`.

C. Overlay 3D de los parches

Los parches se **pintan sobre la cara en el visor 3D**, coloreados por material (los no convexos se triangulan por ear clipping), con el quad separado 4 cm hacia el interior. `IsoViewer.set_patches` crea **un GLMeshItem por parche con color uniforme**.

Nota para quien toque el overlay. El primer intento usaba un mesh combinado con `faceColors`, y **no renderizaba** — un `GLMeshItem` con `shader=None` + `faceColors` no se dibuja en esta escena, y **no es cuestión del modo de profundidad** (translucent, additive y opaque fueron los tres invisibles). Es el mismo gotcha ya documentado en `acoustic_viewer.SourceMarkers`, que por eso migró a `GLLinePlotItem`. El único patrón probado es el de `viewer.set_highlight_faces: color uniforme + shader=None + glOptions='additive'`. Como cada parche tiene un solo material, un ítem por parche evita `faceColors`.

D. Parches en el diálogo Materiales (listado, edición y resaltado)

La tabla de **Materiales...** ahora lista los parches debajo de las caras (↳ Parche (rect/polígono) en <cara>, con área y categoría). Cada fila trae su **combo de material**: cambiarlo actualiza el parche (recolorea el overlay en vivo y recalcula ξ /RT al aplicar). Al **posar el cursor** sobre la fila, el parche se **resalta en el 3D** con el mismo brillo ámbar que las caras (`IsoViewer.set_highlight_patch`, ítem propio que no pisa el overlay permanente). El hover unificado distingue cara (`FaceGroup`) de parche (`AbsorptionPatch`) y son mutuamente excluyentes.

E. Cargar tu propio material (JSON)

Botón **“Cargar tu material...”** en el diálogo Materiales: muestra un cuadro con la sintaxis del JSON, abre un selector de archivo, **valida** (nombre + absorción por banda), lo **copia a materials/** sin pisar los del catálogo, y **recarga la biblioteca en el sitio** (`MaterialLibrary.reload`) para que aparezca en todo el programa sin reiniciar. Formato y ejemplo en §10.6.

Cambios v2.18 (19 de julio 2026): **muebles** como obstáculos en el modelo modal — modelado completo (obstáculo rígido + absorción + reflexión) y manipulación directa en el visor. Uso en §6.4. Seis ejes.

A. Muebles como obstáculo rígido (carve)

Un mueble (caja o cilindro) **talla** la malla del aire: los tetraedros dentro del mueble se quitan del dominio antes de ensamblar K, M (la superficie del hueco queda rígida por condición natural, sin ensamblar nada). Los **modos se corren solos** (exacto, no perturbativo). El carve va entre el

mallado y build_KM, preservando la malla original para la absorción; la API estable del solver no se toca (furniture.carve_mesh; cableado en acoustic_analysis.run_fem_modal[_routed] vía muebles=[] opcional → sin muebles, resultado idéntico al histórico).

B. Absorción del mueble (A36)

Si el mueble tiene un **material**, sus caras (la interfaz aire-mueble, extraída de la malla original) entran como grupos nuevos al mismo mecanismo A36 que las paredes: el ξ por modo se pesa por la presión modal sobre el mueble. Sin material = **rígido** (α por defecto). Un sillón tapizado **debe** absorber: en banda modal su rol dominante es la absorción selectiva, no desplazar volumen.

C. Reflexión del mueble (SBIR)

La cara superior de cada mueble se agrega al análisis SBIR como un **panel finito** (rolloff de Rindel por difracción de borde): el sobre del escritorio o el respaldo del sofá rebota, con la reflexión de graves atenuada por el tamaño finito.

D. Interfaz: grupo Muebles + editor + wireframe

Nuevo grupo **Muebles** en la pestaña Acústica (Añadir / Editar / Quitar / Duplicar). El editor (FurnitureEditDialog) toma tipo, centro, tamaño, orientación (yaw), inclinación (pitch), material y etiqueta — todo por edición numérica exacta. El mueble se dibuja como **wireframe** verde-azulado en el visor 3D (naranja al seleccionarlo), con el patrón probado GLLine-PlotItem (nunca GLMeshItem(shader=None)). Se persiste en el .room junto con su material (furniture_materials, aditivo).

E. Manipulación directa (mismos gestos que las fuentes)

Los muebles se mueven y orientan con el mouse igual que los bafles: **Shift**+arrastrar (mover XY), **Ctrl+Shift** (mover Z), **Alt+Ctrl** (girar yaw en horizontal / inclinar pitch en vertical), **doble-click** (editar). Las fuentes tienen prioridad de selección. La **inclinación (pitch) es física**: afecta el carve (lo que inclinás es lo que se talla), no es solo visual; con pitch=0 el cómputo se reduce exacto al caso sin inclinación.

F. Colisiones — los objetos sólidos no se atraviesan

Un mueble no puede superponerse con otro mueble, con el **bafle de un parlante**, ni salirse de los **límites del recinto** (test por AABB). Al arrastrar, **frena** al tocar el obstáculo; al Añadir/Editar con posición inválida, se avisa el motivo y no se agrega. Además, las **fuentes y el receptor** ahora se **traban en las paredes** del recinto al arrastrarlos (clamp al bounding box: se deslizan pegados al límite en vez de salirse).

Cambios v2.19 (29 de julio 2026): **presets de muebles armados** (con forma) y fixes de manipulación surgidos del test visual. Uso en §6.4. Cuatro ejes.

A. Muebles compuestos (forma física)

Nuevo tipo de mueble **compuesto** (unión de sub-piezas box/cylinder en su frame local): el contains es la unión de las partes, así un mueble con forma (una silla, un escritorio) se **talla, mueve, rota y choca como una sola pieza**. La forma es física (afecta el carve); las partes finas (patas, tensores) no resuelven en la malla, que es correcto. contains/aabb/volumen/persistencia/wireframe despachan por tipo; caja y cilindro **reducen exacto** al comportamiento previo.

B. 27 presets en menú agrupado

Botón “**Insertar preset ▼**” con submenús **General** (7), **Aula** (10) y **Estudio / tratamiento** (10): desde silla/escritorio/biblioteca hasta pupitre, pizarrón, casilleros, gobos, bass traps, difusores, resonadores Helmholtz, nubes acústicas, console desk, racks y sofá de control. Cada uno con material sugerido válido del catálogo y colocación (las nubes suspendidas del techo; el resto en el piso). **Alcance del modelo:** la difusión (QRD) y la sintonía Helmholtz no se simulan (el FEM es LF modal); esos presets entran como geometría + material aproximado. Los absorbentes de banda ancha sí se modelan bien. Detalle en §6.4.

C. Fix de picking por silueta

El mueble se agarra clickeando en **cualquier parte de su bounding box proyectado** en pantalla, no solo cerca del centro. Antes, los muebles grandes (sillón, mesa, biblioteca) tenían el centro en un hueco del wireframe y no se dejaban agarrar.

D. Fix de rotación: solo yaw

Alt + Ctrl + arrastrar ahora gira solo en azimut (yaw). Antes el componente vertical del arrastre inclinaba el mueble sin querer, acumulando decenas de grados de pitch: el mueble “se caía”, su bounding box crecía y chocaba con todo (se trababa). El pitch se edita por el campo del diálogo. El **piso ya no atrapa** al mueble (inclinación un borde bajo $z=0$ es inofensivo para el carve).

Cambios v2.20 (30 de julio 2026): **importar muebles desde CAD y gizmo de rotación de 3 ejes**. Uso en §6.4 y §6.4.1. Cuatro ejes.

A. Muebles desde archivo CAD

Nuevo tipo de mueble **mallá** (.obj, .stl, .ply, .off, .glb, .gltf): la forma la define un archivo CAD en vez de una primitiva. El tallado usa un test punto-adentro sobre la mallá, así que la pieza importada afecta modos, absorción y SBIR igual que un preset. Caso de uso: **escanear un estudio real** con el celular, separar las piezas en SketchUp y exportarlas una por una. Al importar se **repara** la mallá si hace falta (unir vértices, tapar huecos, corregir normales) y se **avisa** si queda abierta, porque el test punto-adentro solo es confiable con superficie cerrada. La mallá se embebe en el .room (archivo autocontenido). Detalle y alcance en §6.4.1.

B. Gizmo de rotación de 3 ejes

Manteniendo **Alt + Ctrl** sobre un mueble aparecen **tres anillos** (yaw celeste, pitch ámbar, roll verde). El anillo bajo el cursor se resalta en magenta y, al arrastrar, el mueble gira **solo sobre ese eje**. El eje se elige **antes** de mover, con un click explícito: por eso ya no puede inclinarse sin querer al rotar (el problema que en v2.19 se había resuelto quitando el gesto). Los anillos se dibujan con los mismos ejes locales que usa el tallado, así el anillo que se ve es el eje que efectivamente se mueve.

C. Tercer eje de rotación (roll)

Los muebles ahora tienen **roll** además de yaw y pitch: vuelca la pieza de costado, girando sobre su frente. Convención de aviación (yaw sobre el vertical del mundo, después pitch sobre el transversal, después roll sobre el frente). Como el pitch, **afecta el carve**, no es cosmético. $roll = 0$ reduce **exacto** al comportamiento anterior y los .room previos cargan con roll nulo, sin cambio de versión de archivo. Editable por el anillo verde del gizmo o por su campo en el diálogo.

D. Fix: muebles que se trababan al duplicar

Duplicar un mueble dejaba la copia en la **posición exacta** del original: los dos quedaban solapados al 100 % y la regla de “los sólidos no se atraviesan” bloqueaba todo movimiento de ambos, sin salida. Ahora la copia nace **desplazada** al costado, y además un mueble que ya esté solapado **puede arrastrarse para afuera** (antes solo se podía frenar, nunca escapar). Afectaba también a los presets, no solo al CAD.

Cambios v2.21 (31 de julio 2026): **parches con espesor** (prisma) y **reglas de superposición** entre objetos de la escena. Uso en §6.4 y §10.5. Seis ejes.

A. Parches como prisma, con espesor

El parche de absorción se dibuja como un **prisma** hacia el interior de la sala en vez de un plano pegado a la pared, con el espesor real del tratamiento (**10 cm** por defecto). Las **aristas** se resaltan para que el prisma se lea con cualquier color de relleno. Con espesor 0 se dibuja plano, como antes.

B. El espesor sale del material, y avisa si no coincide

Al elegir un material del catálogo, el espesor se **autocompleta** leyendo la construcción del nombre: suma lo que aporta profundidad (capa absorbente, cavidad, cámara de aire, descuelgue) y descarta la geometría en-plano (ancho de franjas, intervalos de un panel ranurado). Cubre **240 de los 429** materiales; el resto se queda con el default. Si el espesor dibujado no coincide con el del α elegido, el diálogo avisa. También muestra el $\lambda/4$ del espesor.

Por qué el espesor no entra al solver: el $\alpha(f)$ del catálogo ya se midió con el espesor de esa construcción (para una misma lana, el α a 63 Hz cambia $\sim 15\times$ entre 20 y 100 mm). Sumarlo como obstáculo sería contarlos dos veces, y correr la frontera hacia adentro sería peor: en la banda modal un panel de 10 cm es $\lambda/100$ y la onda lo atraviesa. Detalle en §10.5.

C. Fuente y receptor nunca adentro de un mueble

Un mueble se modela quitando el aire que ocupa. Si el punto de la fuente o del receptor caía en ese hueco, el campo modal evaluaba **NaN** y contaminaba toda la FRF **sin dar error**. Ahora se bloquea en los dos sentidos: no se puede arrastrar el punto adentro de un mueble, ni colocar un mueble encima del punto. El receptor estaba especialmente expuesto por ser un punto sin baffle que lo cubriera.

D. La regla de sólidos ahora vale en los dos sentidos

El manual ya decía que un mueble no puede ocupar el lugar del baffle de un parlante, pero la regla se aplicaba en un solo sentido: el mueble frenaba contra el parlante y el parlante atravesaba al mueble. Ahora el baffle también frena. En todos los casos se mantiene el **escape**: lo que ya está superpuesto se puede arrastrar para afuera.

E. Aviso de mueble tapando un parche

Si un mueble se superpone con el prisma de un parche, aparece un aviso. No se bloquea (el prisma es dibujo y el α sigue sobre la pared), pero la advertencia tiene contenido acústico real: un mueble delante del absorbente lo tapa y su α efectivo baja respecto del catálogo.

F. Fix: muebles y parches se mueven con el origen

Al cambiar la convención de origen (por ejemplo a “esquina inferior”) se trasladaban las fuentes, el receptor y los puntos de escucha, pero **los muebles y los parches se quedaban en el lugar viejo**. Es un arrastre histórico: el origen configurable es de v2.16 y los parches (v2.17) y muebles (v2.18) se agregaron después, sin sumarse a esa lista.

Cambios v2.22 (13 de agosto 2026): **empaquetado del .exe puesto al día**. Uso en §20 (sección nueva). Tres ejes.

A. Bundle 490 MB más liviano

build.bat excluye dependencias que Anaconda arrastra y el proyecto no usa: Qt5WebEngineCore (107 MB, un navegador embebido), panel (101 MB), botocore (92 MB, el SDK de AWS), llvm-lite/numba (66 MB) y bokeh. La carpeta bajó de **1524 a 1032 MB** y el ZIP de **570 a 414 MB**, sin perder ninguna función. Lo que queda es irreducible sin romper algo: los `mk*_*.dll` del BLAS de numpy/scipy (~370 MB, con variantes por tipo de CPU), gmsh (~86 MB) y PyQt5 (~81 MB).

B. El ZIP dejó de mentir sobre su versión

El nombre del paquete estaba escrito a mano como `Prototipo1_v2.12.zip` y quedó congelado **ocho versiones**: al destinatario le llegaba un archivo que decía v2.12 conteniendo v2.21. Ahora `pack_distribution.py` lee la versión del changelog de este manual, que es la única fuente de verdad.

C. Sección §20 nueva + verificador al día

Se documenta el flujo completo (compilar → verificar → smoke test → prueba visual → empaquetar), el desglose del tamaño y —lo más importante— **por qué el ejecutable no va al repositorio** y cuál es la vía correcta (GitHub Releases). El rango de tamaño de `verify_distribution.py` estaba en 100-800 MB, o sea que marcaba FAIL en un bundle correcto; ahora es 700-1400 MB, con la pista de qué revisar si se dispara.

Además se agregó `--collect-all` para trimesh y gmsh en build.bat. No corregía un bug (PyInstaller ya los detectaba por análisis estático), pero los dos resuelven cosas en runtime que ese análisis no ve, así que queda como seguro explícito del feature de CAD.

D. Fix crítico: el .exe salía sin las DLLs de Qt

El ejecutable se generaba, el build decía “BUILD OK”, y al abrirlo moría con `DLL load failed while importing QtCore`. Anaconda no guarda las DLLs de Qt junto a PyQt5 sino en `<entorno>\Library\bin\` y con otro nombre (`Qt5Core_conda.dll`); PyInstaller las busca recorriendo el PATH, así que compilando desde una consola sin el entorno conda activado **las omitía en silencio**. build.bat ahora arma ese PATH por su cuenta, así que el build es reproducible desde cualquier consola.

Los dos chequeos automáticos tampoco lo detectaban, y quedaron corregidos:

- `verify_distribution.py` solo miraba que existiera la **carpeta** `PyQt5/`, que existía (con los `.pyd`); lo que faltaba eran las DLLs. Ahora exige `Qt5Core`, `Qt5Gui` y `Qt5Widgets`.
- `test_distribution_smoke.py` daba OK con “el proceso sigue vivo 15 s”, pero **un diálogo de excepción también es un proceso vivo**. Ahora lee el título de las ventanas del proceso y falla si encuentra una de error.

Cambios v2.23 (19 de agosto 2026): **frecuencia de Schroeder coherente, polaridad de fuentes, amortiguamiento por perturbación de frontera** (modelo nuevo) y **fixes de una auditoría de la simulación**. Seis ejes.

A. La frecuencia de Schroeder dejó de contradecirse a sí misma

Había **dos** cálculos de la frecuencia de Schroeder que no se hablaban: el que muestra el panel (a partir del RT60 real de los materiales) y el que usa el auto-tuner de malla por dentro, que estaba clavado en un $\alpha=0.05$ fijo. Para la misma sala, en la misma sesión, la app decía dos números distintos y podían diferir hasta el doble. Como $f_{\text{Schroeder}} \propto \alpha^{(-1/2)}$, el error iba para los dos lados: una sala tratada mallaba el doble de fino de lo necesario (8x nodos al pedo), y una sala viva mallaba **sin cubrir el régimen modal, en silencio**. Ahora los dos salen del mismo lugar.

Y cuando **ninguna** cara tiene material asignado, aparece una ventana que pregunta de dónde sale la absorción (un coeficiente α uniforme, un preset de sala, o un material del catálogo para todas las caras), en vez de usar por lo bajo el material que quedaba por defecto. La elección se recuerda por la sesión y se muestra en un renglón bajo la frecuencia de Schroeder, que se actualiza cada vez que cambian los materiales.

B. Densidad de malla hasta 30 y cobertura de modos honesta

El tope del control **Densidad voxel** subió de 10 a **30**. El límite físico no es " $\alpha = 0$ " (que da una malla infinita: sin absorción el tiempo de reverberación diverge) sino la superficie más reflectante del catálogo ($\alpha = 0.01$), que en la sala más chica pide una densidad de ~ 28 . El tope de 30 la cubre.

Además, cuando el presupuesto de modos no alcanza para llegar a la frecuencia de Schroeder (típico en salas vivas, donde harían falta miles de modos), el auto-tuner lo dice: informa hasta qué frecuencia llega la cobertura **real** con los modos pedidos, en vez de mallar para una banda que no se va a calcular. La leyenda de modos por Weyl también dejó de aconsejar "refinó la malla" cuando el cuello de botella es la cantidad de modos, no la resolución.

C. Polaridad de fuentes (0° / 180°)

El editor de fuente tiene un interruptor **Polaridad: Invertida (180°)**. Invertir una fuente multiplica su aporte por -1 (la da vuelta en contrafase). Es una propiedad del cableado, **independiente** de la curva de respuesta (FRD/TRF) cargada: invertir la polaridad ya **no borra** la medición, se componen. El estado se lee de vuelta al reabrir el editor y en la lista de fuentes (etiqueta [180°]), y se guarda en el .room.

Con **una sola** fuente, invertir la polaridad no cambia el mapa de presión, porque el visor muestra la magnitud $|p|$ y dar vuelta el signo no la altera (es física, no una falla). El efecto se ve con **dos o más** fuentes, donde cambia la interferencia: un par en contrafase produce cancelaciones profundas. También aparece en la fase exportada de la FRF.

D. Amortiguamiento por perturbación de frontera (modelo nuevo, seleccionable)

En el grupo **Materiales** hay un selector **Amortiguamiento** con dos modelos de cómo la absorción se convierte en el amortiguamiento ξ de cada modo:

- **Sabine por modo (A36)** — el de siempre: parte del RT60 de Sabine ponderado por la forma modal. Con material uniforme da el mismo tiempo de reverberación para todos los modos.
- **Perturbación de frontera** — deriva ξ directamente de la admitancia de la pared y la integral de superficie de cada modo, **sin pasar por el RT60**. Con material uniforme **no** da

lo mismo para todos los modos: captura que un modo axial (que golpea pocas paredes) se apaga más lento que uno oblicuo (que las golpea todas), un efecto real que Sabine no puede ver.

Es teoría de perturbaciones de primer orden sobre los modos de pared rígida (Morse & Ingard, *Theoretical Acoustics*, Ec. 9.4.14; Kuttruff, *Room Acoustics*, Ec. 3.34). Se validó contra el problema de impedancia exacto: coincide a mejor del 1 % hasta absorciones medias ($\alpha \approx 0.3$) y ~4 % en absorciones altas, mientras que Sabine se aparta hasta un ± 18 % del promedio de los modos. Un resultado ordenador: **Sabine resulta ser el caso límite de campo difuso de la perturbación** (el modo oblicuo, “todas las direcciones”). El coeficiente de absorción del catálogo (incidencia aleatoria) se convierte a admitancia con la fórmula de Paris.

El modelo por defecto sigue siendo Sabine (A36): la perturbación se activa a propósito y no cambia ningún resultado sin que se lo pida. Se guarda en el .room.

E. Auditoría de la simulación: dos números que estaban mal en salas no rectangulares

- **Uniformidad espacial (FoM) falsa fuera de la caja.** La grilla de receptores con la que se calcula la figura de mérito de uniformidad salía de la caja envolvente de la sala, así que en una planta no rectangular (pentágono, hexágono afinado, planta en L) parte de los puntos caía **fuera** del recinto y entraba al promedio con presión cero, que en decibeles es un valor enorme por lo bajo. Resultado: la uniformidad espacial se disparaba a ~70-98 dB cuando el valor real era ~5 dB. Ahora esos puntos se descartan (y la grilla se densifica para no perder tamaño de muestra), y el cálculo avisa en vez de rellenar con ceros. En una caja rectangular no cambia nada.
- **Absorción de paredes oblicuas subestimada.** Por el mismo motivo (puntos de la superficie que no caían en la malla escalonada), el amortiguamiento perdía área de las paredes inclinadas en silencio. Ahora se integra solo sobre la superficie efectivamente muestreada y se re-escala por su cobertura.

F. Correcciones menores

El campo de presión **se actualiza al editar una fuente** (antes solo se recomputaba al moverla, así que cambiar la polaridad o la curva por el diálogo dejaba el mapa viejo en pantalla). El renglón que informa de dónde sale la absorción sigue los cambios de material en vez de quedar congelado.

Cambios v2.24 (20 de agosto 2026): **RT60 efectivo por banda desde la perturbación** (Etapas 2a). Cuando el modelo de amortiguamiento es **Perturbación de frontera** y hay modos resueltos, el tiempo de reverberación de la **banda modal** (por debajo de la frecuencia de Schroeder) ya no sale de Sabine sino del **decaimiento real de los modos**, y con eso se alimentan los números escalares que dependían de él.

RT60 efectivo: T30 del decaimiento modal, no la media de las tasas

Cada modo tiene su propio amortiguamiento ξ_n (por eso existe el modelo de perturbación). Para colapsar ese conjunto en un RT60 por banda no se promedian las tasas: un conjunto de modos con decaimientos distintos **no** decae como una sola exponencial. Se arma la curva de energía de la banda como la suma de las exponenciales de sus modos, se integra a la Schroeder y se le mide la pendiente **T30** (–5 a –35 dB), que es la definición de norma (ISO 3382) del tiempo de reverberación. La media de tasas $6.91/\langle\delta\rangle$ queda ~10 % por debajo de ese T30, porque la cola del decaimiento la manda el modo **menos amortiguado** (el axial). Medido en una sala de 5×4×3 m, la banda de 32 Hz suena **~40 % más** de lo que dice Sabine: son los modos axiales bajos ringueando, un efecto que Sabine (plano en α) no puede mostrar.

Por encima del modo más alto (régimen difuso) el RT60 lo sigue dando Sabine, que ahí sí es válido. El cruce entre ambos es justo la frontera física entre el régimen modal y el difuso.

Qué cambia en pantalla (solo con el modelo de perturbación activo)

- **RT60 medio** (grupo Materiales): con perturbación agrega la nota «*T30 perturbación (banda modal)*» y refleja el decaimiento por modo. Con Sabine (por defecto) no cambia ni un dígito.
- **Ver RT60 calculado**: botón nuevo «+ **Perturbación (T30, banda modal)**» que superpone la curva T30 para compararla contra Sabine/Eyring. Requiere modos resueltos.
- **Cruce de solapamiento modal (f_{cross})**: el ancho de media potencia $B_{\text{HP}} = 2.2/\text{RT60}$ usa el RT60 de la perturbación en la banda modal, así que f_{cross} refleja el amortiguamiento por modo.

Frecuencia de Schroeder de dos pasadas (Etapa 2b)

La frecuencia de Schroeder tiene un problema de orden: se necesita **antes** de resolver los modos, para dimensionar la malla, pero el amortiguamiento por modo solo existe **después** de resolverlos. Se resuelve en dos pasadas:

- **Antes de resolver (dimensionar la malla)**: $f_{\text{Schroeder}}$ se estima con Sabine. Es un estimador de tamaño de malla, no un resultado físico, así que Sabine alcanza.
- **Después de resolver (con el modelo de perturbación)**: $f_{\text{Schroeder}}$ se recalcula con el T30 por banda. Como los graves suenan más largo que lo que dice Sabine, $f_{\text{Schroeder}}$ sube. El programa refresca el valor y, si la malla quedó corta para cubrir esa banda nueva, **avisa** cuánto subir la densidad y volver a resolver (no re-malla solo). Una segunda resolución ya usa este $f_{\text{Schroeder}}$ refinado.

Predicción de ubicación con amortiguamiento por modo (Etapa 2c)

La predicción de **ubicación** (colocar las fuentes en un recinto fijo) resuelve el FEM del recinto real, así que ahí también hay modos. Con el modelo de perturbación activo y materiales por superficie asignados, el optimizador de ubicación usa el amortiguamiento **por modo** de la perturbación en lugar del ξ uniforme (un solo RT para todos los modos). Esto hace que las figuras de mérito con las que se rankean las ubicaciones vean el mismo amortiguamiento selectivo que la pestaña Acústica. Sin materiales por superficie (o con el modelo Sabine), sigue con el ξ uniforme de siempre.

La predicción de **geometría** (que genera y compara muchas salas candidatas) sigue con Sabine: no tiene sentido resolver el FEM completo de cada candidato solo para el ranking de formas.

La perturbación es ahora el modelo por defecto (Etapa 3)

Con la frecuencia de Schroeder, el cruce modal, el RT60 y la predicción de ubicación ya coherentes con el modelo elegido, la perturbación de frontera pasa a ser el **modelo por defecto** de una sesión nueva: es más exacta que Sabine por debajo de Schroeder (validada a mejor del 1 % contra el problema de impedancia exacto hasta $\alpha \approx 0.3$), que es el objetivo del programa. Sabine sigue disponible en el combo (es el límite de campo difuso de la perturbación).

Compatibilidad hacia atrás: un archivo .room guardado antes de esta versión (sin el dato del modelo) se abre en **Sabine**, para conservar exactamente los números con los que se guardó. Un .room nuevo guarda su modelo real, así que reabrirlo lo restaura tal cual.

La absorción es un dato consciente, no un default silencioso (Opción C)

Un número físico no debería salir de una absorción que no elegiste. Ahora:

- **Calcular los modos NO exige absorción.** Las frecuencias y formas modales son de pared rígida, no dependen del material, así que se resuelven igual. Si no elegiste absorción, aparece un aviso: los modos son válidos, la malla se dimensiona con un $\alpha = 0.05$ conservador, y la frecuencia de Schroeder, el RT60 y la FRF no se muestran hasta que asignes materiales o un α .
- **Los números que dependen del material quedan en «— asigná absorción»** hasta que elijas: el RT60 medio y la frecuencia de Schroeder.
- **Al pedir un número dependiente del material** (Calcular $f_{\text{Schroeder}}$, Calcular FRF) se abre el diálogo de absorción. Si lo cancelás, la absorción queda sin elegir (ya no se asume un $\alpha = 0.05$ por lo bajo).

Nº de modos necesario para cubrir hasta $f_{\text{Schroeder}}$

Al **Calcular $f_{\text{Schroeder}}$** , el campo «Nº modos» se **auto-carga** con la cantidad de modos necesaria para cubrir hasta esa frecuencia (densidad de Weyl), topeada en 500. Así la primera corrida ya llena todas las bandas por debajo de $f_{\text{Schroeder}}$ (antes, con pocos modos, la curva de RT de la perturbación mostraba una sola banda). Si la sala pide más de 500 modos (salas muy vivas), lo deja en el tope y avisa que cubrir $f_{\text{Schroeder}}$ por completo no es alcanzable ahí. Si todavía no elegiste absorción, no se puede calcular $f_{\text{Schroeder}}$ y sale el aviso correspondiente.

Cambios v2.25 (25 de agosto 2026): **modo Rotar** (para mouse sin rueda), **delay y fase de fuentes que se aplican solos y se recuerdan**, y **arreglos de arranque al correr desde código en macOS**. Tres ejes.

Modo Rotar: girar la vista y los objetos sin botón central

Pensado para mouses sin rueda ni botón central (por ejemplo el Magic Mouse de Mac), que no podían orbitar la vista. Arriba a la izquierda del visor 3D hay un botón «**⤵ Rotar**»; se activa con el botón o con la tecla **1** (y se sale con el botón, la tecla **1** o **Esc**). Con el modo activo, **arrastrar con el botón izquierdo** orbita la vista si el cursor está sobre espacio vacío, o rota la fuente/mueble que esté debajo del cursor. Un cartel «**MODO ROTAR**» avisa que está activo. El zoom sigue siendo con el scroll (dos dedos en el trackpad/Magic Mouse). La tecla **1** no interfiere con la escritura: si estás editando un campo numérico, escribe el 1 normalmente.

Delay y fase de fuentes: se aplican al Aceptar y se recuerdan

Antes había que apretar un botón «Aplicar» para que el delay y la fase de una fuente tomaran efecto, y al reabrir la fuente los valores aparecían en cero aunque los hubieras cargado. Ahora el **delay** y la **fase** son propiedades de la fuente (como la polaridad): se aplican solos al dar **Aceptar** (ya no hay botón «Aplicar»), al reabrir la fuente **muestran el valor que le pusiste**, se guardan en el .room, y el preview de fase se actualiza en vivo mientras editás. Internamente se **componen** con la respuesta medida (FRD) y la polaridad en vez de pisarlas. Con delay 0 y fase 0 el comportamiento es idéntico al de siempre. Compatibilidad hacia atrás: un .room viejo abre con delay y fase en cero.

Arranque al correr desde código en macOS

El paquete «correr desde fuente» de Mac instala las librerías frescas, y las versiones nuevas de NumPy y Qt destaparon tres cortes de arranque que ya están resueltos: la incompatibilidad con **NumPy 2** (funciones renombradas), la **ruta del plugin de plataforma de Qt** (que impedía abrir la ventana en Mac), y unos **avisos de consola** al calcular (una hoja de estilo mal formada que el Qt nuevo rechazaba, más una fuente inexistente en Mac). Nada de esto afecta al .exe de Windows, que empaqueta sus propias librerías.

Cambios v2.26 (26 de agosto 2026): **SBIR con la transferencia modal de la sala y guardar/cargar curvas de RT en CSV**. Dos ejes.

SBIR con la transferencia modal de la sala

El SBIR clásico mira solo la interferencia entre el sonido directo del parlante y sus reflexiones de primer orden, tratando cada camino como campo libre; no ve los modos de la sala. La ventana de SBIR ahora tiene una casilla «**Incluir transferencia modal de la sala (híbrido en f_Schroeder)**» que agrega el efecto del recinto. Con la casilla activa se dibujan, sobre la misma referencia (0 dB = anecoico), tres curvas: el **SBIR** de imágenes de siempre, la **transferencia modal** de la sala (la respuesta modal FEM entre parlante y receptor, con sus resonancias), y una curva **Total híbrido** que combina las dos en su régimen de validez: usa la respuesta modal por **debajo de la frecuencia de Schroeder** (donde la solución modal es exacta y ya contiene las reflexiones de esas frecuencias) y el peine de imágenes por **encima** (donde la densidad modal es alta y el FEM truncado deja de ser confiable), con un cruce suave alrededor de $f_{\text{Schroeder}}$ marcada con una línea vertical. Requiere tener los modos resueltos (si no, la casilla no aparece). El export a CSV incluye las columnas de la curva modal y de la híbrida.

Guardar y cargar curvas de RT para comparar configuraciones

La ventana «**Ver RT60 calculado**» se limpia al cerrarla (cómodo para trabajar rápido), y ahora tiene un grupo «**Guardar / cargar curvas (CSV)**» para comparar tratamientos sin perder el trabajo. Con «**Guardar curva seleccionada...**» elegís una curva de la lista (Sabine, Eyring o Perturbación; T60 o T30) y la guardás en un archivo **CSV** eligiendo carpeta y nombre; el archivo es legible (se abre en Excel o cualquier editor) y guarda el método, la métrica y la tabla banda-por-banda. Después cambiás los revestimientos, reabrís la ventana, y con «**Cargar curva(s) desde CSV...**» traés uno o varios de esos CSV, que se superponen punteados para comparar contra la configuración actual. Como cada curva es su propio archivo, podés acumular todas las variantes que quieras.

Cambios v2.27 (26 de agosto 2026): **construcciones de pared** (impedancia por superficie). Un eje.

Construcciones de pared: modelar la impedancia de cada superficie

Hasta ahora la absorción de una superficie se resumía en su coeficiente α (medido con incidencia aleatoria, ISO 354). Eso alcanza para el tiempo de reverberación, pero en la banda modal (por debajo de la frecuencia de Schroeder) una pared real hace dos cosas que el α solo no captura: **amortigua** cada modo según cuánto absorbe en esa frecuencia, y **corre la frecuencia** del modo según la reactancia de la construcción (una cámara de aire, la masa de un panel, el resorte de un resonador). Una pared perfectamente rígida no corre nada; una construcción resonante sí, y ese corrimiento es audible.

El botón «**Construcciones de pared...**» (en el grupo de Materiales) abre una ventana donde asignás una **construcción** a una o varias superficies: **paredes, parches y muebles**, todo en la misma lista. Con «**Nueva construcción y asignar...**» se abre un editor con cuatro tipos:

- **Panel perforado**: una placa con orificios sobre una cámara de aire (resonador de Helmholtz distribuido). Parámetros: espesor de la placa, diámetro del orificio, porcentaje de perforación y profundidad de la cámara.
- **Microperforado (MPP)**: lo mismo con orificios de menos de 1 mm, que dan absorción de banda ancha sin material poroso (modelo de Maa).

- **Membrana / panel:** una placa impermeable que vibra sobre una cámara (resonador masa-resorte), típico para graves. Parámetros: masa por metro cuadrado, profundidad de la cámara y pérdidas internas.
- **Poroso + cámara:** un material poroso (lana, espuma) sobre una cámara de aire opcional, con los modelos Miki, Delany-Bazley o JCA.

Mientras editás, el panel de la derecha muestra en vivo la curva de **absorción** de esa construcción y su **frecuencia de resonancia**, así ves de una qué controla. Al aceptar, la superficie queda con esa construcción (en azul en la lista) y el cálculo de los modos usa su impedancia: el amortiguamiento por banda **más** el corrimiento de las frecuencias modales, que se ve reflejado en la respuesta en frecuencia. Las superficies sin construcción siguen usando el α de su material como siempre, así que un proyecto que no toca esto no cambia en nada.

Las construcciones **solo actúan con el modelo de amortiguamiento «Perturbación de frontera»** (el que deriva el amortiguamiento de la admitancia de la pared); con Sabine se avisa. Se guardan en el archivo .room. Nota: en esta versión la impedancia se evalúa a incidencia normal (pared de reacción local), que es el supuesto habitual para paneles perforados y membranas.

Cambios v2.28 (27 de agosto 2026): **Capa 0 a la vista (Δf_n y ξ_n por modo), un acabado por superficie** (material o construcción, no los dos), **crear material propio por tercios de octava**, y **el npm que escribís se respeta**. Cuatro ejes.

Cada modo, con su frecuencia corrida y su amortiguamiento a la vista

Cuando una construcción de pared corre las frecuencias modales (sección anterior), ese corrimiento ya se veía en la respuesta en frecuencia, pero no como número. Ahora es explícito. En el grupo de **Modos** (pestaña Acústica):

- Debajo del selector de modo hay una **línea de lectura** del modo elegido: su frecuencia rígida, la frecuencia efectiva a la que resuena de verdad (corrida por la construcción), el corrimiento Δf_n entre ambas, el amortiguamiento modal ξ_n y el **RT60 de ese modo aislado**. Sin construcciones, $\Delta f_n = 0$ (la pared no aporta reactancia, solo absorbe).
- El propio **selector de modo** anota el corrimiento al lado de cada entrada cuando lo hay (por ejemplo 2: $f = 75.70 \text{ Hz } (\Delta -1.30)$), así se ve de un vistazo qué modos se movieron y cuánto.
- El botón **«Ver modos (Δf_n , ξ_n)...»** abre una tabla con **todos** los modos: n , frecuencia rígida, frecuencia efectiva, Δf_n , ξ_n y $RT60_n$. La tabla se exporta a **CSV, TXT o PNG** para llevarla a un informe.

La frecuencia efectiva es la que usan la respuesta en frecuencia, el mapa de campo (slice y heatmap 2D, alineados en esta versión) y las figuras de mérito; la *forma* del modo no cambia (es una perturbación de primer orden). El $RT60$ del modo aislado sale de su amortiguamiento: $RT60_n = 6.908/(\xi_n \cdot 2\pi \cdot f)$. Sentido físico del corrimiento: por debajo de la resonancia de la construcción actúa como rigidez (resorte) y la frecuencia **sube**; por encima actúa como masa y **baja**; en la resonancia cruza por cero. Nada de esto altera el cálculo: es la Capa 0 (el modelado de impedancia) hecha visible.

Un acabado acústico por superficie

Cada superficie (cara, parche o mueble) tiene **un solo** acabado: o el α de un material de catálogo, o una construcción de pared (impedancia Z). Antes se podían cargar los dos sobre la misma superficie y el programa calculaba con una definición pisando a la otra sin avisar. Ahora:

- Si una superficie tiene una construcción, en «Materiales...» aparece **«→ definido por construcción»** (bloqueada): su impedancia reemplaza al α . Para volver a un material, se quita la construcción en «Construcciones de pared...».

- Si dibujás un parche con material sobre una cara que tiene construcción (o al revés), el programa **avisa** y te deja elegir: que el parche **herede** la construcción de la cara, o que **mantenga** su material (override local explícito).

Crear un material propio sin escribir JSON

En «Materiales...», el botón «**Crear material...**» abre un formulario con una casilla de absorción **por tercio de octava** (50–5000 Hz): completás las bandas que mediste, le ponés **nombre** y **notas**, y **Guardás**. Queda en la biblioteca (materials/) y disponible en Acústica y Predicción. Las bandas que dejes vacías se interpolan; el modelo usa tu resolución de tercios tal cual la cargaste (no la colapsa a octava), así una medición propia se respeta con su detalle.

El npm que escribís se respeta

Con el motor de malla en «Automático», el auto-tuner sugiere una densidad de voxel (npm) que cubre hasta la frecuencia de Schroeder. Antes, al calcular los modos, ese valor **pisaba** el que hubieras escrito a mano. Ahora tu npm es un **piso**: si pedís **más** densidad que la recomendada (malla más fina, más válida en frecuencia, p. ej. para dar validez a más modos), se respeta; el auto-tuner solo lo **sube** cuando tu valor no alcanza a cubrir $f_{\text{Schroeder}}$.

Cambios v2.29 (31 de agosto 2026): **cargar parlantes en formato CLF, filtro de crossover/EQ por fuente** (Butterworth, Linkwitz-Riley, Bessel, Chebyshev, elíptico), **todos los diálogos en fondo blanco/letra negra**, y **la absorción se comparte entre Acústica y Predicción**. Cuatro ejes.

Cargar la respuesta de un parlante en formato CLF

El botón de respuesta de fuente ahora es «**Cargar FRD/TRF/CLF...**» y acepta archivos **CLF** binarios (.cf2, .cf1), el estándar de datos de parlantes (Common Loudspeaker Format). Al cargar uno, el programa extrae la **respuesta en eje** (sensibilidad SPL @ 1W/1m, por tercios de octava de 50 Hz a 20 kHz) y la usa como la curva Q(f) de la fuente, igual que un FRD.

Lo que **no** se usa del CLF es el globo de directividad, y es a propósito: por debajo de la frecuencia de Schroeder la fuente es acústicamente omnidireccional (la longitud de onda es enorme comparada con el parlante), así que la directividad no moldea el campo modal. Meterla sería precisión falsa. El programa te avisa de esto al cargar el archivo. El lector se validó contra los valores que muestra el CLF Viewer para el mismo parlante (coincidencia exacta).

Un filtro por fuente (crossover / EQ)

Cada fuente tiene ahora un grupo «**Filtro (crossover / EQ)**» en su diálogo. Elegís la **familia** de filtro, la **banda** (pasabajos o pasaaltos), el **orden** y la **frecuencia de corte**; para las familias que lo usan, aparecen además el ripple de banda de paso y la atenuación de rechazo. Familias disponibles (todas las de uso profesional en audio):

- **Butterworth**: máxima planitud en la banda de paso; -3 dB en la frecuencia de corte.
- **Linkwitz-Riley** (LR2/LR4/LR8): el estándar de crossovers; -6 dB en el corte (dos Butterworth en cascada), pasa-bajo y pasa-alto suman en fase.
- **Bessel**: retardo de grupo plano (fase casi lineal), útil cuando importa la forma temporal.
- **Chebyshev I / II**: corte más abrupto a cambio de ripple (en la banda de paso o en la de rechazo).
- **Elíptico (Cauer)**: el corte más abrupto para un orden dado, con ripple en ambas bandas.

El filtro se **compone** sobre la curva del parlante (magnitud y fase), la polaridad y el delay; con «Sin filtro» el resultado es idéntico a no tener filtro. El preview del diálogo muestra el filtro en vivo. El filtro se guarda en el .room y viaja al duplicar la fuente.

Todos los diálogos en fondo blanco, letra negra

Las ventanas de diálogo (materiales, construcciones, FRF, SBIR, tabla de modos, editor de fuente, etc.) y los avisos emergentes pasaron a **fondo blanco con letra negra**, coherente con los gráficos (que ya eran claros). La ventana principal y los paneles laterales siguen con el tema oscuro. Es un cambio visual, no afecta ningún cálculo.

La absorción elegida se comparte entre Acústica y Predicción

Antes, si definías la absorción de las superficies en Acústica (por ejemplo un α uniforme en el gate de Schroeder) y después ibas a Predicción, Predicción te la volvía a pedir: eran dos decisiones separadas. Ahora se **comunican en los dos sentidos**: Predicción **hereda** la absorción de Acústica (y te avisa que lo hizo) en vez de preguntar de nuevo, y si la cambiás en cualquiera de los dos paneles, el otro la **adapta**. El α uniforme se sincroniza como número; los materiales se mapean a piso/paredes/techo (mismo modelo de tres zonas en los dos paneles), así que cambiar el material en Predicción reasigna también los de Acústica. La última elección que hagas manda en ambos paneles.

Cambios v2.30 (2 de septiembre 2026): **modelo de fuente exacto para subs enfrentados (DBA/CABS)** — una herramienta nueva para diseñar y analizar subs enfrentados y aplicarlos a la sala, un **modelo físico de driver (Thiele-Small)** por fuente, y el **lector CLF generalizado**. Tres ejes.

Subs enfrentados (DBA / CABS)

Motivación: la fuente del simulador era un **monopolo puntual**, y con eso los **subs enfrentados** (Double Bass Array / Controlled Acoustic Bass System) no se simulaban bien. El DBA/CABS funciona porque un array frontal lanza una **onda plana** y un array trasero la **absorbe**, dejando la sala sin la onda estacionaria en el grave. Eso necesita una **fente distribuida** sobre la pared (una integral de superficie, no un valor puntual), que es lo que agrega esta versión. El respaldo físico es Kuttruff (*Room Acoustics* §3.6, la función de Green modal con excitación por velocidad de superficie) y la implementación de referencia de Santillán (JASA 2001) y Nielsen & Celestinos (CABS).

En la zona **FRF** del panel hay un botón nuevo «**Subs enfrentados (DBA / CABS)...**». Abre una herramienta que trabaja sobre la **caja rectangular** de la sala (los subs enfrentados están definidos para cuartos rectangulares). Elegís:

- el **eje** de enfrentamiento (por defecto el más largo de la sala),
- cuántos **subs por pared** en cada dirección transversal (una grilla $n \times n$),
- el **drive** del array trasero: **mínimos cuadrados (Santillán)**, el óptimo, o **retardo + inversión (naive)**, la versión clásica,
- el **amortiguamiento** ξ y la frecuencia máxima del análisis.

Al **Calcular** ves la **FRF en el receptor** antes (CABS off, solo el frente) vs después (CABS on, frente + trasero), y tres métricas de colapso: **planitud espectral**, **varianza espacial** y decay. Un dato clave que se muestra en vivo es **$f_{\max} = c/d$** (con d el espaciado entre subs): el DBA **solo ecualiza hasta $f_{\max}$** ; por encima hay aliasing espacial y deja de servir (esa zona sale sombreada en el gráfico). Más subs → menor espaciado → mayor $f_{\max}$. Por eso las métricas se miden **solo en la banda válida** [$f_{\min}$, $f_{\max}$].

El botón «**Aplicar a la sala**» materializa el diseño como **fuentes reales** en la lista (etiquetadas DBA-F* al frente y DBA-R* atrás), con su drive: en modo naive el trasero lleva el retardo L/c y la

polaridad invertida; en modo LS cada fuente lleva su curva de drive $q(f)$. A partir de ahí el **resto de la app** (FRF, campo 3D, comparar puntos de escucha, escuchar) ve el DBA. Si tenés otras fuentes activas, te ofrece **mutedarlas** para que el A/B sea limpio (medir solo el DBA, sin sumar el sistema anterior).

Cómo testear el efecto: (1) con tus 2 subs normales, calculá y exportá la FRF; (2) diseñá el DBA hasta que la banda válida cubra lo que te importa; (3) aplicalo y aceptá mutedar las otras; (4) volvé a calcular la FRF (con $f_{\text{máx}} \leq f_{\text{max}}$) y compará: más plana en la banda válida; (5) usá «Comparar puntos de escucha» en varios asientos para ver la **uniformidad espacial**, que es lo que un sub común no da. Nota: la herramienta usa un modelo analítico rectangular exacto y la FRF principal usa FEM, así que la **tendencia** coincide pero los números no son idénticos.

Modelo físico de driver (Thiele-Small)

El diálogo de cada fuente tiene ahora un grupo «**Driver físico (Thiele-Small)**». En vez de una curva plana o medida, podés derivar la respuesta $Q(f)$ de la **física del parlante** (caja sellada): ingresás **fc + Qtc**, o los parámetros crudos **fs, Qts, Vas + Vb** (volumen de caja), y «Aplicar como curva $Q(f)$ » genera la respuesta (pasa-altos de 2º orden con la fase correcta) y la usa como la curva de la fuente, igual que un FRD. El nivel lo pone la sensibilidad; la forma (rolloff bajo fc), el driver.

Lector CLF generalizado

El lector de archivos **CLF** (.cf2/.cf1) ya no depende de una posición fija dentro del archivo: ahora **ancla en la estructura** (la corrida de tensión de referencia que precede a la respuesta en eje, invariante a la impedancia del parlante) y **detecta la versión** del formato. Sigue extrayendo solo la respuesta en eje (la directividad se descarta por diseño bajo Schroeder). Es más robusto para archivos de otros exportadores, aunque se validó sobre exports de EASE.

Cambios v2.31 (3 de septiembre 2026): **impedancia por default en cada material**. Hasta ahora el corrimiento de las frecuencias modales (la reactancia de la Capa 0) solo aparecía si asignabas una **construcción de pared** a mano. Ahora **cada material trae su propia impedancia $Z(f)$ por default**, así el efecto reactivo se ve sin cargar nada. Un eje.

Impedancia por default en cada material

El modelo de perturbación de frontera usa la admitancia de pared $\beta = \rho_0 c / Z$ para dos cosas: la **parte real** amortigua (mueve el RT60) y la **parte imaginaria** (reactancia) **corre la frecuencia** de cada modo. Antes, un material del catálogo solo aportaba amortiguamiento (β real, sin corrimiento); la reactancia requería asignar una construcción explícita. Ahora la app le sintetiza a cada material una $Z(f)$ por default:

- El **amortiguamiento** sigue saliendo **exacto** del α medido del catálogo (no cambia ni un dígito respecto de antes: la absorción medida es sagrada).
- La **reactancia** se le injerta **solo a los materiales porosos**. La app ajusta la **resistividad al flujo σ** (Pa·s/m²) de un poroso equivalente cuya absorción reproduce el α del material (modelo de Miki), y toma de ahí la parte imaginaria de la impedancia. Los materiales **duros o resonantes** (hormigón, vidrio, panel perforado) **no** reciben reactancia inventada: quedan con β real, como antes.

El criterio de “poroso o no” lo decide la **forma del α** , no el nombre: si la absorción es alta y con forma porosa, entra; si es baja o resonante, no. Sobre el catálogo, alrededor del 40 % de los materiales reciben reactancia (alfombras, cortinas, espumas, lanas), y las superficies rígidas/vidrios/perforados quedan afuera.

Se ve en dos lugares:

- En «**Ver modos (Δf_n , ξ_n)...**»: con una sala alfombrada y **sin ninguna construcción**, la columna Δf_n ya trae corrimiento (del orden de +1 a +2 % con alfombra pesada; el signo es positivo, la reactancia porosa a baja frecuencia sube las f_n). El encabezado de la tabla aclara si el corrimiento viene de construcciones o de la Z por default de los materiales.
- En «**Construcciones de pared...**»: cada cara sin construcción ahora muestra, en gris, **su material y su Z por default** (por ejemplo «*Alfombra fina · Z auto (poroso equiv., resistividad $\sigma \approx 1.5e6 \text{ Pa}\cdot\text{s}/\text{m}^2$)*» o «*Hormigón · β real, sin reactancia*»), y se actualiza al cambiar el material. La construcción explícita, si la asignás, sigue pisando la Z del material.

Sobre el σ que aparece: es la **resistividad al flujo** del poroso equivalente (cuánto cuesta empujar aire por el material, ISO 9053), ajustada para reproducir el α de catálogo. Es una cantidad de modelo, no una medición del material puntual; la reactancia que produce es la que tendría un absorbente poroso con esa absorción.

Manual actualizado al 3 de Septiembre de 2026 — v2.31.